

Contents

pyqula user guide	3
Contents	4
Setting up a Hamiltonian	4
Including second and third neighbor hopping	5
Including an onsite energy	5
Including an external Zeeman field	6
Including an external orbital field	7
Setting a filling	7
Observables	7
Electronic band structures	7
Density of states	8
Local density of states	9
Momentum resolved spectral functions	11
Fermi surfaces	11
Quasiparticle interference	12
Spin splitting of an altermagnet	14
Operators	16
Nonlinear spin current as a measurement of altermagnetic order	16
Spin operators	19
Listing the available operators	19
Listing the other names pyqula accepts	19
Location operator	20
Bulk-edge operator	20
Valley operator	20
In-plane valley operators	21
Nambu operators	22
Berry curvature operator	22
Inverse participation ratio operator	22
Non-Hermitian Hamiltonians	23
Choosing which part is the energy: eigmode	24
What is and is not available	24
Superconductivity	25
s-wave superconductivity	26
Spin-triplet d-vector and non-unitary superconductivity	27
Superfluid weight and BKT temperature	28
Interactions at the mean-field level	29
The collinear Hubbard model	29
Non-collinear Hubbard model	31
Superconducting mean-field	31

Long range interactions	32
Spin-spin exchange interactions	33
Abrikosov-pseudofermion (spinon) mean field for Heisenberg models	37
Abrikosov-pseudofermion (Read-Newns) mean field for the Kondo lattice	39
Spatially resolved density of states	41
Electronic structure folding and unfolding	41
Surface spectral functions	42
Twisted bilayer graphene structural relaxation	43
Topological insulators	43
Topological invariants	44
Quantum geometric tensor (multiorbital/multiband)	45
Berry curvature density in frequency space	46
Berry curvature density in real-space	47
Chern number in real-space	47
Topological surface states	48
Topological markers	48
Entanglement	49
Entanglement entropy and entanglement spectrum	49
Response functions	50
Optical conductivity	50
Charge-charge response function	51
Generic operator-operator response function	52
RKKY response function	52
Spin susceptibility and RPA	53
Excitons and the Bethe-Salpeter equation	62
Quantum transport	70
Magnetoresistance in metal-metal transport	70
Superconductor-metal transport	71
Transport through an arbitrary finite region	71
Landauer transmission through a real-space device	72
Multiple Andreev reflection and AC-Josephson current	73
Single defects in infinite systems	79
Wannierization	79
Symmetry-enforced Wannierization	80
Disentanglement	81
Chebyshev kernel polynomial (KPM) methods	82

Classical spin models	83
Lattice gas models	84
Ising models	85
Parallelism and reproducibility	86
Errors and unsupported inputs	86
Main functions and methods	88
Geometry functions and methods	88
Hamiltonian functions and methods	89
Heterostructure functions and methods	122
SpinModel functions and methods	124
LatticeGas functions and methods	127
LatticeIsing functions and methods	131

pyqula user guide

pyqula computes electronic structure of tight-binding models on lattices: band structures, densities of states and spectral functions, self-consistent (mean-field) interacting Hamiltonians, superconductivity, topological invariants, response functions, quantum transport, and classical spin and lattice-gas models.

Almost everything in this guide follows the same four steps – build a geometry, get its Hamiltonian, add terms to it, then ask the Hamiltonian for an observable:

```
from pyqula import geometry
g = geometry.honeycomb_lattice()    # 1. a lattice
h = g.get_hamiltonian()              # 2. its tight-binding Hamiltonian
h.add_zeeman([0.,0.,0.3])           # 3. add terms (in place)
(k,e) = h.get_bands()                # 4. compute an observable
```

Install with `pip install pyqula`, or from a clone of the repository with `pip install -e .` from its root. Note that `import pyqula` on its own exposes nothing – always import the submodule you need (`from pyqula import geometry`).

Each code block below carries its own imports and runs on its own, except where a block picks up the variables of the one before it inside the same section (a second call on an `h` that was just built, say).

The snippets here stop at the computed arrays and leave the plotting to you. Two other places in the repository take it further:

- `examples/` holds several hundred runnable scripts organized by dimensionality (0d/ 1d/ 2d/ 3d/, plus `transport/`, `embedding/`, `wannier/`, `classicalspin/`, `latticegas/`, `latticeising/`, `spinon/`,

`kondolattice/`, `minimal/` and `readme_examples/`), most of them ending in a figure. Most sections below point at the relevant ones

- `jupyter-notebooks/functionalities/` holds 53 executed notebooks, one per feature, grouped the same way as the README's functionality list (single-particle Hamiltonians, mean field, topology, spectral functions, KPM, Wannierization, transport). Each carries its physics discussion and its output plots inline, so they are the place to look for what a result should actually look like

The last chapter, Main functions and methods, is a reference of the `Geometry/Hamiltonian` methods and their arguments, followed by the `Heterostructure`, `SpinModel`, `LatticeGas` and `LatticeIsing` classes. It is not exhaustive: it covers the methods the sections above use, not every public method on those classes.

Contents

- Setting up a Hamiltonian
- Observables
- Operators
- Non-Hermitian Hamiltonians
- Superconductivity
- Interactions at the mean-field level
- Spatially resolved density of states
- Electronic structure folding and unfolding
- Surface spectral functions
- Twisted bilayer graphene structural relaxation
- Topological insulators
- Entanglement
- Response functions
- Quantum transport
- Single defects in infinite systems
- Wannierization
- Chebyshev kernel polynomial (KPM) methods
- Classical spin models
- Lattice gas models
- Ising models
- Parallelism and reproducibility
- Errors and unsupported inputs
- Main functions and methods

Setting up a Hamiltonian

In this basic tutorial we will address how to compute the band structure of a one dimensional tight binding model.

The Hamiltonian of a one dimensional tight binding chain takes the form

$$H = \sum_n c_n^\dagger c_{n+1} + h.c.$$

This model can be diagonalized analytically, giving rise to a diagonal Hamiltonian of the form

$$H = \sum_k \epsilon_k \Psi_k^\dagger \Psi_k$$

where energy momentum dispersion takes the form

$$\epsilon_k = 2 \cos k$$

With the pyqula library, the previous band structure can be computed as

```
from pyqula import geometry
g = geometry.chain() # geometry of the 1D chain
h = g.get_hamiltonian() # generate the Hamiltonian
(k,e) = h.get_bands() # compute band structure
```

Including second and third neighbor hopping

By default, the Hamiltonian generated includes only first neighbor hopping $t_1 = 1$. However, we may want to consider a generalized Hamiltonian of the form

$$H = \sum_n c_n^\dagger c_{n+1} + t_2 \sum_n c_n^\dagger c_{n+2} + t_3 \sum_n c_n^\dagger c_{n+3} + h.c.$$

To compute the eigenvalues in this generalized model taking $t_2 = 0.2$ and $t_3 = 0.3$, we write

```
from pyqula import geometry
g = geometry.chain() # geometry of the 1D chain
h = g.get_hamiltonian(tij=[1.0,0.2,0.3]) # Hamiltonian with t1,t2,t3
(k,e) = h.get_bands() # compute band structure
```

Including an onsite energy

The Hamiltonian can have an onsite energy term, that is equivalent to a chemical potential that takes the form

$$H = \mu \sum_n c_n^\dagger c_n$$

This can be added to the Hamiltonian as

```
from pyqula import geometry
g = geometry.chain() # geometry of the 1D chain
h = g.get_hamiltonian() # generate the Hamiltonian
mu = 0.3 # value of the onsite
h.add_onsite(mu) # add onsite energy
```

Possible inputs

- Float: the same onsite energy is added to all the sites
- Iterable (list or array): adds a different onsite energy to each site in the geometry
- Callable (function): adds a different onsite energy to each site according to its location $\mathbf{r}$

Including an external Zeeman field

In the following we will consider that we want to add an external Zeeman field to the electronic system. We now include the existence of a spin degree of freedom, considering the Hamiltonian

$$H = H_0 + H_Z$$

where H_0 is the original tight binding Hamiltonian

$$H_0 = \sum_{n,s} c_{n,s}^\dagger c_{n+1,s} + h.c.$$

and

$$H_Z = \sum_{n,s,s'} \vec{B} \cdot \vec{\sigma}^{s,s'} c_{n,s}^\dagger c_{n,s'}$$

with n running over the sites and s, s' running over the spin degree of freedom. The magnetic field takes the form $\vec{B} = (B_x, B_y, B_z)$, and σ_α are the spin Pauli matrices. To add a magnetic field of the form $\vec{B} = (0.1, 0.2, 0.3)$ to our chain we write

```
from pyqula import geometry
g = geometry.chain() # geometry of the 1D chain
h = g.get_hamiltonian() # generate the Hamiltonian
h.add_zeeman([0.1, 0.2, 0.3]) # add the Zeeman field (modifies h in place)
(k, e) = h.get_bands() # compute band structure
```

Including an external orbital field

An external magnetic field can be included using the Peierls substitution

$$t_{\alpha\beta} \rightarrow t_{\alpha\beta} e^{i \int_{r_\alpha}^{r_\beta} \vec{A} \cdot d\vec{l}}$$

where $\vec{A}$ is the magnetic potential so that $\vec{B} = \nabla \times \vec{A}$. It can be used as shown in the example below

```
from pyqula import geometry
N = 20 # number of unit cells as the width
g = geometry.square_ribbon(N) # ribbon
h = g.get_hamiltonian() # generate the Hamiltonian
B = 0.02 # magnetic field in quantum flux unit
h.add_orbital_magnetic_field(B) # add an out-of plane magnetic field
(k,e) = h.get_bands() # compute the Landau-level band structure
```

Setting a filling

If you want to enforce a certain filling ν in a Hamiltonian, so that

$$\langle c_n^\dagger c_n \rangle = \nu$$

use

```
from pyqula import geometry
g = geometry.chain() # chain
h = g.get_hamiltonian()
h.set_filling(0.7) # enforce a filling
```

Possible inputs

- float: enforce the filling on average
- array: enforce that each site has a specific filling

Observables

Electronic band structures

For any system that is periodic in space, can compute the electronic band structure as given by

$$H = \sum_{k,\alpha} \epsilon_{k,\alpha} \Psi_{k,\alpha}^\dagger \Psi_{k,\alpha}$$

where α is the band index.

The previous calculation can be performed as

```

from pyqula import geometry
g = geometry.honeycomb_lattice() # geometry of the 2D model
h = g.get_hamiltonian() # generate the Hamiltonian
(k,e) = h.get_bands() # compute band structure

```

Optional arguments - **kpath**: k-path to use, either a list of high-symmetry labels (e.g. ["G","K","M"]) or explicit k-vectors; defaults to the geometry's standard path - **nk**: number of k-points along the path - **operator**: color/weight each band by the expectation value of an operator (or a list of operators), returning (k,e,c) instead of (k,e) - **num_bands**: for large sparse Hamiltonians, only compute this many bands around **central_energy** with ARPACK, instead of the full spectrum

```

from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian(has_spin=True)
(k,e,c) = h.get_bands(operator="velocity") # bands colored by group velocity

```

Passing a list of operators computes the expectation value of each of them for every eigenstate, returning one extra column per operator ((k,e,c1,c2,...) instead of (k,e,c))

```

(k,e,c_sz,c_site) = h.get_bands(operator=["sz","site"]) # two operators at once

```

For a very large system (e.g. a moire supercell), diagonalizing the full Hamiltonian at every k-point is wasteful if only a handful of bands around the Fermi level are of interest. Passing **num_bands** switches to a sparse ARPACK solver that targets only those bands

```

(k,e) = h.get_bands(num_bands=20) # only the 20 bands closest to central_energy

```

See `examples/2d/velocity_bands/main.py` and `examples/2d/strain_TBG/main.py` for runnable versions.

Density of states

The density of states counts how many states are in a certain energy window. It is defined as

$$D(\omega) = \int \delta(\omega - \epsilon_k) dk$$

where ϵ_k are the eigenenergies of the Hamiltonian. It can be used as shown below

```

from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
(es,ds) = h.get_dos()

```


Optional arguments - energies: array with the energies for which the DOS is computed - delta: smearing of the DOS - operator: operator to which the DOS is projected - mode: how the DOS is computed – "ED" (default, broadens a k-mesh band structure), "Green"/"RG" (sums a Green's function per energy, useful when only a handful of energies are needed), "KPM" (Chebyshev kernel-polynomial expansion, for large sparse systems – see the “Chebyshev kernel polynomial (KPM) methods” section), or "adaptive" - nk: number of k-points in the mesh, for "ED" and "KPM". "adaptive" does not sample a mesh at all – it integrates over the Brillouin zone with error-controlled quadrature, tuned by `error=1e-1`, and reads `nk` only as a subdivision limit. "Green"/"RG" pass `nk` to the Brillouin-zone sum behind the self-energy, where it matters for `gmode="full"` and (in 2D) `gmode="renormalization"` but not for the default `gmode="adaptive"`

Whatever the mode, the result is normalized as a density of states: integrating it over the energies gives the number of states per unit cell. On a spinful chain, whose answer is 2, all four modes reproduce it to better than 1% on the same window, so they can be compared with each other directly – and so can `h.get_dos()`, the DOS written next to an LDOS map by `h.get_multidos()`, and the file `dos.dos_ewindow` writes

```
from pyqula import geometry
import numpy as np
g = geometry.chain()
h = g.get_hamiltonian(tij=[0.5,0.,0.,0.5],has_spin=True)
h.add_rashba(0.7)
energies = np.linspace(-4.,4.0,60)
(e1,d1) = h.get_dos(energies=energies,delta=1e-2,mode="ED",nk=1000)
(e2,d2) = h.get_dos(energies=energies,delta=1e-2,mode="Green")
```

An operator can be passed to project the DOS onto a subspace, e.g. the sublattice-resolved DOS of a gapped honeycomb lattice

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian(has_spin=True)
h.add_sublattice_imbalance(.4)
(es,ds) = h.get_dos(operator="sublattice",nk=40,delta=5e-2)
```

See `examples/1d/dos_GF/main.py` and `examples/2d/operator_dos/main.py` for runnable versions.

Local density of states

The local density of states resolves the density of states by site: it counts how many states are in a certain energy window, weighted by how much of each state sits on site n . It is defined as

$$D(\omega, n) = \int \delta(\omega - \epsilon_k) |\langle \Psi_k | n \rangle|^2 dk$$

where ϵ_k are the eigenenergies of the Hamiltonian. It can be used as shown below

```
from pyqula import geometry
g = geometry.honeycomb_zigzag_ribbon() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
(x,y,d) = h.get_ldos()
```

Optional arguments - e: energy at which the LDOS is evaluated - delta: smearing of the LDOS - operator: operator to which the LDOS is projected (a name such as "sz", a matrix, or an `Operator`), giving e.g. a spin-resolved real-space map instead of the charge one. The two evaluation modes weight it differently and both integrate over sites to the same operator-resolved DOS: `mode="arpack"` (the default) uses the expectation value $\langle \Psi | A | \Psi \rangle$ times the local density $|\Psi(i)|^2$, while `mode="green"` uses the local matrix element $\text{Re } \Psi^*(i)(A\Psi)(i)$, which is the genuinely local quantity for states that are not eigenstates of the operator. A momentum-dependent operator ("valley") only works in `mode="arpack"`, since `mode="green"` has already integrated over the Brillouin zone - mode: "arpack" (default, diagonalization on a k-mesh) or "green" (Green's function, 2D Hamiltonians only) - projection: "TB" (default, one value per lattice site), "TBRs" (same, but interpolated onto a continuous real-space map for smoother plotting), or "atomic" (projected onto atomic orbitals rather than tight-binding sites) - num_bands: for large sparse Hamiltonians, use ARPACK to only compute this many states around the target energy

```
from pyqula import geometry
g = geometry.honeycomb_zigzag_ribbon() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
(x,y,d) = h.get_ldos(e=0.0,projection="TBRs") # interpolated real-space map
```

`h.get_multildos()` computes the LDOS at many energies at once, writing one file per energy to a MULTILDOS/ folder (useful for building an LDOS(x,y,E) movie/stack), plus a MULTILDOS/DOS.OUT holding the corresponding total DOS

```
import numpy as np
h.get_multildos(energies=np.linspace(-2.0,2.0,100),projection="atomic")
```

The maps and the DOS.OUT beside them carry the same normalization as `h.get_ldos()` and `h.get_dos()` on the same system, so a map can be read against a single-energy LDOS and the DOS.OUT against `h.get_dos()` without rescaling. An `operator=` is honoured here as it is by `get_ldos` (the older spelling `op=` still works, but passing both is a `TypeError`), and the weight it applies is the expectation value $\langle \Psi | A | \Psi \rangle$ of the eigenstate – a gauge-invariant number – times the local density, the same convention

`get_ldos(mode="arpack")` uses. `projection="atomic"` does not accept an operator and says so

See `examples/0d/island/main.py` (single-energy, `projection="TBRs"`, superconducting island) and `examples/readme_examples/ldos_island/main.py` (`get_multildos, projection="atomic"`) for runnable versions.

Momentum resolved spectral functions

Apart from the band structure, in certain cases it is interesting to compute the momentum resolved spectral function, that takes the form

$$A(k, \omega) = \delta(\omega - \epsilon_k) |\langle \Psi_k | A | \Psi_k \rangle|^2$$

where A is a certain operator. The previous quantity allows define a heatmap of the momentum-resolved spectral function. For the example, in a superconducting state, if operator is chosen to be projection onto the electron-sector, the previous quantity shows the electronic spectral function

```
from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
h.get_kdos_bands()
```

Optional arguments - energies: array with the energies for which the DOS is computed - delta: smearing of the DOS - operator: operator to which the DOS is projected

Fermi surfaces

For a 2D periodic Hamiltonian, `h.get_fermi_surface()` computes the spectral weight on a (k_x, k_y) mesh at a single energy (by default the Fermi level, `e=0.0`), i.e. a single constant-energy cut

```
from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
(kx, ky, fs) = h.get_fermi_surface(e=0.0, nk=50, delta=1e-1)
```

Optional arguments - e: energy of the cut - nk: number of k-points per direction - delta: broadening - operator: project/weight the Fermi surface by an operator, giving e.g. a spin- or valley-textured Fermi surface

```
from pyqula.specialhamiltonian import NbSe2
h = NbSe2(soc=0.9) # multi-orbital spin-orbit-coupled Hamiltonian
(kx, ky, fs) = h.get_fermi_surface(e=0., nk=100, delta=3e-1, operator="sz")
```

`h.get_multi_fermi_surface()` computes the same kind of map at many energies at once, writing one file per energy to a `MULTIFERMISURFACE/` folder –

convenient for scanning how the Fermi surface evolves away from the Fermi level

```
import numpy as np
h.get_multi_fermi_surface(energies=np.linspace(-4,4,100),delta=1e-1)
```

Passing `operator="unfold"` together with `nsuper` unfolds the Fermi surface of a defective/disordered supercell back onto the primitive Brillouin zone (see the “Electronic structure folding and unfolding” section); as with QPI unfolding, the supercell must be built with `store_primal=True`

```
import numpy as np
from pyqula import geometry
g0 = geometry.triangular_lattice()
n = 3 # size of the supercell
g = g0.get_supercell(n,store_primal=True)
h = g.get_hamiltonian()
h.add_onsite(lambda r: 100.0 if np.linalg.norm(r-g.r[0])<1e-1 else 0.0) # a point defect

out = h.get_multi_fermi_surface(nk=50,energies=np.linspace(-4,4,100),delta=0.1,
                                nsuper=n,operator="unfold")
```

See `examples/readme_examples/fermi_surface/main.py`, `examples/2d/operator_fermi_surface/main.py` and `examples/readme_examples/unfolding_FS/main.py` for runnable versions.

Quasiparticle interference

Quasiparticle interference (QPI) maps the momentum-space scattering pattern that a defect or impurity produces, and is what an STM quasiparticle-interference measurement probes. `h.get_qpi()` is only available for 2D Hamiltonians; unlike the other observables here it does not return arrays – it writes its output to disk, one file per energy in an output folder (default `MULTIQPI/`) plus a combined `DOS.OUT`

```
import numpy as np
from pyqula import geometry
g = geometry.triangular_lattice()
h = g.get_hamiltonian(has_spin=False)
h.get_qpi(mode="pm",nk=50,delta=1e-1,energies=np.linspace(-6.,6.,100))
```

Optional arguments - `energies`: array of energies to compute - `nk`: number of k-points per direction - `delta`: broadening - `mode`: “pm” (“poor man’s”) autoconvolves the actual k-resolved spectral weight of the (possibly defective) system in q-space – the physically meaningful QPI signal for a real scatterer; “response” (default) instead computes a cheaper Lindhard-like joint-DOS convolution from the clean band structure only, ignoring wavefunction form factors - `nunfold`: for a defect embedded in an `nunfoldxnunfold` supercell, unfold the QPI signal back onto the primitive Brillouin zone

A single point defect embedded in a supercell, with the resulting QPI unfolded back onto the primitive cell, is a realistic use case. The supercell must be built with `store_primal=True` so pyqula remembers the primitive-cell reference needed to unfold; `operator="unfold"` then resolves to the corresponding unfolding operator

```
import numpy as np
from pyqula import geometry
g0 = geometry.honeycomb_lattice()
ns = 2
g = g0.get_supercell(ns, store_primal=True)
h = g.get_hamiltonian(has_spin=False)
h.add_onsite(lambda r: 100.0 if np.linalg.norm(r-g.r[0])<1e-1 else 0.0) # a strong point de

h.get_qpi(mode="pm", delta=1e-2, operator="unfold", nsuper=2, nk=140, nunfold=ns)
```

This is the most expensive snippet in the guide: `mode="pm"` diagonalizes on an $n_k \times n_k$ mesh and then autoconvolves the result, so the cost grows quadratically with n_k and the $n_k=140$ above takes minutes. Drop to $n_k=60$ (about 40 seconds) while setting a calculation up, and raise n_k only for the final figure – the q-space resolution of the QPI pattern is what it buys.

See `examples/2d/multiqpi/main.py` (clean system, `mode="pm"`) and `examples/2d/multiqpi_unfold/main.py` (defect in a supercell, unfolded) for runnable versions.

Real-space-impurity QPI

`h.get_qpi()`'s modes are all reciprocal-space methods (they convolve or scatter k-resolved spectral weight, never touch real-space impurities). `h.get_qpi_impurity()` instead takes the direct route: it builds a supercell of `h`, adds one or more actual real-space impurities to it, computes the real-space LDOS map with ARPACK partial diagonalization (only the eigenstates nearest each requested energy, so this stays tractable for large supercells, unlike full diagonalization), and Fourier transforms that map directly (a discrete sum over the atoms' actual positions, not a grid FFT) to get the QPI(q) signal. Unlike `get_qpi()`, it returns arrays rather than only writing to disk

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian(has_spin=False)
r, ldos_r, q, qpi_q = h.get_qpi_impurity(nsuper=10,
    impurities=[{"position": [0., 0., 0.], "onsite": 3.0}],
    energies=0.3, num_waves=60, nk=2, delta=0.2)
```

Optional arguments - `nsuper`: supercell size (scalar or (n_1, n_2)) - `impurities`: list of dicts, each an onsite potential (`{"position": [x, y, z], "onsite": v}`, or `{"index": i, "onsite": v}` for a specific supercell site index) or a va-

cancy (`{"position": [x,y,z], "vacancy": True}`). A vacancy is modeled as a strong onsite potential rather than true site removal, since deleting sites from a large sparse supercell Hamiltonian would require densifying it - energies: a single energy or an array - num_waves: a starting guess for the number of ARPACK eigenstates nearest the requested energies - automatically grown (more ARPACK calls, not a correctness risk) until the diagonalization covers `margin*delta` past every requested energy and never cuts a degenerate manifold in half, since summing over a partial manifold isn't basis-independent and would otherwise leak spurious QPI weight, dependent on ARPACK's starting vector, even for a clean (impurity-free) supercell - nk, delta: as in `get_ldos`

The Hamiltonian is kept sparse throughout (the primitive cell is turned sparse before the supercell is built, and impurities are added as a sparse diagonal), so no dense matrix of the supercell's size is ever built. No unfolding step is needed either: this never diagonalizes supercell bands and projects them onto primitive Bloch states (which is what `get_qpi`'s `nunfold/store_primal` are for) - it only Fourier transforms a real-space scalar density, evaluated directly at $\mathbf{q}$ spanning the full primitive Brillouin zone. $\mathbf{q}$ is fixed at exactly the `nsuper1xnsuper2` points commensurate with the supercell (`nsuper` sets the achievable $\mathbf{q}$ resolution, not the BZ range); evaluating the direct-sum Fourier transform at any other $\mathbf{q}$ would show finite-size leakage even for a perfectly clean system, since only the commensurate points are free of it.

See `examples/2d/qpi_realspace_impurity/main.py` for a runnable version that plots both the real-space LDOS and $\text{QPI}(\mathbf{q})$.

Spin splitting of an altermagnet

In a collinear magnet whose spin-up and spin-down bands are split without any net magnetization - an altermagnet - the natural quantity to characterize the order is how far apart the two spin channels are pushed, resolved by the energy at which that happens. Both quantities below diagonalize the two spin blocks separately and pair the resulting bands by index, so for every $\mathbf{k}$ -point and band index n there is a splitting $\text{Delta}_n(\mathbf{k}) = E_{\text{up}_n}(\mathbf{k}) - E_{\text{dn}_n}(\mathbf{k})$ sitting at the mean energy $E_{\text{bar}_n}(\mathbf{k}) = (E_{\text{up}_n}(\mathbf{k}) + E_{\text{dn}_n}(\mathbf{k}))/2$.

The two differ in how they reduce that set. `get_spin_splitting_density` broadens every pair into a smooth weighted density, giving the *typical* splitting at each energy. `get_spin_splitting_vs_energy` instead keeps the *largest* $|\text{Delta}|$ found in each energy bin, so its global maximum is a bound on the spin splitting anywhere in the Brillouin zone. The distinction matters when reporting a single number for a material: a maximum taken along one cut through $\mathbf{k}$ -space (a circle of fixed radius, say) depends on choosing where to look, while the binned maximum over a full mesh does not.

```
import numpy as np
from pyqula import specialhamiltonian
```

```

h = specialhamiltonian.square_altermagnet(am=1.)

# largest spin splitting in each energy bin, over the whole BZ
(es, ds) = h.get_spin_splitting_vs_energy(nk=100, nbins=400)
print("largest spin splitting anywhere in the BZ:", ds.max())

# the smooth counterpart, on the same axes
(xs, ys) = h.get_spin_splitting_density(nk=100, delta=1e-1, energies=es)

```

Both return (energies, values) with the same shape convention, so they can be plotted together. Bins containing no states come back as 0.0 rather than NaN, and states falling outside an explicitly requested energy window are dropped rather than piled onto the end bins.

Two things about that index pairing are worth knowing, because together they decide whether the number you report is the number you meant.

The first is reassuring. In a collinear altermagnet the two spin channels are related by a point-group operation acting on $\mathbf{k}$ rather than by a state-by-state correspondence at fixed $\mathbf{k}$: the sorted spectra satisfy $E_{\text{up},n}(\mathbf{k}) = E_{\text{dn},n}(\mathbf{R}\mathbf{k})$ exactly, so $\Delta_n(\mathbf{k}) = E_{\text{dn},n}(\mathbf{R}\mathbf{k}) - E_{\text{dn},n}(\mathbf{k})$ compares the same sorted index within one channel at two related momenta. No band-identification ambiguity survives, and the usual worry – that a splitting exceeding the band spacing makes the n -th up and n -th down band different bands – does not arise, because the symmetry supplies the correspondence. It is worth identifying $\mathbf{R}$ for a new system: compare `sorted( $E_{\text{up}}(\mathbf{k})$ )` with `sorted( $E_{\text{dn}}(\mathbf{R}\mathbf{k})$ )` over the point group and look for the operation giving $\sim 1\text{e-}14$. For the square altermagnet it is the $\mathbf{k}_1 \leftrightarrow \mathbf{k}_2$ mirror.

The second is a genuine dependence that the symmetry does *not* remove: n labels whatever band set your unit cell produces, and folding changes that set. On a `supercell(2)` of the square altermagnet the mirror identity still holds to $4\text{e-}16$, yet the reported maximum drops from 4*am to 2*am , because the folded bands at one $\mathbf{k}$ come from four primitive $\mathbf{k}$ -points and index pairing compares across them. So give this the true magnetic unit cell. If the converged order repeats with a smaller period than the cell it was solved in, the result is a lower bound rather than the maximum.

Both assume spin is a good quantum number, since they are built on `remove_spin`, which keeps one spin block and discards the off-diagonal one. `get_spin_splitting_vs_energy` checks this and raises if the spin off-diagonal block of the Bloch Hamiltonian is not negligible – with Rashba coupling, any other spin-orbit term, or non-collinear magnetic order the splitting defined above is not a meaningful quantity, and a silently wrong number would be worse than an error. Diagonalization is dense throughout, deliberately: a sparse solver returns only the eigenvalues nearest $E=0$, and the splitting commonly peaks far away from there.

See `examples/2d/spin_splitting_vs_energy/main.py` for a runnable

version plotting both curves (and showing the redundant-cell trap), and `examples/2d/altermagnetism_density/main.py` for the density alone.

Operators

Both when computing band structures, density of states and expectation values we could define operators to filter the results. In this section we elaborate on some important operators that are available, and we comment on their physical meaning.

Operators in `pyqula` have some important properties. First, for periodic Hamiltonian they can have an intrinsic momentum dependence. Second, `pyqula` allows for native algebra between them, namely they can be summed or multiplied, automatically accounting for intrinsic momentum dependences. Third, they can be non-linear, providing a generalization of matrix operators.

Nonlinear spin current as a measurement of altermagnetic order

The spin splitting above tells you *how big* the altermagnetic order is. A harder question is *which kind* it is – d-wave, g-wave, i-wave – and that turns out to have a purely electrical answer, one that needs no spin-orbit coupling at all.

The spin-splitting form factor of an X-wave collinear magnet is a k-space harmonic of order $l+1$: $k_x k_y$ for d-wave, $k_x k_y (k_x^2 - k_y^2)$ for g-wave, $k_x k_y (3k_x^2 - k_y^2) (k_x^2 - 3k_y^2)$ for i-wave. In the semiclassical Boltzmann treatment the l -th order nonlinear Drude conductivity is a Brillouin-zone integral of the $(l+1)$ -th derivative of the band energy,

$$\sigma_{s\{x^{l1} y^{l2} ; b\}} = (-e/\hbar)^{(l+1)} / (i \omega + 1/\tau)^{l1} \\ * \int d^2k / (2\pi)^2 f_s^{(0)} d^{(l+1)} \epsilon_s / (dk_x^{l1} dk_y^{l2} dk_b)$$

with $\sigma_{\text{spin}} = (\sigma_{\text{up}} - \sigma_{\text{dn}})/2$ and $\sigma_{\text{charge}} = \sigma_{\text{up}} + \sigma_{\text{dn}}$. A harmonic of order $l+1$ has a nonzero constant $(l+1)$ -th derivative and nothing below it survives the zone integral, so the spin response switches on at exactly one order and is silent below it:

wave	p	d	f	g	i
lowest order l	0	1	2	3	5

Reading off the lowest order at which a nonlinear spin current appears therefore identifies the wave index. This is the content of Ezawa, *Phys. Rev. B* **111**, 125420 (2025) (arXiv:2411.16036).

```
from pyqula import specialhamiltonian
```



```

h = specialhamiltonian.iwave_altermagnet(J=0.3)    # or dwave_/fwave_/gwave_

# largest |sigma_spin| over every component of each order l
orders = h.get_nonlinear_drude_orders(lmax=6, nk=48, T=0.02, mu=-5.5)
# -> zero to machine precision for l = 0..4, nonzero at l = 5

# one component, and every component of a given order
s = h.get_nonlinear_drude_conductivity(field="yyyyy", current="x", mu=-5.5)
c = h.get_nonlinear_drude_components(5, nk=48, mu=-5.5)    # {"x^l1 y^l2; b": value}

```

field carries one character per power of the electric field, so field="yyyyy" is the fifth-order response to E_y ; current is the direction the current is measured in. channel selects "spin", "charge", "up" or "dn". Units are $e = \hbar = 1$ and the zone integral carries the $1/(2\pi)^D$ of a density, as in the rest of conductivity.py; fermi_volume uses the same normalization, so ratios to it reproduce the paper's analytic results directly.

The X-wave lattice models themselves are specialhamiltonian.xwave_magnet(wave=...), with pwave_magnet, dwave_altermagnet, fwave_magnet, gwave_altermagnet and iwave_altermagnet as named aliases. Each is Ezawa's tight-binding model on the host lattice its symmetry requires – square for p, d and g, triangular for f and i – built by expanding his sine-product form factors exactly into real-space hoppings.

Three things are worth knowing before quoting a result.

Orders above the threshold do not vanish. On a lattice the form factor carries higher harmonics beyond the leading one, so a d-wave altermagnet responds at $l = 1$ but also at $l = 3$ and $l = 5$. It is the *absence of everything below* the threshold that carries the information, not the presence of a single isolated order.

An insulator gives zero at every order, which is a trap. With the chemical potential in a gap, f is 1 on every valence band and 0 on every conduction band, so the integrand is a pure k -derivative of $\text{Tr}(P H)$ with P the valence projector; that is smooth and periodic, and the zone integral of a derivative of a smooth periodic function vanishes. The result is an exact zero at every order – including the fifth. So a gapped system reproduces the “nothing below fifth order” pattern trivially, and the fingerprint only identifies i-wave order if the fifth order is simultaneously shown to be present. That requires a metal.

A spin-degenerate state is flagged, not silently answered. The spin channel is only meaningful if the two spin channels have different bands. When they do not – a compensated Neel state on a bipartite lattice is PT symmetric and exactly spin degenerate, an *antiferromagnet* rather than an altermagnet – what comes back is rounding, and it does not look small: on a multiorbital cell the two channels are diagonalized independently, so near a degeneracy their eigenvector gauges differ and the high-order derivatives drift apart far more than the band energies do. Measured on a 44-site honeycomb antidot Neel state,

bands agreeing to $1.2\text{e-}14$ produced a fifth-order “spin response” of $1.3\text{e-}3$, which looks exactly like a signal. The module therefore checks the band splitting itself and raises a `RuntimeWarning` when it is below $1\text{e-}10$ of the bandwidth. A genuine altermagnet is nowhere near that floor: the response stays exactly linear in the order parameter down to $J \sim 1\text{e-}9$.

Which X-waves a real Hamiltonian can carry. Reality of the Hamiltonian is itself a selection rule, and a useful one when building or diagnosing a model. With real hoppings and no spin-orbit coupling, $H_s(\mathbf{k})^* = H_s(-\mathbf{k})$, so $\text{eps}_s(\mathbf{k}) = \text{eps}_s(-\mathbf{k})$ and the splitting satisfies $\text{Delta}(\phi+180 \text{ deg}) = \text{Delta}(\phi)$ – every *odd* harmonic is forbidden. Add a C3 axis, which kills every 1 not divisible by 3, and only $l = 6, 12, \dots$ survives: **a real, SOC-free, C3-symmetric cell is necessarily i-wave**, and no choice of vacancy pattern or supercell can make it f-wave. The models here bear this out – p-wave and f-wave come out with *imaginary* hoppings, while d, g and i wave are real – so an f- or g-wave fixture cannot be built from a real C3 model and needs complex hoppings or lower symmetry.

Spin must be a good quantum number. The derivation treats each spin channel as an independent single-band problem, so a Hamiltonian with spin-orbit coupling is rejected rather than silently mistreated: with Rashba or Kane-Mele terms the nonlinear response also picks up quantum-metric and Berry-curvature-dipole contributions (arXiv:2409.09241) that this formula does not contain.

Multiorbital cells are supported and are the interesting case – superlattices, antidot lattices, multilayers. There the band energies are eigenvalues rather than matrix elements, and their derivatives are obtained by Rayleigh-Schroedinger perturbation theory carried out in a truncated two-variable Taylor ring, which delivers every mixed derivative up to sixth order to machine precision. Finite differences are not an option here and the module does not offer them: the selection rule is a statement that certain derivatives vanish *identically*, and a difference stencil manufactures a spurious nonzero value exactly there – measured at $1\text{e-}4$ against a true zero on a case where the surviving response is 0.6, which would read as the response appearing at the wrong order and so invert the conclusion. The expansion is done in the block (Kato) form, grouping degenerate multiplets and using the trace of the block effective Hamiltonian, because a cell with a C3 or C6 axis has two-dimensional irreducible representations and hence exactly degenerate bands at its high-symmetry points – unavoidable for the superlattices this is most useful for. A `supercell(2)`, whose folded bands are degenerate along whole curves and so exercises that treatment everywhere, reproduces the primitive-cell answer on the equivalent k-mesh to $1\text{e-}16$ at first order and $1\text{e-}14$ at fifth – with the four orders below the fifth staying at machine zero in both, which is the property that actually matters.

See `examples/2d/xwave_nonlinear_spin_current/main.py` for a runnable version printing the whole selection-rule table.

Spin operators

The simplest operators are the spin operators

$$S_\alpha = \sum_n \sigma_\alpha^{\mu\nu} c_{n,\mu}^\dagger c_{n,\nu}$$

with σ_α the Pauli matrices, that can be obtained as

```
from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
sx = h.get_operator("sx") # Spin x component
sy = h.get_operator("sy") # Spin y component
sz = h.get_operator("sz") # Spin z component
```

Listing the available operators

Every routine that takes an operator argument (`h.get_bands`, `h.get_dos`, `h.get_ldos`, `h.get_kdos_bands`, `h.get_vev`, the topological invariants...) accepts either an object – an `Operator`, a `Hamiltonian`, a `Potential`, a callable of the position, a bare matrix – or the name of one of the operators that `pyqula` knows how to build. The names are kept in a single registry, and can be listed

```
from pyqula import operatorlist
print(operatorlist.get_operator_names()) # every name h.get_operator accepts
```

Passing a name that is not in that list raises a `ValueError` quoting the offending name and the accepted ones, rather than failing further downstream.

Listing the other names pyqula accepts

Named operators are not the only thing selected by a string. The mean-field initialization (`mf=`), the superconducting pairing symmetry (`mode=`), the high-symmetry kpoint labels of a band path, and the quantities `h.extract` knows how to pull out of a Hamiltonian are all chosen the same way, and each of them keeps its names in a registry that the dispatch itself reads. So the accepted values can always be listed, and a name that is not accepted raises a `ValueError` quoting both the offending name and the full list rather than failing somewhere downstream

```
from pyqula import meanfield, extract
from pyqula.sctk import pairing
from pyqula.kpointstk import labels

print(meanfield.get_guess_names()) # every mf= mean-field initialization
print(pairing.get_pairing_modes()) # every h.add_pairing(mode=...) symmetry
print(labels.get_label_names()) # every high-symmetry kpoint label
print(extract.get_extractable_names()) # every h.extract(...) quantity
```

Because each list is derived from the dispatch rather than written out beside it, the two cannot disagree: a symmetry that is advertised is one that is built.

Location operator

To understand the spatial location of the states we can use the spatial operators, that denote where wavefuctions are located in real space

$$R_\alpha = \sum_{r,s} r_\alpha c_{r,s}^\dagger c_{r,s}$$

with r_α is the component of the position of site r

```
from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
x = h.get_operator("xposition") # x component
y = h.get_operator("yposition") # y component
z = h.get_operator("zposition") # z component
```

Bulk-edge operator

In order to know if a state is located at the edge or in the bulk of the system you can use the bulk-edge location operators. The edge operator takes value 1 for sites on the edge, and 0 for sites in the bulk.

$$\hat{E} = \sum_{r \in \text{Edge}, s} c_{r,s}^\dagger c_{r,s}$$

The bulk operator takes value 1 for sites on the bulk, and 0 for sites on the edge.

$$\hat{B} = \sum_{r \in \text{Bulk}, s} c_{r,s}^\dagger c_{r,s}$$

```
from pyqula import geometry
g = geometry.honeycomb_zigzag_ribbon() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
b = h.get_operator("bulk") # bulk operator
e = h.get_operator("edge") # edge operator
```

Valley operator

For hoenycomb like systems, including aligned and twisted multilayers, an operator that allows to extract the valley degree of freedom can be extracted. This operator takes the form

$$V = i \sum_{\langle\langle ij \rangle\rangle, s} \nu_{ij} \sigma_{ij} c_{r_i, s}^\dagger c_{r_j, s}$$

where $\nu = \pm 1$ and $\sigma = \pm 1$ for clockwise/anticlockwise, sublattice A/B. This the so-called anti-Haldane hopping, and takes opposite values in opposite valleys. It can be obtained for honeycomb systems as

```
from pyqula import geometry
g = geometry.honeycomb_zigzag_ribbon() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
vall = h.get_operator("valley") # valley operator
```

In-plane valley operators

The operator above is the out-of-plane valley pseudospin τ_z . The two remaining components of the valley pseudospin, τ_x and τ_y , can also be obtained, giving access to the full valley vector (τ_x, τ_y, τ_z) – the valley-space analogue of (S_x, S_y, S_z) for real spin. They are built from a chiral Kekule coupling (symmetrized over the 3 inequivalent Kekule registries so the result is exactly C_3 -covariant about every atom, not only about special high-symmetry points) rather than from the second-neighbor coupling behind τ_z

```
from pyqula import geometry
g = geometry.honeycomb_lattice().supercell(3) # Kekule-commensurate cell
h = g.get_hamiltonian(has_spin=False) # get the Hamiltonian
taux = h.get_operator("valley_x") # tau_x
tauy = h.get_operator("valley_y") # tau_y
```

Both require a honeycomb-like geometry with a sublattice index; for a periodic (non-0d) Hamiltonian they additionally require a Kekule-commensurate cell (already a 3x3, or other multiple-of-3, supercell of the primitive honeycomb cell) to be well-defined – a finite (0d) flake needs no such commensurability.

A single vacancy in an otherwise pristine honeycomb flake is an atomically-sharp, intervalley-scattering defect, and induces a vortex in the in-plane valley pseudospin around it – a nice way to see τ_x, τ_y in action

```
from pyqula import islands
from pyqula import spectrum
g = islands.get_geometry(name="honeycomb", n=8, nedges=6)
gv = g.remove(g.get_central()[0]) # flake with a single vacancy
hv = gv.get_hamiltonian(has_spin=False)
dvx = spectrum.real_space_vev(hv, operator=hv.get_operator("valley_x"))
dvy = spectrum.real_space_vev(hv, operator=hv.get_operator("valley_y"))
```

`h.add_valley_exchange(v)`, with $v=(v_x, v_y, v_z)$, adds a valley-space exchange term $\vec{v} \cdot (\tau_x, \tau_y, \tau_z)$ to the Hamiltonian – the valley-pseudospin analogue of `add_exchange` for real spin

```

from pyqula import geometry
g = geometry.honeycomb_lattice().supercell(3) # Kekule-commensurate cell
h = g.get_hamiltonian(has_spin=False)
h.add_valley_exchange([0.1,0.05,0.2]) # (vx,vy,vz)

```

See `examples/0d/valley_vortex_vacancy/main.py` and `examples/2d/valley_vortex/main.py` for runnable versions of the vacancy-vortex example.

Nambu operators

In the presence of superconductivity, you can project onto the electron or hole component of the Nambu spinor using the electron-hole operators. The Hamiltonian must already be in the Nambu (BdG) basis – i.e. some pairing has been added – otherwise there is no hole sector to project onto and these raise

```

from pyqula import geometry
g = geometry.triangular_lattice() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
h.add_swave(0.2) # add pairing, doubling the basis into Nambu space
electron = h.get_operator("electron") # electron component
hole = h.get_operator("hole") # hole component
(k,e,c) = h.get_bands(operator=electron) # electron weight of each state

```

Berry curvature operator

The Berry curvature operator is a first example of an operator that is intrinsically momentum dependent. The Berry curvature operator is defined as

$$O|\Psi_k\rangle = \Omega(k, \epsilon_k)|\Psi_k\rangle$$

where $\Omega(k, \omega)$ is the Berry curvature evaluated at the momentum k and energy ω of the eigenstate $|\Psi\rangle$. In particular, this operator allows to directly see the contribution to the Berry curvature of different states in the band structure.

Inverse participation ratio operator

So far we have considered operators that are linear, namely that fulfill the condition

$$A(|\Psi_1\rangle + |\Psi_2\rangle) = A|\Psi_1\rangle + A|\Psi_2\rangle$$

There is however one operator that it is interesting to consider that does not fulfill such condition. The operator is the so-called inverse participation ration, which we define as

$$O|\Psi\rangle = \sum_i |\langle i|\Psi\rangle|^4 |\Psi\rangle$$

In particular, the previous operator allows to identify states that are highly localized in a few lattice sites, becoming useful to highlight impurity states and localized modes.

```
from pyqula import geometry
g = geometry.honeycomb_zigzag_ribbon() # get the geometry
h = g.get_hamiltonian() # get the Hamiltonian
h.add_onsite(0.3) # add a sublattice imbalance
ipr = h.get_operator("IPR") # IPR operator
```

Non-Hermitian Hamiltonians

An open quantum system – one exchanging particles or energy with an environment – is often described by an effective Hamiltonian that is no longer Hermitian: gain and loss enter as imaginary onsite energies, and non-reciprocal hopping ($t_{ij} \neq t_{ji}^*$) as an asymmetric hopping matrix. The eigenvalues are then complex, their imaginary parts being amplification and decay rates rather than energies, and the eigenvectors are no longer orthogonal. This is the setting of photonic and acoustic lattices with gain and loss, of the non-Hermitian skin effect, and of $\mathcal{PT}$ -symmetric models.

pyqula builds such a Hamiltonian with the `non_hermitian=True` flag, after which the usual observables route to non-Hermitian implementations (`pyqula.nonhermitiantk`) instead of the Hermitian ones. Nothing else about building the model changes – `add_onsite` with a complex-valued function is what puts the gain and loss in.

```
import numpy as np
from pyqula import geometry

n = 20
g = geometry.chain().get_supercell(n)
h = g.get_hamiltonian(has_spin=False, non_hermitian=True, tij=[-1.])
# a purely imaginary Aubry-Andre modulation, with no Hermitian part at
# all: gain where the cosine is positive, loss where it is negative
h.add_onsite(lambda r: 0.6j*np.cos(2.*np.pi*r[0]/n))

(ks, es) = h.get_bands(kpath=[[0., 0., 0.]], write=False)
print("eigenvalues are complex:", es.dtype)
print("largest gain rate Im(E):", np.round(np.max(es.imag), 4))
```

The eigenvalues come back complex rather than real, so anything that plots them has to choose a part: `es.real` for the energy axis, `es.imag` for the gain/decay axis, or both as a scatter in the complex plane.

Choosing which part is the energy: eigmode

`eigmode` is the argument that exists only on this path. It says which part of the complex eigenvalue is to play the role of “the energy” – the quantity a broadening is centred on, or a band structure is written out as. It takes `"complex"` (the default, keep the whole eigenvalue), `"real"`, or `"imag"`; anything else raises `ValueError` listing the three.

In `h.get_bands()` it selects what the returned energy row and the written `BANDS.OUT` carry. With the default `"complex"` the file gains one extra column, so its layout is `k, Re E, Im E`, then one column per operator – rather than silently dropping the imaginary part, which is the physics the calculation was done for.

In `h.get_ldos()` it decides which axis the requested energy `e` lives on. With `eigmode="imag"` the states are selected by their amplification rate instead of their energy, which is how one asks where the most amplified mode of the model above actually sits:

```
import numpy as np
from pyqula import geometry
n = 20
g = geometry.chain().get_supercell(n)
h = g.get_hamiltonian(has_spin=False, non_hermitian=True, tij=[-1.])
h.add_onsite(lambda r: 0.6j*np.cos(2.*np.pi*r[0]/n))
(ks, es) = h.get_bands(kpath=[[0., 0., 0.]], write=False)

imax = np.argmax(es.imag) # the most amplified state
(x, y, d) = h.get_ldos(e=es[imax].imag, delta=1e-2, eigmode="imag", nrep=1)
print("it lives at x =", x[np.argmax(d)])
```

which returns the site where the gain is largest, $x = -0.5$ here – the maximum of the modulation above.

`h.get_dos()` broadens the real part whatever `eigmode` says, so `"complex"` and `"real"` give the same density of states there and `"imag"` gives the distribution of decay rates instead.

What is and is not available

The non-Hermitian path re-implements the band structure, the density of states, the LDOS and the Berry curvature; everything else on a `non_hermitian=True` Hamiltonian runs the ordinary Hermitian code, which may or may not be meaningful for a complex spectrum, so check before relying on it. The self-consistent mean field is one of those: it is not re-implemented here, so `h.get_mean_field_hamiltonian()` on a non-Hermitian Hamiltonian runs the ordinary SCF loop (see [jupyter-notebooks/functionalities/interacting_mean_field_hamiltonians/08](#)) and the density matrix it builds is the Hermitian one.

One restriction is enforced rather than left to the caller: `h.get_dos()` ac-

cepts only `mode="ED"`, and refuses `use_kpm=True` and every other mode with a `NotImplementedError`, because the Chebyshev and adaptive expansions both assume a real spectrum. `h.get_ldos()` has the same limitation – only its default `mode="diagonalization"` is implemented here – but does not enforce it, so pass no other mode.

Operators work as usual, including `operator="unfold"`, so a supercell calculation can be unfolded back onto the primitive Brillouin zone with a complex spectrum – see `examples/1d/unfolding_non_hermitian/main.py`. `num_bands` also works, selecting the few eigenvalues nearest `central_energy` with ARPACK rather than diagonalizing fully.

See `examples/1d/NH_ldos/main.py` (the model above, resolved mode by mode), `examples/0d/non_hermitian_aah/main.py` and `examples/0d/non_hermitian_aah_dos/main.py` (a non-Hermitian Aubry-Andre chain swept over the modulation phase), and `tests/nonhermitian/` for the invariants these paths are held to.

Superconductivity

Up to now we have focused on Hamiltonians that contain only normal terms, namely that the full Hamiltonian can be written as

$$H_0 = \sum_{ijss'} t_{ijss'} c_{i,s}^\dagger c_{j,s'}$$

where ij runs over sites and ss' over spins.

In the presence of superconductivity, an anomalous term appears in the Hamiltonian taking the form

$$H_{SC} = \sum_{ijss'} \Delta_{ij}^{ss'} c_{i,s} c_{j,s'} + h.c.$$

To solve the Hamiltonian

$$H = H_0 + H_{SC}$$

we define a Nambu spinor that takes the form

$$\Psi_n = \begin{pmatrix} c_{n,\uparrow} \\ c_{n,\downarrow} \\ c_{n,\downarrow}^\dagger \\ -c_{n,\uparrow}^\dagger \end{pmatrix}$$

and rewrite the Hamiltonian as

$$H = \Psi^\dagger \mathcal{H} \Psi$$

where $\mathcal{H}$ is the nambu Hamiltonian. In this new basis, the Hamiltonian can be written in a diagonal form as

$$H = \sum_{\alpha} \epsilon_{\alpha} \Psi_{\alpha}^{\dagger} \Psi_{\alpha}$$

where ϵ_{α} are the Nambu eigenvalues.

s-wave superconductivity

The simplest form of superconductivity is spin-singlet s-wave superconductivity. A minimal superconducting term of this form can be written as

$$H_{SC} = \Delta_0 \sum_n c_{n,\uparrow} c_{n,\downarrow} + h.c.$$

In the following, we address the electronic structure of a triangular lattice with s-wave superconductivity, whose Hamiltonian takes the form

$$H = H_0 + H_{SC}$$

with

$$H_0 = \sum_{\langle ij \rangle} c_i^{\dagger} c_j + h.c.$$

The previous Hamiltonian can be computed for $\Delta_0 = 0.2$ as

```
from pyqula import geometry
g = geometry.triangular_lattice() # geometry of the 2D model
h = g.get_hamiltonian() # generate the Hamiltonian
h.add_swave(0.2) # add s-wave superconductivity
(k,e) = h.get_bands() # compute band structure
```

Note that due to the BdG nature of the Hamiltonian, the bandstructure shows both the electron and hole states

Spin-triplet d-vector and non-unitary superconductivity

For a spin-triplet superconductor the pairing is a symmetric 2×2 matrix in spin space, which is conventionally parametrized by a complex three-component d-vector $\vec{d}$ as

$$\Delta = i(\vec{d} \cdot \vec{\sigma})\sigma_y = \begin{pmatrix} -d_x + id_y & d_z \\ d_z & d_x + id_y \end{pmatrix}$$

so that d_z is the equal-spin-antiparallel $\Delta_{\uparrow\downarrow}$ component and d_x, d_y encode the equal-spin $\Delta_{\uparrow\uparrow}, \Delta_{\downarrow\downarrow}$ ones. Only $\Delta_{\uparrow\downarrow}$ can also host a spin-singlet contribution, which is antisymmetric under exchanging the two sites; pyqula projects it out before reading off d_z , so a state with both singlet and triplet order still gives the triplet d-vector alone.

The product

$$\Delta\Delta^\dagger = |\vec{d}|^2 \mathbb{1} + \vec{q} \cdot \vec{\sigma}, \quad \vec{q} = i(\vec{d} \times \vec{d}^*)$$

splits the state into two classes. When $\vec{q} = 0$ the state is **unitary**: $\Delta\Delta^\dagger$ is proportional to the identity, the two quasiparticle branches are degenerate, and the Cooper pairs carry no net spin. When $\vec{q} \neq 0$ the state is **non-unitary**: the gap is different for the two spin branches and the pairs carry a spin moment $\vec{q}$, which is real by construction (the cross product of a vector with its own conjugate is purely imaginary). A $\vec{d}$ with all components sharing a common phase, i.e. a $\vec{d}$ that is a real vector times a global phase, is always unitary; non-unitarity requires a relative phase between components, as in $\vec{d} \propto (1, i, 0)$, which is a pure $\Delta_{\uparrow\uparrow}$ pairing with $\vec{q}$ along $+z$. Non-unitary states therefore require broken time-reversal symmetry, and appear naturally in ferromagnetic superconductors, where $\vec{q}$ is parallel to the magnetization because the pairs form in the majority band.

`h.get_dvector_non_unitarity()` returns $\vec{q}$ per site, as an array of shape (number of sites, 3), with $\vec{d}$ evaluated on a uniform k-mesh, $\vec{q}$ averaged over that mesh and the pairing partners of each site summed over. As an illustration, an explicit p-wave pairing with $\vec{d} \propto (1, i, 0)$ is non-unitary with the pair spin along $+z$

```
from pyqula import geometry
g = geometry.chain() # geometry of the 1D model
h = g.get_hamiltonian() # generate the Hamiltonian
h.setup_nambu_spinor() # initialize the Nambu basis
h.add_pairing(delta=0.3, mode="pwave", d=[1., 1j, 0.]) # up-up pairing
q = h.get_dvector_non_unitarity(nk=20) # -> [[0., 0., 0.36]]
```

whereas the same p-wave pairing with a real d-vector, $\vec{d}=[1., 0., 0.]$, is unitary and returns zero. Non-unitary states also arise on their own from a self-

consistent calculation: adding attractive first-neighbor interactions on top of a large Zeeman splitting, as in the example of the “Long range interactions” section, generates a spin-triplet order whose $\vec{q}$ follows the magnetization

```
from pyqula import geometry
g = geometry.triangular_lattice() # generate the geometry
h = g.get_hamiltonian() # create Hamiltonian of the system
h.add_exchange([0.,0.,3.]) # add exchange field
h.setup_nambu_spinor() # initialize the Nambu basis
# perform a superconducting non-collinear mean-field calculation
h = h.get_mean_field_hamiltonian(V1=-1.0,filling=0.3,mf="random",nk=4)
q = h.get_dvector_non_unitarity() # antiparallel to +z, as is h.get_magnetization()
```

For an inhomogeneous system, `h.write_non_unitarity()` writes the same quantity as a real-space map in `NON_UNITARITY_MAP.OUT`, with columns (x, y, z, q_x, q_y, q_z) . If all that is needed is whether the state is non-unitary at all, and not in which direction, `h.get_average_dvector(non_unitarity=True)` returns the k-averaged squared components of $\vec{q}$, a magnitude that carries no sign; the same method with its default `non_unitarity=False` returns $(|d_x|^2, |d_y|^2, |d_z|^2)$ and is what `superconductivity.identify_superconductivity` uses to label a state.

Superfluid weight and BKT temperature

A finite pairing amplitude does not by itself make a superconductor: what carries the supercurrent is the *superfluid weight* (superfluid stiffness)

$$D_s^{ab} = \frac{1}{V} \frac{\partial^2 \Omega}{\partial Q_a \partial Q_b}$$

the rigidity of the grand potential against winding the phase of the order parameter, at frozen $|\Delta|$, with $\mathbf{Q}$ a Cartesian twist wavevector. pyqula evaluates the general multiband BdG expression of Liang *et al.*, PRB **95**, 024515 (2017) – a Kubo current-current plus diamagnetic sum over the Bogoliubov spectrum – as the primary result, and can split it into a *conventional* part built from band velocities and a *quantum-geometric* part built from interband current matrix elements. In the isolated-flat-band limit the latter reduces to the integral of the quantum metric (Peotta & Törmä, Nat. Commun. **6**, 8944 (2015)): a flat band has no velocity, so its entire stiffness is geometric, which is why a flat band can superconduct at all. The quantum-metric integral is *not* used as a definition – the decomposition is offered on top of the Kubo result and is refused with a `ValueError`, rather than silently reported, when its assumptions (uniform on-site pairing, time-reversal symmetry, a resolvable normal-state gap) do not hold.

```
from pyqula import geometry
g = geometry.square_lattice() # geometry of the 2D model
```

```

h = g.get_hamiltonian() # generate the Hamiltonian
h.add_onsite(-0.6) # move away from half filling
h.add_swave(0.3) # BdG Hamiltonian with an s-wave gap

D = h.get_superfluid_weight(nk=20) # Cartesian (dim,dim) tensor
out = h.get_superfluid_weight(nk=20,decompose=True)
print(out["total"],out["conventional"],out["geometric"])
print(h.get_bkt_temperature(nk=20)) # Nelson-Kosterlitz criterion

```

The twist is the physical Peierls substitution with the *full* bond vector $\mathbf{R}+\mathbf{r}_j-\mathbf{r}_i$, not just the lattice vector $\mathbf{R}$. This matters: with the lattice vector alone, D_s comes out anisotropic on the honeycomb lattice (0.220 vs 0.173, forbidden by C3) and changes when the very same crystal is redescribed with a supercell, while the full bond vector gives an isotropic, supercell-invariant answer. The two agree only for cells holding a single orbital. `gauge="lattice"` selects the other convention, which is the one used in the Peotta/Törmä literature and by pyqula's own `h.get_quantum_metric()` – the fixed- $|\Delta|$ superfluid weight genuinely depends on the orbital embedding, see Huhtinen, Herzog-Arbeitman, Chew, Bernevig & Törmä, PRB **106**, 014518 (2022).

In two dimensions `h.get_bkt_temperature()` solves the Nelson-Kosterlitz criterion $T_{\text{BKT}} = (\pi/8)D_s(T_{\text{BKT}})$ self-consistently by bisection, at frozen $|\Delta|$ (there is no $\Delta(T)$ feedback, so it is an upper estimate). Setting `mode="finite_difference"` differentiates the grand potential numerically instead: much slower, but assumption-free, and it is the oracle the analytic route is tested against in `tests/superfluid/`. One caveat worth knowing: at $T = 0$ with a *gapless normal state* and zero or tiny pairing, the paramagnetic/diamagnetic cancellation is carried by a $-\partial f/\partial E$ that collapses to a delta function, which a finite k-mesh cannot resolve, so D_s comes out at the normal state's Drude weight rather than zero; use a temperature the mesh resolves when checking that a marginal state has no stiffness. See `examples/2d/superfluid_weight/main.py` for a runnable version and `src/pyqula/sctk/superfluidweight.py` for the implementation.

Interactions at the mean-field level

In this section we address how interactions can be treated at the mean-field level.

The collinear Hubbard model

We will start with the simplest interaction term, a local repulsive interaction in a spinful system. Our full Hamiltonian takes the form

$$H = \sum_{\langle ij \rangle} c_i^\dagger c_j + h.c. + U \sum_i c_{i,\uparrow}^\dagger c_{i,\uparrow} c_{i,\downarrow}^\dagger c_{i,\downarrow}$$

The interaction term $U \sum_i c_{i,\uparrow}^\dagger c_{i,\uparrow} c_{i,\downarrow}^\dagger c_{i,\downarrow}$ is solved at the mean-field level. The mean-field approximation consists on replacing the previous four fermion operator, by all the terms that arise by taking expectation value in two of the fermions. In particular, in its simplest collinear form, the mean-field term takes the form

$$H_U^{MF} = U \sum_i \langle c_{i,\uparrow}^\dagger c_{i,\uparrow} \rangle c_{i,\downarrow}^\dagger c_{i,\downarrow} + c_{i,\uparrow}^\dagger c_{i,\uparrow} \langle c_{i,\downarrow}^\dagger c_{i,\downarrow} \rangle$$

where $\langle \rangle$ denotes the ground state expectation value of those operators. The full Hamiltonian thus takes the form

$$H^{MF} = \sum_{\langle ij \rangle} c_i^\dagger c_j + h.c. U \sum_i \langle c_{i,\uparrow}^\dagger c_{i,\uparrow} \rangle c_{i,\downarrow}^\dagger c_{i,\downarrow} + c_{i,\uparrow}^\dagger c_{i,\uparrow} \langle c_{i,\downarrow}^\dagger c_{i,\downarrow} \rangle$$

As a result, the mean-field Hamiltonian depends on the specific ground state of the system, and the ground state depends of course on the specific mean-field Hamiltonian. The previous circular dependence between the ground state and the mean-field Hamiltonian gives rise to a selfconsistent problem.

This selfconsistent condition is solved as follows. We start with an initial guess for the full many-body ground state, that we call $|GS_0\rangle$. With this initial state, we compute the mean-field Hamiltonian H_0^{MF} . This mean-field Hamiltonian allows to compute a new many-body ground state $|GS_1\rangle$, which in turn allows to compute a new mean-field Hamiltonian H_1^{MF} . The previous algorithm is represented as

$$|GS_0\rangle \rightarrow H_1^{MF} \rightarrow |GS_1\rangle \rightarrow H_1^{MF} \rightarrow |GS_2\rangle \rightarrow H_2^{MF} \rightarrow \dots$$

This iterative calculation is performed until $H_n^{MF} = H_{n+1}^{MF}$, at which point the algorithm has converged.

Two important notes can we taken from the previous approach. First, the final solution may be sensitive to the initial guess for the ground state. This guess corresponds to the initialization of the Hamiltonian, and it can be important for system whose energy landscape has several local minima. A second point is that the update procedure from one iteration to the next can be done adiabatically, or very suddenly. This corresponds to the mixing between solutions, and for systems close to the critical point can lead to tricky convergence.

Let now show an example of a mean-field calculation. We will take now a square lattice, make a 2x2 supercell and include local repulsive interactions at half filling. The obtained ground state is an antiferromagnetic Neel state that opens a gap at half filling

```
from pyqula import geometry
g = geometry.square_lattice() # geometry of a square lattice
```

```

g = g.get_supercell([2,2]) # generate a 2x2 supercell
h = g.get_hamiltonian() # create hamiltonian of the system
h = h.get_mean_field_hamiltonian(U=2.0,filling=0.5,
                                mf="random") # perform SCF
(k,e) = h.get_bands() # calculate band structure
m = h.get_magnetization() # get the magnetization

```

Non-collinear Hubbard model

In the mean-field ansatz considered above, only a single term in the Wick contraction was considered. This term is the collinear term in the z-direction, and allows accounting for solutions that have magnetization in the z-direction. However, in the presence of frustration, external magnetic field or spin-orbit coupling, the magnetization of a system may be non-collinear and pointing in an arbitrary direction. To account for that phenomenology, the mean-field Hamiltonian must include the non-collinear term that takes the form

$$H_U^{ncMF} = -U \sum_i \langle c_{i,\downarrow}^\dagger c_{i,\uparrow} \rangle c_{i,\uparrow}^\dagger c_{i,\downarrow} + h.c.$$

When including this additional term, the full mean-field Hubbard Hamiltonian is rotationally invariant, meaning that it respects SO(3) spin rotational symmetry. This rotationally symmetric form is the default form implemented in the library.

With the previous point in mind, we now solve a system that develops a non-collinear magnetic state. We take the square lattice considered in the section above, and we add an external magnetic field. The competition between Zeeman energy and antiferromagnetic correlations gives rise to a canted magnetic state

```

from pyqula import geometry
g = geometry.square_lattice() # geometry of a square lattice
g = g.get_supercell([2,2]) # generate a 2x2 supercell
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_zeeman([0.,0.,0.1]) # add out-of-plane Zeeman field
h = h.get_mean_field_hamiltonian(U=2.0,filling=0.5,
                                mf="random") # perform SCF
(k,e,c) = h.get_bands(operator="sz") # calculate band structure
m = h.get_magnetization() # get the magnetization

```

Superconducting mean-field

In the cases above we focused on local repulsive interactions that promote collinear or non-collinear magnetism. However, local interactions can promote a different type of symmetry breaking, in particular gauge symmetry breaking associated to superconductivity. The emergence of superconductivity is associated to one of the Wick contractions of the mean-field, the anomalous term, that takes the form

$$H_U^{aMF} = U \sum_i \langle c_{i,\uparrow} c_{i,\downarrow} \rangle c_{i,\downarrow}^\dagger c_{i,\uparrow}^\dagger + h.c.$$

The previous term in the mean-field Hamiltonian can become non-zero for $U < 0$, and yields an interaction induced superconducting state. This term in the mean-field Hamiltonian is automatically accounted for in Hamiltonian with Nambu degree of freedom, of course apart from the collinear and non-collinear terms in the mean-field. We show below how an interaction induced superconducting state can be computed with pyqula

```
from pyqula import geometry
import numpy as np
g = geometry.triangular_lattice() # geometry of a triangular lattice
h = g.get_hamiltonian() # get the Hamiltonian
h.setup_nambu_spinor() # setup the Nambu form of the Hamiltonian
h = h.get_mean_field_hamiltonian(U=-1.0,filling=
                                0.15,mf="swave") # perform SCF
# electron spectral-function
h.get_kdos_bands(operator="electron",nk=400,
                 energies=np.linspace(-1.0,1.0,100))
```

Long range interactions

Up to now we have considered interacting Hamiltonians that only have local (attractive or repulsive) Hubbard interactions. In the following we are going to consider systems that have many-body interactions also to a certain number of neighbors. Long range interactions are crucial to stabilize specific symmetry broken states, and in particular charge density waves, Peierls instabilities and unconventional superconductivity. The full Hamiltonian we will consider takes the form

$$H = \sum_{\langle ij \rangle} c_i^\dagger c_j + h.c. + U \sum_i c_{i,\uparrow}^\dagger c_{i,\uparrow} c_{i,\downarrow}^\dagger c_{i,\downarrow} + V_1 \sum_{\langle ij \rangle, s, s'} c_{i,s}^\dagger c_{i,s} c_{j,s'}^\dagger c_{j,s'}$$

where U parametrizes onsite interactions and V_1 interactions between first neighbors. The previous Hamiltonian gives rise to a variety of terms when performing a mean-field decoupling. By default, pyqula includes all the Wick contractions of the mean-field, and in the presence of Nambu spinors it includes all the anomalous contractions. Let us now briefly elaborate on some of the additional terms that arise due to the first neighbor interaction V_1 .

The first term is the charge order term, that takes the form

$$H^{MF} \sim \langle c_{i,s}^\dagger c_{i,s} \rangle c_{j,s'}^\dagger c_{j,s'}$$

this term can give rise to a different charge imbalance between different sites, and it leads to charge density wave states.

The second term we consider is the bond order, that takes the form

$$H^{MF} \sim \langle c_{i,s}^\dagger c_{j,s} \rangle c_{j,s}^\dagger c_{i,s} + h.c.$$

which leads to an interaction-enhanced hopping. If this happens in a non-uniform way in the system, the resulting state has a Peierls distortion.

Among the anomalous terms, the mean-field Hamiltonian can generate

$$H^{MF} \sim \langle c_{i,\uparrow}^\dagger c_{j,\uparrow}^\dagger \rangle c_{j,\downarrow} c_{i,\downarrow} + h.c.$$

$$H^{MF} \sim \langle c_{i,\downarrow}^\dagger c_{j,\downarrow}^\dagger \rangle c_{j,\uparrow} c_{i,\uparrow} + h.c.$$

$$H^{MF} \sim \langle c_{i,\uparrow}^\dagger c_{j,\downarrow}^\dagger \rangle c_{j,\downarrow} c_{i,\uparrow} + h.c.$$

where the first two-terms corresponds to the odd superconducting order, and the third term account both for even and odd orders.

Below, we show an example in which an interaction-induced spin-triplet term is generated. By considering a electronic structure with a large Zeeman splitting and attractive first neighbor interactions, a state with non-zero $\Delta_{\uparrow\uparrow}$ and $\Delta_{\downarrow\downarrow}$ emerges.

```
import numpy as np
from pyqula import geometry
g = geometry.triangular_lattice() # generate the geometry
h = g.get_hamiltonian() # create Hamiltonian of the system
h.add_exchange([0.,0.,1.]) # add exchange field
h.setup_nambu_spinor() # initialize the Nambu basis
# perform a superconducting non-collinear mean-field calculation
h = h.get_mean_field_hamiltonian(V1=-1.0,
                                filling=0.3,mf="random")
# electron spectral-function
h.get_kdos_bands(operator="electron",nk=400,
                 energies=np.linspace(-2.0,2.0,400))
```

Spin-spin exchange interactions

The interactions considered above are all of density-density type, $c_{i,s}^\dagger c_{i,s} c_{j,s'}^\dagger c_{j,s'}$. A different kind of interaction, relevant for localized-moment magnetism, is a direct spin-spin coupling $\vec{S}_i \cdot \vec{S}_j$ between the (Pauli) spin operators at two sites,

$$H = J_z \sum_{\langle ij \rangle} S_i^z S_j^z + J_x \sum_{\langle ij \rangle} S_i^x S_j^x + J_y \sum_{\langle ij \rangle} S_i^y S_j^y$$

with $J > 0$ the antiferromagnetic (Heisenberg) sign convention and $J < 0$ favoring a ferromagnetic instability. Writing $S_i^z = (n_{i,\uparrow} - n_{i,\downarrow})/2$,

$$S_i^z S_j^z = \frac{1}{4} (n_{i,\uparrow} n_{j,\uparrow} - n_{i,\uparrow} n_{j,\downarrow} - n_{i,\downarrow} n_{j,\uparrow} + n_{i,\downarrow} n_{j,\downarrow})$$

is already a density-density interaction between spin-orbitals, so $S_i^z S_j^z$ is solved with exactly the same Hartree-Fock machinery as the $U/V_1/V_2/V_3$ interactions above – `h.get_szsz_mean_field_hamiltonian(J1=...)` (first-neighbor J_z ; J_2/J_3 add second/third neighbors, J_r a general distance-dependent coupling, following the same convention as $V_1/V_2/V_3/V_r$ in `get_mean_field_hamiltonian`). $S_i^x S_j^x$ and $S_i^y S_j^y$ are obtained by a global spin rotation that maps the x (or y) axis onto the computational z axis, solving the $S^z S^z$ problem there, and rotating the converged Hamiltonian back – `h.get_sxsx_mean_field_hamiltonian(...)` and `h.get_sysy_mean_field_hamiltonian(...)`. Because the bare interaction is $SU(2)$ -symmetric, the converged total energy of $SzSz$, $SxSx$ and $SySy$ at the same coupling only differs by which axis the moment orders along.

```
from pyqula import geometry
g = geometry.chain() # a chain, prone to ferromagnetic order away from half filling
h = g.get_hamiltonian(has_spin=True)
h = h.get_szsz_mean_field_hamiltonian(J1=-2.0,filling=0.2,
                                     mf="ferroZ", # ferromagnetic Sz-Sz coupling
                                     nk=10,mix=0.3,maxite=300)
m = h.get_magnetization() # uniform moment along z
```

Getting these SCF loops to converge. All the mean-field entry points return `None` instead of a Hamiltonian when the loop does not converge, so check the result before using it. Two defaults are worth overriding explicitly for a partially filled metal like this chain:

- `maxite=None` (the default) means *no iteration limit*, so a loop that settles into a limit cycle instead of a fixed point never returns. Always pass a finite `maxite` while exploring parameters – you then get a `None` and a “no convergence” message in a few seconds instead of a hung session
- `nk=8` (the default k-mesh) is often the actual culprit rather than the mixing. At `filling=0.2` this chain does *not* converge at `nk=8` for any mixing, because the mesh does not resolve the Fermi points and the occupied set flips between iterations; `nk=10` converges in a fraction of a second. If an SCF refuses to converge, change `nk` before reaching for a smaller `mix`

`mix` (0.1 by default, linear mixing of successive mean fields) is the knob for a loop that oscillates around a fixed point rather than one that never approaches

one; `maxerror` (1e-5) sets the convergence threshold. `tests/scf/` is a good source of known-converging parameter sets for each coupling.

The three channels can also be combined into a single anisotropic-exchange SCF loop, `h.get_exchange_mean_field_hamiltonian(Jx1=...,Jy1=...,Jz1=...)`, which decouples the z channel directly and the x/y channels through the same rotate-solve-rotate-back trick, each SCF iteration:

```
h = g.get_hamiltonian(has_spin=True)
h = h.get_exchange_mean_field_hamiltonian(Jz1=-1.0,Jx1=-0.5,
                                           filling=0.2,mf="ferroZ",
                                           nk=10,mix=0.3,maxite=300)
```

Density-density interactions ($U/V_1/V_2/V_3/V_r$, as in `get_mean_field_hamiltonian`) and spin-spin exchange can also be solved together, self-consistently, in a single combined SCF loop with `h.get_combined_mean_field_hamiltonian(U=...,V1=...,J1=...,...)`. This is not new physics: a density-density interaction and $S_i^z S_j^z$ are both density-density interactions in the spin-orbital basis (just with a different sign pattern across the four spin blocks), and the Hartree-Fock decoupling is linear in the interaction, so the density-density contribution is simply added into the same z -channel matrix the exchange term already uses. Exchange here follows a $V_1/V_2/V_3$ -like convention: $J_1/J_2/J_3$ ($+J_r$) are isotropic Heisenberg couplings, $J(S_i^x S_j^x + S_i^y S_j^y + S_i^z S_j^z)$, for the first/second/third neighbor shells, and $J_{1x}/J_{1y}/J_{1z}$ are an optional anisotropic correction added on top of J_1 for the first-neighbor shell only (e.g. the effective first-neighbor J_z coupling is $J_1 + J_{1z}$); all default to 0

```
h = g.get_hamiltonian(has_spin=True)
h = h.get_combined_mean_field_hamiltonian(U=5.0,J1=-1.0,
                                           filling=0.2,mf="ferroZ")
```

The same combined SCF loop can instead get its density matrix through a per-k Chebyshev-moment (Kernel Polynomial Method) expansion, never diagonalizing the Bloch Hamiltonian, with `integration="kpm"`:

```
h = g.get_hamiltonian(has_spin=True)
h = h.get_combined_mean_field_hamiltonian(U=5.0,J1=-1.0,filling=0.2,
                                           mf="ferroZ",integration="kpm")
```

This is meant for large/sparse systems where per-iteration exact diagonalization is the bottleneck – **but currently measures far slower than `integration="ed"` at the small/moderate sizes actually tested (order 100-500 sites, see the reference entry below for numbers)**, so benchmark before relying on it for a given system.

The same combined SCF loop can instead be solved with a JAX-derivative-based solver, `use_jax=True`: rather than the default plain-mixing loop, this builds a JAX-differentiable version of one SCF iteration $x = f(x)$ and drives it to its fixed point with a genuine root-finder using JAX-computed derivatives of f (`solver="newton"`, the default once `use_jax=True`), a matrix-free variant

that scales to larger systems (`solver="newton_krylov"`), by minimizing the squared residual $\|f(x) - x\|^2$ as a proper nonlinear least-squares problem via matrix-free Levenberg-Marquardt (`jax.jvp/ jax.vjp` plus `scipy's lsqr`, `solver="error_gradient"`), or with a robust black-box mixing scheme, `solver="broyden_mixing"` – a regularized, limited-memory multisecant form of Broyden's second method following Marks & Luke, *Robust Mixing for Ab-Initio Quantum Mechanical Calculations* (arXiv:0801.3098). Unlike the root-finder/gradient-based solvers above, it only ever evaluates f itself (no Jacobian, no autodiff), tracking the last few SCF steps as simultaneous secant conditions, regularizing the resulting least-squares solve (Tikhonov), and adaptively bounding the step length – this is the combination the paper credits for converging on cases (e.g. “charge sloshing” between two badly-scaled subsets of the mean field) that defeat a single fixed linear-mixing factor. It first runs a plain-linear- mixing warm-up (reusing `mix`) until the residual drops below a threshold (`warmup_tol`, default `1e-2`) before switching on the multisecant machinery – benchmarked across several small (5-13 atom) systems, starting the multisecant phase directly on a cold guess (the paper's own literal algorithm) regularly failed to converge, while the warm-up fixed every observed case and also converged 2-10x faster than plain linear mixing alone; see `scftk.broydenmixing's` module docstring for the algorithm and that benchmark. See the reference entry below for the full solver list and scope restrictions (normal-state only, no `constrains`), and `scftk.vjinteraction_jax's` module docstring for why `solver="error_gradient"` minimizes the SCF residual rather than the physical free energy directly.

Which solver is most likely to converge at all (as opposed to fastest) matters more than raw speed when the system's symmetry properties aren't known in advance (e.g. no explicit symmetry-breaking bias applied to the Hamiltonian). Measured across a range of system sizes and both biased and fully generic (unbiased) Hamiltonians, `solver="error_gradient"` was consistently the most robust of the four `use_jax=True` solvers, converging in nearly every case tried – including the hardest one, a larger unbiased system where `"linear_mixing"` failed outright and `"broyden_mixing"` converged only a minority of the time (outside the small-system regime it was validated on above). `"newton_krylov"` was also reliable at larger sizes but considerably slower there, and was the least robust solver of the four on small systems, where its GMRES step can fail outright against a near-singular Jacobian. Net recommendation: default to `solver="error_gradient"` for a generic system whose symmetry isn't known to be already broken; reach for `"newton_krylov"/"broyden_mixing"` instead once the system is known to be well-conditioned (or explicitly biased) and the extra speed matters more than robustness.

```
h = g.get_hamiltonian(has_spin=True)
h = h.get_combined_mean_field_hamiltonian(U=5.0,J1=-1.0,filling=0.2,
                                           mf="ferroZ",use_jax=True,
                                           solver="newton")
```

Needs the optional jax extra (`pip install pyqula[jax]`).

All of the spin-spin exchange functions above also work on BdG (Nambu) Hamiltonians (`h.turn_nambu()/h.setup_nambu_spinor()`). `get_szsz_mean_field_hamiltonian/get_sxsx_mean` need no special handling: `get_mean_field_hamiltonian`'s existing Hartree-Fock-plus-anomalous decoupling already dispatches generically for any density-density-shaped interaction, including $S_i^z S_j^z$'s. `get_combined_mean_field_hamiltonian/get_exchange_mean` decouple the exchange (J) channels with the same full normal-plus-anomalous treatment as $U/V_1/V_2/V_3$ for a Nambu Hamiltonian, so exchange can itself induce superconducting pairing, not just density-density interactions: an antiferromagnetic isotropic J alone (no U/V at all), seeded with a small coherent pairing guess (e.g. `h.add_swave(0.1)` on top of the Hamiltonian used as `mf`; a purely random guess has no reliable overlap with this instability and often relaxes back to zero pairing instead), can spontaneously decouple into a purely superconducting, singlet-paired state (the same RVB-like mechanism behind exchange-driven superconductivity), while the ferromagnetic sign has no such pairing tendency and stays magnetic. A state with both magnetic and superconducting order can also still emerge from combining an exchange field with an attractive V_1 . CAVEAT: the reported `total_energy` only ever subtracts the normal (Hartree-Fock) double-counting correction, never a matching one for the anomalous/pairing channel, so it is systematically off whenever any channel (exchange or V/U) converges to a nonzero pairing amplitude; the converged Hamiltonian itself is unaffected by this, only the `total_energy` scalar:

```
h = g.get_hamiltonian(has_spin=True)
h.add_exchange([0.,0.,0.3])
h.turn_nambu()
h = h.get_combined_mean_field_hamiltonian(V1=-1.0,J1z=-0.3,
                                          filling=0.3,mf="random")
```

Abrikosov-pseudofermion (spinon) mean field for Heisenberg models

Rather than seeding a spin-spin exchange on top of an existing tight-binding Hamiltonian, a pure spin- $\frac{1}{2}$ Heisenberg model $H = J \sum_{\langle ij \rangle} \vec{S}_i \cdot \vec{S}_j$ can be treated on its own terms with the Abrikosov-pseudofermion (parton) representation $\vec{S}_i = \frac{1}{2} f_i^\dagger \vec{\sigma} f_i$ (Savary & Balents, *Quantum Spin Liquids: a review*, arXiv:1601.03742, Sec. 4): each spin is written in terms of an auxiliary (“spinon”) fermion subject to the hard local constraint $f_i^\dagger f_i = 1$, exactly one fermion per site, and the exchange term is Wick-decoupled into an RVB bond order parameter $\chi_{ij} = \langle f_i^\dagger f_j \rangle$ – physically the same Fock/ Hartree-Fock decoupling `get_combined_mean_field_hamiltonian`'s J channel already performs, just on a Hamiltonian with zero bare hopping (a pure spin model has no bare electron kinetic term) and with the local constraint enforced at *every* site individually, not only on lattice average. `SpinonHamiltonian`

(pyqula.spinon) packages exactly this:

```
from pyqula import geometry
from pyqula.spinon import SpinonHamiltonian

g = geometry.triangular_lattice() # a canonical frustrated-Heisenberg lattice
h = SpinonHamiltonian(g) # zero bare hopping -- couplings come from J1/J2/...
h2 = h.get_mean_field_hamiltonian(J1=1.0,nk=12,mix=0.1,maxerror=1e-4)

h2.local_occupation # <n_i> per site -- exactly 1.0 at convergence
h2.constraint_lambda # converged per-site Lagrange multiplier (local chemical potential)
h2.get_bands() # spinon dispersion
```

filling= cannot be passed to `SpinonHamiltonian.get_mean_field_hamiltonian` – the representation is only valid at exactly one fermion per site, so it is always requested internally as the per-site array `get_combined_mean_field_hamiltonian`’s own `filling` kwarg now accepts (one target per site, instead of only a single lattice-averaged Fermi level), enforced via a per-site Lagrange multiplier warm-started and co-converged with the RVB mean field across the same SCF loop; `scf.converged` (equivalently, a non-None return here) already implies the local constraint converged to within `maxerror`, not only the mean field itself. Only the U(1) (RVB bond-only) ansatz is implemented – a Z2 ansatz (allowing the pairing/anomalous channel J can also induce, as above) would need a Nambu-doubled `SpinonHamiltonian`, not yet supported. All other `get_mean_field_hamiltonian` kwargs (`mf`, `nk`, `mix`, `maxerror`, `maxite`, `constrains`, an additional $V1/V2/V3/U$ density-density term, ...) are forwarded unchanged.

On a frustrated lattice (triangular, kagome, ...) the converged state is ansatz-dependent, not unique: several distinct self-consistent RVB flux sectors can coexist at the same J , and which one an unseeded random `mf` guess lands on is itself part of the physics, not SCF noise – “it is not possible to search for all possible self-consistent mean field solutions... calculations are usually carried out by assuming a particular decoupling scheme” (Savary & Balents, Sec. 4.1). A 1-site-unit-cell chain has a unique solution (no frustration), so repeated calls agree to within `maxerror`; on a frustrated lattice, pass an explicit `mf=` to select a definite ansatz deliberately rather than comparing energies across differently-seeded runs.

An external Zeeman/magnetic field couples to $\vec{S}_i = \frac{1}{2}f_i^\dagger \vec{\sigma} f_i$ exactly, not via any mean-field decoupling (it is already bilinear in f), so it is added as an ordinary single-particle term with the same `Hamiltonian.add_zeeman/add_exchange` used everywhere else in pyqula – call it on the `SpinonHamiltonian` instance *before* `get_mean_field_hamiltonian`:

```
h = SpinonHamiltonian(g)
h.add_zeeman([0., 0., 0.3]) # or h.add_exchange([0., 0., 0.3])
h2 = h.get_mean_field_hamiltonian(J1=1.0, nk=12)
```

`h2.get_magnetization()` # induced $\langle S \rangle$ per site

`add_zeeman`'s argument is the coefficient of $\vec{\sigma}$ (Pauli matrices), not of $\vec{S} = \vec{\sigma}/2$, so the physical field h in $H = -h \cdot S_i$ is twice the value passed in – the same convention `add_exchange` uses on an ordinary electronic Hamiltonian elsewhere in this guide. The local one-fermion-per-site constraint is a total-occupation constraint, not a spin constraint, so it stays exactly satisfied under a field while $\langle S_i \rangle$ is free to grow with it, saturating once the field dominates J (see `tests/spinon/test_spinon_zeeman.py`).

Abrikosov-pseudofermion (Read-Newns) mean field for the Kondo lattice

The Kondo lattice / periodic Anderson model – localized moments exchange-coupled to a conduction electron at the same site – is the standard minimal model of heavy fermion compounds. Following P. Coleman, *Heavy Fermions: electrons at the edge of magnetism*, arXiv:cond-mat/0612006, Sec. III.C, its Coqblin-Schrieffer form is $H = \sum_k \epsilon_k c_k^\dagger c_k + \frac{J}{N} \sum_j S_{ab}(j) c_{jb}^\dagger c_{ja}$ ($N = 2$ for a spin- $\frac{1}{2}$ moment – **not** the coefficient of a bare $J\vec{S}_j \cdot \vec{s}_j$ Heisenberg-form Kondo term, see the caveat below). Each moment is represented by an Abrikosov pseudofermion $\tilde{S}_j = \frac{1}{2} f_j^\dagger \vec{\sigma} f_j$ subject to the constraint $f_j^\dagger f_j = 1$, and the exchange term is Hubbard-Stratonovich decoupled into a self-consistent hybridization field $V_j = -\frac{J}{2} \langle f_j^\dagger c_j \rangle$ (a “composite fermion”, half electron and half spin-flip) plus a Lagrange multiplier λ_j enforcing the local constraint – physically the large- N ($N = 2$) Read-Newns saddle point of the Kondo-lattice path integral. `KondoLatticeHamiltonian` (`pyqula.kondolattice`) packages this: given a conduction-electron Hamiltonian, it fuses on a second, initially decoupled sublattice of localized f-sites (one per conduction site) with zero bare hopping, and self-consistently solves for V_j and λ_j :

```
from pyqula import geometry
from pyqula.kondolattice import KondoLatticeHamiltonian

gc = geometry.chain()
hc = gc.get_hamiltonian(has_spin=True) # conduction electrons
h = KondoLatticeHamiltonian(hc)

seed = ([0.3+0.0j],[0.0]) # (V,lam) -- see the caveat below for why
h2 = h.get_mean_field_hamiltonian(J=1.5,filling=0.15,nk=200,mf=seed)

h2.local_occupation # <n_f> per localized site -- exactly 1.0 at convergence
h2.hybridization    # converged V per localized site
h2.constraint_lambda # converged per-site Lagrange multiplier
```

J is Coleman's Coqblin-Schrieffer coupling (entering the interaction as J/N with $N = 2$), not the coefficient of a bare $J\vec{S}_j \cdot \vec{s}_j$ Heisenberg-form Kondo term

– the two differ by a numerical factor that Coleman’s Eq. 73-78 already fixes, so this class follows the paper’s convention exactly. `filling` sets a lattice-wide chemical potential *once*, from the bare ($V = 0$) bands, and holds it fixed through the SCF loop rather than re-solving it every iteration (Coleman’s Eq. 83 is a fixed- μ , grand-canonical Hamiltonian; the electron count is meant to float, even expand, as V, λ converge, Eq. 91-92) – the local $\langle n_f \rangle = 1$ constraint is enforced separately by λ_j , not by `filling`. All other `get_mean_field_hamiltonian` kwargs (`mf`, `nk`, `mix`, `maxerror`, `maxite`, `T`) are forwarded to the SCF loop unchanged.

$V = 0$ is always itself a self-consistent solution, exactly like the trivial root of the BCS gap equation – an unseeded run (`mf=None`, the default) starts there and stays there even for a J that also supports a genuine hybridized state, so a nonzero seed (as above) is generally needed to find it. Where both solutions coexist, the hybridized state is the true (lower-energy) ground state. **Avoid a filling that lands the chemical potential inside the bare f-sector’s flat, macroscopically degenerate band** (at $V = 0$, every f-orbital sits at exactly λ , so a wide range of `filling` values – roughly 0.25-0.75 for a single conduction orbital per site – all give exactly the same, numerically ill-posed starting point); `filling=0.15` above keeps μ inside the dispersing conduction band instead. **The finite Fermi-Dirac smearing T this SCF loop necessarily runs at** (needed for the λ feedback’s numerical stability – see `scftk.kondolattice.kondo_lattice_mean_field`’s docstring) turns the textbook, continuous $T_K = D e^{-1/(J\rho)}$ onset into a genuine finite-temperature Kondo crossover: below a T -dependent threshold in J , thermal smearing washes out the hybridization entirely and $V = 0$ becomes the *only* self-consistent solution, and right at the threshold V jumps directly to an $O(1)$ value rather than growing continuously from zero.

An external Zeeman/magnetic field couples exactly to both fermion species here (the conduction electron and the localized moment $\vec{S}_j = \frac{1}{2} f_j^\dagger \vec{\sigma} f_j$, already bilinear in f), so – exactly as for `SpinonHamiltonian` above – it is added as an ordinary single-particle term with `add_zeeman/add_exchange`, called on the `KondoLatticeHamiltonian` instance *before* `get_mean_field_hamiltonian`:

```
h = KondoLatticeHamiltonian(hc)
h.add_zeeman([0., 0., 0.05])
h2 = h.get_mean_field_hamiltonian(J=1.5, filling=0.15, nk=150, mf=seed)
```

`add_zeeman` applies to every site of the fused geometry, i.e. both the conduction and the f sublattice (offset in z) – pass a position- dependent callable instead of a constant vector to target only one of them. The $\langle n_f \rangle = 1$ constraint (a total-occupation, not spin, constraint) stays exact under a field. A field competes with the Kondo singlet: the self-consistent $|V|$ *shrinks* as the field grows at fixed J , and a strong enough field destroys the hybridized state – genuine physics, not a bug, and the SCF correctly reports non-convergence (`None`) there rather than a spuriously small but nonzero V , exactly the “decays toward the always-

self-consistent $V = 0$ branch” signal a subcritical J already produces above (see `tests/kondolattice/test_kondolattice_zeeman.py`).

Spatially resolved density of states

```
from pyqula import islands
g = islands.get_geometry(name="honeycomb",n=3,nedges=3) # get an island
h = g.get_hamiltonian() # get the Hamiltonian
h.get_multildos(projection="atomic") # get the LDOS
```

Electronic structure folding and unfolding

Building a supercell folds the bands of the primitive cell into the smaller supercell Brillouin zone. The inverse operation, unfolding, recovers the primitive-cell-like spectral weight of a supercell calculation (e.g. a defect, a moire pattern, or a non-primitive choice of cell), and is essential to compare supercell calculations directly against ARPES-like band structures. Unfolding is implemented as a special operator, "unfold", that projects onto the Bloch states of the primitive cell; it can be passed to any of the k-resolved observables (`h.get_bands()`, `h.get_kdos_bands()`, `h.get_multi_fermi_surface()`...). It requires the supercell to have been built keeping track of the primitive geometry

```
from pyqula import geometry
import numpy as np
g = geometry.honeycomb_lattice() # primitive geometry
n = 3
gs = g.get_supercell(n,store_primal=True) # supercell, keeping the primitive cell info
h = gs.get_hamiltonian() # Hamiltonian of the supercell
(k,e,d) = h.get_kdos_bands(operator="unfold",delta=1e-1) # unfolded spectral function
```

`d` holds the unfolded spectral weight at each $(\mathbf{k},e)$; plotting a scatter of $\mathbf{k},e$ colored/sized by `d` recovers the primitive-cell band structure out of the supercell calculation. The same `operator="unfold"` can be passed to `h.get_multi_fermi_surface()` to unfold constant-energy cuts. See `examples/2d/unfolding/main.py`, `examples/1d/unfolding/main.py` and `examples/readme_examples/unfolding_FS/main.py` for runnable versions.

Unfolding also works when atoms have been removed from the supercell (e.g. `gs = gs.remove(...)` before `gs.get_hamiltonian()`), such as a vacancy or an irregularly-shaped flake cut out of a supercell: pyqula matches each remaining atom back to its primitive-cell replica by position instead of assuming every replica is fully present, so no extra arguments are needed — `operator="unfold"` transparently falls back to this slower, defect-tolerant path whenever the supercell's atom count doesn't match a complete replication of the primitive cell, and uses the original fast path otherwise. This position match requires the remaining atoms to sit exactly where they were in the original, undefective supercell,

so don't call anything that moves atoms (e.g. `gs.center()`, a geometry relaxation) between `gs.remove(...)` and `gs.get_hamiltonian()` — doing so raises a `ValueError` rather than silently unfolding onto the wrong replica.

Unfolding also works for a general, non-diagonal/non-orthogonal supercell, built by passing a 3x3 integer matrix `M` to `get_supercell` instead of a plain `(n1,n2,...)` size (`gs.a1,gs.a2,gs.a3` become integer combinations of the primitive vectors, `gs = M @ g`). No change is needed at the unfolding call site — `get_supercell(M,...)` records, per surviving atom, which primitive replica it came from, and `operator="unfold"` reads that bookkeeping directly (both for a complete supercell and after removing atoms):

```
from pyqula import geometry
g = geometry.honeycomb_lattice() # primitive geometry
M = [[2,1,0],[0,1,0],[0,0,1]] # non-diagonal supercell matrix, det(M)=2
gs = g.get_supercell(M,store_primal=True) # supercell, keeping the primitive cell info
h = gs.get_hamiltonian() # Hamiltonian of the supercell
(k,e,d) = h.get_kdos_bands(operator="unfold",delta=1e-1) # unfolded spectral function
```

This bookkeeping-based path only supports 1D/2D lattices (matching the diagonal case); a 3x3 `M` on a 3D bulk geometry is not yet implemented.

Surface spectral functions

In this section we address how we can compute the surface spectral function of a semi-infinite system, i.e. a system that is bulk-like far from a boundary but is cleaved along one direction. This is obtained from the surface Green's function, computed with a renormalization (decimation) technique for the semi-infinite bulk

```
from pyqula import geometry
g = geometry.honeycomb_lattice() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_soc(0.05) # Kane-Mele spin-orbit coupling opens a topological gap
h.add_rashba(0.1) # break z-mirror symmetry to expose the edge states
(k,e,ds,db) = h.get_surface_kdos(delta=1e-2) # surface and bulk spectral functions
```

`ds` and `db` are, respectively, the surface and bulk spectral weight at each `(k,e)`; plotting `k,e` colored by `ds` shows the topologically-protected edge states living at the boundary, absent from the bulk spectrum `db`. See `examples/readme_examples/surface_2dTI/main.py` for a runnable version.

The `k`-integrated counterpart is `dos.surface_dos(h,...)`, which returns the surface and bulk densities of states at a set of energies. It takes an `operator=` and projects onto it, giving e.g. the spin-resolved surface DOS of a magnetized lead rather than the charge one. A momentum-dependent operator such as "valley" is refused with a `NotImplementedError` naming it, since the Green's function it is built on has already been integrated over the Brillouin zone.

This whole family – `dos.surface_dos`, `dos.dos_surface`, `dos.bulkandsurface`, `dos.surface2bulk` and the surface writers in `kdos` – reports $-\text{Im Tr } G$ without the $1/\pi$ that `h.get_dos()` applies, so their values are π times a density of states. They agree with each other; compare them among themselves rather than against `h.get_dos()`.

Twisted bilayer graphene structural relaxation

Rigidly twisting two graphene layers is only an approximation: below a few degrees of twist, the real lattice relaxes so that the energetically costly AA-stacked regions shrink and triangular AB/BA (Bernal) domains grow around them, separated by solitonic domain walls (Nam & Koshino, arXiv:1706.03908). `GrapheneGeometry` wraps any graphene multilayer `Geometry` (bilayer, twisted bilayer, twisted trilayer, ...) and adds a `.relax()` method that reproduces this effect by minimizing a phenomenological energy over an in-plane relaxation displacement field: the interlayer Generalized Stacking Fault Energy (GSFE, a closed-form periodic function of the local interlayer registry, fit to graphene’s AA/AB/BA stacking energies) plus the intralayer linear-elastic energy, both taken from Carr, Massatt, Torrisi, Cazeaux, Luskin, Kaxiras, arXiv:1805.06972, Table 1. The minimization runs entirely in-plane (no out-of-plane corrugation yet) using `jax` autodiff gradients passed to a `scipy` L-BFGS-B solver.

`GrapheneHamiltonian` builds the actual tight-binding Hamiltonian from a (relaxed or rigid) `GrapheneGeometry`, defaulting to the same distance-decaying hoppings as `specialhamiltonian.twisted_bilayer_graphene` – since those hoppings depend on the true 3D interatomic distance, the relaxed positions feed into the electronic structure automatically

```
from pyqula import specialgeometry
from pyqula.graphenetc.geometry import GrapheneGeometry
from pyqula.graphenetc.hamiltonian import GrapheneHamiltonian

g0 = specialgeometry.twisted_bilayer(m0=15) # ~2 degree twist
g = GrapheneGeometry(g0).relax() # AA area shrinks, AB/BA domains grow
h = GrapheneHamiltonian(g)
(k,e) = h.get_bands(num_bands=20)
```

See `examples/2d/graphene_relax/main.py` for a runnable version comparing the rigid and relaxed lattices, and `tests/moire/test_graphene_relax.py` for the physical invariants this is checked against (AA is the GSFE maximum and AB/BA the degenerate minima; relaxed bond lengths stay physical; the local relaxation amplitude grows monotonically as the twist angle shrinks).

Topological insulators

Here we provide a discussion of observables related with topological insulators

Topological invariants

Chern number

The Chern number characterizes two-dimensional topological insulators with broken time reversal symmetry. It is defined as

$$C = \frac{1}{2\pi} \int \Omega(\mathbf{k}) d^2\mathbf{k}$$

where Ω is the Berry curvature. The Chern number can be computed with the following code

```
from pyqula import geometry
from pyqula import kdos
g = geometry.honeycomb_lattice() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_haldane(0.05) # Add Haldane coupling
C = h.get_chern() # Chern number
```

Tensor-cross-interpolation (qtci) integration By default the Brillouin-zone integral above is a plain sum over a uniform $n_k \times n_k$ mesh, so its cost grows as n_k^2 and the accuracy is set by how finely that mesh resolves the Berry curvature. When the curvature is sharply peaked – near a gap closing, or a nearly-flat band – the mesh has to be very fine before the answer settles.

`integration="qtci"` instead evaluates the integral by *quantics tensor cross interpolation*: the integrand is treated as a function on a binary-refined grid and learned adaptively, sampling only where the function actually varies, with Gauss-Kronrod quadrature on the resulting representation. The number of evaluations then grows roughly logarithmically rather than quadratically in the effective resolution.

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian()
h.add_haldane(0.05)
C = h.get_chern(integration="qtci",nk=20) # tensor-cross-interpolated BZ integral
```

The same backend can compute the density matrix in a mean-field calculation, replacing the k-mesh sum there:

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian()
hscf,e = h.get_mean_field_hamiltonian(U=2.0,filling=0.5,mf="antiferro",
nk=8,maxerror=1e-4,return_total_energy=True,integration="qtci")
```

This is backed by `qutecipy`, a pure-Python port of `TensorCrossInterpolation.jl` vendored into `pyqula` at `src/pyqula/qutecipytk/` (MIT, no extra install needed). Both entry points are checked against their plain counterparts – the `qtci` Chern number against the analytic value on trivial and topological Haldane models, and the `qtci` density matrix against the full dense one – in `tests/topology/test_haldane_chern.py` and `tests/scf/test_densitydensity_qtci.py`. Runnable versions are in `examples/2d/chern_qtci/main.py` and `examples/2d/mean_field_qtci/main.py`. Note the density-matrix path currently supports 2D Hamiltonians only.

Z2 invariant

The Chern number characterizes two-dimensional topological insulators with time reversal symmetry. It can be computed with the following code

```
from pyqula import geometry
from pyqula import kdos
g = geometry.honeycomb_lattice() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_soc(0.05) # Add spin-orbit coupling
from pyqula import topology
z2 = topology.z2_invariant(h) # Z2 invariant
```

Quantum geometric tensor (multiorbital/multiband)

The quantum geometric tensor (QGT) generalizes the Berry curvature: its antisymmetric part is the Berry curvature and its symmetric part is the quantum metric, a measure of the distance between neighboring Bloch states. For a chosen band subspace S (e.g. the occupied bands) it is

$$Q_{ij}^{mn}(\mathbf{k}) = \sum_{l \notin S} \frac{\langle u_m | \partial_{k_i} H | u_l \rangle \langle u_l | \partial_{k_j} H | u_n \rangle}{(E_m - E_l)(E_n - E_l)}, \quad m, n \in S$$

with the quantum metric $g_{ij}^{mn} = \text{Re} Q_{ij}^{mn}$ (symmetric part) and Berry curvature $\Omega_{ij}^{mn} = -2 \text{Im} Q_{ij}^{mn}$ (antisymmetric part) recovered in the band-trace (“Abelian”) case; setting `non_abelian=True` instead returns the full band-pair-resolved (“non-Abelian”) tensor. Because only states *outside* S enter the energy denominators, this stays well defined even when S contains an exactly or nearly degenerate multiplet of bands – e.g. an exactly spin-degenerate pair, or several orbitals meeting at a high-symmetry point – which an ordinary single-band Kubo formula cannot handle; this is what makes it suitable for genuinely multiorbital/multiband tight-binding models, not just single isolated bands. `dH/dk_i` is evaluated with `pyqula`’s exact analytic multicell k -derivative (no finite-difference error), with k in the same reduced (dimensionless, period-1) coordinates as the rest of `pyqula`’s k -space code (e.g. `topology.berry_curvature`) – not Cartesian k , so the quantum metric’s absolute scale is reciprocal-lattice-dependent

if you convert to Cartesian coordinates yourself. `occ_idx`s defaults to the bands with $E < 0$, the same convention `h.get_chern()` uses, so it tracks `h.shift_fermi(...)`.

```
from pyqula import geometry
from pyqula import topology
g = geometry.honeycomb_lattice() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system (spinful by default)
h.add_haldane(0.2) # Add Haldane coupling
h.shift_fermi(0.3) # put the Fermi level safely mid-gap (gap is [-0.9,0.9])

Q = h.get_quantum_geometric_tensor(k=[0.1,0.2,0.],occ_idx=[0,1]) # at a k-point
g_metric = h.get_quantum_metric(k=[0.1,0.2,0.],occ_idx=[0,1]) # quantum metric only

# band-pair-resolved (non-Abelian) tensor of the two occupied bands
Qna = topology.quantum_geometric_tensor(h,k=[0.1,0.2,0.],occ_idx=[0,1],
    non_abelian=True)

# along a k-path, and integrated over the BZ (cross-checked against
# the independent Wilson-loop h.get_chern() in tests/topology/test_quantum_geometric_tensor.py)
inds,gpath,omegapath = topology.quantum_geometric_tensor_path(h,occ_idx=[0,1])
C = topology.chern_from_qgt(h,nk=20,occ_idx=[0,1])
```

See `examples/2d/quantum_geometric_tensor/main.py` for a runnable version, and `src/pyqula/topologytk/qgt.py` for the implementation and references. There is also an older, unrelated Green's-function/Kubo estimator of the whole-occupied-manifold quantum geometry trace (not band- or band-pair-resolved), `pyqula.topologytk.quantumgeometry.get_QG_kpath`, see `examples/2d/quantum_geometry/main.py`.

Berry curvature density in frequency space

The berry curvature in frequency space is defined as

$$\Omega(\mathbf{k}) = \int_{-\infty}^{\epsilon_F} \Xi(\mathbf{k}, \omega) d\omega$$

where Ω is the Berry curvature of the occupied bands and $\Xi(\mathbf{k}, \omega)$ is the energy-resolved Berry curvature. `topology.chern_density` integrates $\Xi(\mathbf{k}, \omega)$ over the whole Brillouin zone at a set of energies, giving the frequency-resolved Berry-curvature density and its cumulative (energy-integrated) sum

```
from pyqula import geometry
from pyqula import topology
import numpy as np
g = geometry.honeycomb_lattice() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system
```

```
h.add_haldane(0.05) # Add Haldane coupling
(es,cs,csi) = topology.chern_density(h,nk=10,es=np.linspace(-1.0,1.0,40))
```

`es` are the energies, `cs` the Berry-curvature density at each energy, and `csi` its cumulative integral. In the gap, `csi` should plateau at a value related to the total Chern number of the occupied bands, but this Green's-function-based estimator is numerically delicate (it involves a finite-difference k -derivative, so it needs a fine enough `nk/dk` and a large enough `delta` to avoid spurious peaks near quasi-degenerate k -points) and its overall sign/normalization is not guaranteed to match `h.get_chern()` – treat it as a qualitative frequency-resolved profile and cross-check any quantitative reading against `h.get_chern()`. The k -resolved counterpart at a single energy, $\Xi(\mathbf{k},\omega)$ over a full k -mesh, can be obtained with `topology.dOmega_dE_kmap(h,nk=40)`, which writes the map to `BERRY_DENSITY_KMAP.OUT`. See `examples/2d/berry_density_kmap/main.py` for a runnable version.

Berry curvature density in real-space

The berry curvature in real-space is defined as

$$\Omega(\mathbf{k}) = \int \Gamma(\mathbf{k}, \mathbf{r}) d^2\mathbf{r}$$

where Ω is the Berry curvature of the occupied bands and $\Gamma(\mathbf{k}, \mathbf{r})$ is the spatially-resolved Berry curvature. Note that this object is meaningful for periodic systems with very large unit cells. In `pyqula`, the real-space Berry curvature is obtained from a Bianco-Resta-type commutator of position and projector operators, evaluated on a large real-space (0-dimensional) supercell or island with `topology.real_space_chern`

```
from pyqula import islands
from pyqula import topology
g = islands.get_geometry(name="honeycomb",n=8,nedges=3) # a honeycomb island
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_haldane(0.05) # Add Haldane coupling
(r,c) = topology.real_space_chern(h) # spatially-resolved Berry curvature
```

`r` are the site positions and `c` the local Berry-curvature marker at each site (the call also writes `REAL_SPACE_CHERN.OUT`). See `examples/0d/real_space_chern/main.py` for a runnable version.

Chern number in real-space

The berry curvature in real-space is defined as

$$C = \int F(\mathbf{r}) d^2\mathbf{r}$$

where C is the total Chern number of the occupied bands and $F(\mathbf{r})$ is the spatially-resolved Chern number. Note that this object is meaningful for periodic systems with very large unit cells. $F(\mathbf{r})$ is the local marker computed by `topology.real_space_chern`; because it is built from a commutator $C = PXPYP - PYPXP$, its trace over the *entire* finite sample is exactly zero by construction, so summing it over every site is not how the invariant is recovered. Instead, deep in the interior of a large enough island – away from the boundary, where the local environment looks like the infinite periodic bulk – the marker plateaus at (approximately) the quantized bulk Chern number, while the edge sites carry compensating opposite-sign weight that cancels the bulk contribution exactly. This exact real-space cancellation is itself a manifestation of the bulk-boundary correspondence

```
from pyqula import islands
from pyqula import topology
import numpy as np
g = islands.get_geometry(name="honeycomb",n=8,nedges=3) # a honeycomb island
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_haldane(0.05) # Add Haldane coupling
(r,c) = topology.real_space_chern(h) # local marker, per site
r = np.array(r)
bulk = np.argsort(np.linalg.norm(r-r.mean(axis=0),axis=1))[:len(r)//4] # innermost sites
C = np.mean(c[bulk]) # bulk plateau value approximates the total Chern number
```

Topological surface states

```
from pyqula import geometry
from pyqula import kdos
g = geometry.honeycomb_lattice() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system
h.add_haldane(0.05) # Add Haldane coupling
kdos.surface(h) # surface spectral function
```

Topological markers

The real-space Berry curvature/Chern marker of the two sections above is an example of a topological marker: a local, position-resolved quantity, computable from ground-state projectors alone, that reveals a bulk topological invariant without relying on translational symmetry or a clean Brillouin zone. This makes topological markers well suited to disordered systems, finite flakes and islands, or systems with spatially varying parameters (e.g. a Haldane mass that changes sign across a boundary, or a topological insulator with dilute vacancies), where the marker density directly visualizes where the invariant is carried. See `topology.real_space_chern` above for the code that computes it.

Entanglement

Entanglement entropy and entanglement spectrum

For a Slater determinant – any non-interacting or mean-field ground state – the reduced density matrix of a spatial region A is itself the exponential of a free-fermion operator, so it is completely fixed by the one-particle correlations inside A . Diagonalizing the restricted correlation matrix $C_{ij} = \langle c_i^\dagger c_j \rangle$ ($i, j \in A$) gives occupations $\zeta_n \in [0, 1]$, from which

$$S = - \sum_n [\zeta_n \ln \zeta_n + (1 - \zeta_n) \ln(1 - \zeta_n)], \quad \xi_n = \ln \frac{1 - \zeta_n}{\zeta_n}$$

are the entanglement entropy and the single-particle entanglement spectrum (Peschel, J. Phys. A **36**, L205 (2003); Peschel & Eisler, J. Phys. A **42**, 504003 (2009)). Only a matrix the size of the region is ever diagonalized, never the exponentially large reduced density matrix.

The region is given as a list of site indices, a boolean mask, a callable on positions (the same convention as `sculpt`), or simply a fraction of the cells; spin, sublattice and Nambu components are treated as extra orbitals of a site, so a region is always specified in terms of *sites*. A periodic Hamiltonian is first folded into a ring of `nsuper` unit cells, so region A has **two** entanglement boundaries rather than one – worth keeping in mind when comparing against single-cut results in the literature.

The entanglement spectrum is where this becomes a topological probe. For a 2d Hamiltonian the momentum parallel to the cut remains a good quantum number, and $\xi_n(k_{\parallel})$ is the Li-Haldane entanglement spectrum (Li & Haldane, PRL **101**, 010504 (2008)): for a Chern insulator its mid-gap branches flow across $\xi = 0$ and count $2|C|$ ($|C|$ chiral modes per boundary, two boundaries), mirroring the model's edge spectrum, while a trivial insulator's entanglement spectrum stays gapped. This is entirely a bulk ground-state calculation – no ribbon and no open boundary is ever constructed.

```
from pyqula import geometry
g = geometry.honeycomb_lattice() # create a honeycomb lattice
h = g.get_hamiltonian(has_spin=False) # get the Hamiltonian
h.add_haldane(0.1) # Chern insulator, C = 1
print(h.get_chern(nk=20)) # 1.0

# Li-Haldane entanglement spectrum xi_n(k_par) across the BZ
(ks,xis) = h.get_entanglement_spectrum(nsuper=10,nk=101)

# entanglement entropy, per parallel unit cell, averaged over the BZ
print(h.get_entanglement_entropy(nsuper=10,nk=20))
```

Nambu/BdG Hamiltonians are handled with the full anomalous correlation matrix, whose basis doubling is divided out. Occupation is a hard $T = 0$ cut, and a level sitting exactly at the Fermi energy raises rather than silently returning the entropy of an arbitrarily chosen determinant. `tests/entanglement/` pins the absolute normalization against the $c = 1$ CFT law $S = (c/3) \ln[(L/\pi) \sin(\pi l/L)]$ of a critical chain, the area law of a gapped 2d insulator, and the Li-Haldane counting against pyqula's own `h.get_chern()`. See `examples/1d/entanglement_entropy_chain/main.py` and `examples/2d/entanglement_spectrum_haldane/main.py` for runnable versions, and `src/pyqula/entanglement.py` for the implementation and references.

Response functions

Here we discuss how response functions can be computed

Optical conductivity

The frequency-dependent conductivity tensor of a periodic Hamiltonian is computed with the Kubo-Greenwood formula, summing velocity matrix elements over a k-mesh in the Lehmann representation,

$$\sigma_{ab}(\omega) = \frac{ie^2\hbar}{N_k V_{\text{cell}}} \sum_{\mathbf{k}} \sum_{n \neq m} \frac{f_n - f_m}{E_m - E_n} \frac{v_{nm}^a v_{mn}^b}{\hbar\omega + i\eta - (E_m - E_n)}$$

following the Wannier90/postw90 convention (Yates, Wang, Vanderbilt & Souza, PRB **75**, 195121 (2007)). The full complex tensor is returned, so $\text{Re } \sigma_{xx}$ is the optical absorption, σ_{xy} the magneto-optical (Kerr/Faraday) response, and the $\omega \rightarrow 0$ limit of σ_{xy} the anomalous Hall conductivity – quantized to $-C e^2/h$ for a Chern insulator. The intraband (Drude) channel comes from the degenerate limit $(f_n - f_m)/(E_m - E_n) \rightarrow -\partial f/\partial E$ of the same sum, which also protects the formula against 0/0 on spin-degenerate multiplets; `intraband/interband` switch the two channels independently.

Results are in units of $e^2/\hbar$, so one conductance quantum e^2/h is $1/(2\pi)$ of the returned value, times a^{2-d} for a d -dimensional lattice. `T` sets the temperature and `delta` the Lorentzian broadening η . The absolute normalization is fixed by the f-sum rule, $\int \text{Re } \sigma_{aa}(\omega) d\omega = \pi W_{aa}$, with W the diamagnetic weight available as `h.get_sum_rule_weight()`; the Drude weight tensor is `h.get_drude_weight()`.

```
import numpy as np
from pyqula import geometry
g = geometry.honeycomb_lattice() # create a honeycomb lattice
h = g.get_hamiltonian(has_spin=False) # get the Hamiltonian
```

```

h.add_haldane(0.2) # Chern insulator
h.shift_fermi(0.3) # put the Fermi energy in the gap

(ws,s) = h.get_optical_conductivity(energies=np.linspace(0.,4.,100),
                                   nk=40,T=0.02,delta=0.05)
absorption = s[:,0,0].real # Re sigma_xx, the optical absorption

# the DC Hall response is quantized to the Chern number
(w0,s0) = h.get_optical_conductivity(energies=[0.],nk=40,T=0.01,delta=1e-3)
print("sigma_xy(0) in e^2/h:",2.*np.pi*s0[0,0,1].real) # -1

```

A note on the velocity operator, which matters for any multi-site unit cell: pyqula builds $H(\mathbf{k})$ in the *lattice* gauge, whose Bloch phase carries only the lattice vector $\mathbf{R}$, so dH/dk alone silently drops every *intracell* bond – on the honeycomb lattice (one of the three nearest-neighbour bonds is intracell) that visibly breaks C3 symmetry. The velocity used here is therefore the Peierls current operator built from the full bond vector $\mathbf{d}_{ij} = \mathbf{R} + \mathbf{r}_j - \mathbf{r}_i$, equivalently $v_a = \partial_a H + i[H, r_a]$, i.e. the *atomic* gauge. With it, $\sigma_{xx} = \sigma_{yy}$ to machine precision and graphene reproduces its universal $\pi e^2/4h$. Superconducting (Nambu) and 3d Hamiltonians raise `NotImplementedError`. See `examples/2d/optical_conductivity/main.py` and `examples/1d/optical_conductivity_chain/main.py`, and `src/pyqula/conductivity.py` for the implementation and references.

Charge-charge response function

The charge-charge response function for a spinless system is computed as

$$\chi(\omega, i, j) = \sum_{n,m} f(\epsilon_n)(1 - f(\epsilon_m)) \frac{\Psi_n(i)\Psi_m(j)\Psi_m^*(i)\Psi_n^*(j)}{\epsilon_n - \epsilon_m - \omega + i\delta}$$

where $f(\epsilon)$ is the Fermi-Dirac distribution

```

from pyqula import geometry
g = geometry.chain() # create honeycomb lattice
h = g.get_hamiltonian() # create hamiltonian of the system
(es,chis) = h.get_chi(q=[0.,0.,0.]) # get response function

```

Optional arguments - q: momentum transfer of the response function - energies: array of frequencies - delta: broadening - nk: number of k-points used in the Brillouin-zone integration

The charge-charge response can also be scanned over momentum transfer to build a $\chi(q, \omega)$ map

```

import numpy as np
g = geometry.chain()
h = g.get_hamiltonian()

```

```
qs = np.linspace(-1.,1.,40)
chis = [h.get_chi(q=[q,0.,0.],energies=np.linspace(-3,3,100),nk=40,delta=0.1)[1] for q in qs]
See examples/1d/charge_response/main.py for a runnable version.
```

Generic operator-operator response function

`h.get_chi` computes the charge-charge response by default, but any pair of operators can be used instead, giving the generalized response

$$\chi_{AB}(\omega, q) = \sum_{k,n,m} f(\epsilon_{k,n})(1 - f(\epsilon_{k+q,m})) \frac{\langle \Psi_{k,n} | A | \Psi_{k+q,m} \rangle \langle \Psi_{k+q,m} | B | \Psi_{k,n} \rangle}{\epsilon_{k,n} - \epsilon_{k+q,m} - \omega + i\delta}$$

```
from pyqula import geometry
import numpy as np
g = geometry.chain()
h = g.get_hamiltonian(has_spin=True)
sz = h.get_operator("sz") # any of the operators from the "Operators" section
(es,chi) = h.get_chi(A=sz,B=sz,energies=np.linspace(-2.,2.,100),nk=40,delta=0.1)
```

By default $A=B$ =identity, which recovers the charge-charge response above. Any operator from the “Operators” section (spin, valley, sublattice, location, Nambu...) can be plugged in to build the corresponding susceptibility.

RKKY response function

The RKKY (Ruderman-Kittel-Kasuya-Yosida) interaction between two magnetic impurities is the effective exchange coupling mediated by the itinerant electrons, and can be obtained from the same non-interacting response-function machinery. `rkky.rkky_map` computes it between a reference site and every other site in the system, as a function of distance

```
from pyqula import geometry
from pyqula import rkky
g = geometry.chain()
h = g.get_hamiltonian()
h.add_on_site(1.)
m = rkky.rkky_map(h,n=10,mode="LR",nk=200) # linear-response RKKY vs distance
```

Optional arguments - mode: "LR" (linear-response theory, using the same machinery as `get_chi`) or "pm" (“poor man’s”, computed by explicitly adding a small magnetic perturbation at each site and evaluating the total energy change) - n: how many neighboring cells/distances to compute - nk: number of k-points used in the Brillouin-zone integration

`m` is an array whose columns are (distance, ..., ..., RKKY energy); the RKKY energy is in the last column

```
x,e = m[:,0],m[:,3]
```

See `examples/1d/RKKY/main.py` and `examples/1d/rkky_minimal/main.py` (which compares "pm" and "LR" on the same system) for runnable versions.

Spin susceptibility and RPA

The methods above compute the bare (non-interacting) response. For an interacting system, `h.get_spinchi_ladder` and `h.get_spinchi_full` dress the same response with the random phase approximation (RPA), using the Hubbard U stored on a mean-field Hamiltonian (`h.V`, see “Interactions at the mean-field level”), giving the physically relevant spin-excitation spectrum of e.g. a magnetically ordered state

```
from pyqula import geometry
import numpy as np
g = geometry.bichain() # two sites per cell, so Neel order fits in the cell
h = g.get_hamiltonian(has_spin=True)
seed = h.copy() ; seed.add_antiferromagnetism(0.5) # symmetry-breaking seed
hmf = h.get_mean_field_hamiltonian(U=3.0,filling=0.5,mf=seed,nk=100) # magnetic state
(es,chis) = hmf.get_spinchi_ladder(energies=np.linspace(0.,2.,100),q=[0.1,0.,0.],nk=40,delta
```

- `get_spinchi_ladder` computes the transverse (S^+/S^-) response, i.e. spin-wave-like excitations
- `get_spinchi_full` computes the full (S_x, S_y, S_z) tensor response
- `RPA=True` (default) dresses the response with the interaction; `RPA=False` returns the bare response
- `h.get_qdos_iets` scans `get_spinchi_full` over a q-path instead of a single q, directly giving a spin-excitation dispersion map along high-symmetry directions (needs a 2D lattice for the "G", "K", "M"-style path labels)

```
g2 = geometry.honeycomb_lattice() # already two sublattices per cell
h2 = g2.get_hamiltonian(has_spin=True)
hmf2 = h2.get_mean_field_hamiltonian(U=3.0,filling=0.5,mf="antiferro")
qdisp = hmf2.get_qdos_iets(energies=np.linspace(0.,2.,60),qpath=["G","K","M"],nq=30,nk=20,delta
```

- `h.get_iets_ldos` instead computes the spatially resolved response at a single energy, i.e. a real-space map of the spin-flip (“inelastic tunneling spectroscopy”, IETS) signal, which combined with `h.get_ldos` gives elastic + inelastic STM-like maps

Both snippets depend on the mean-field state actually being ordered, which is worth checking with `hmf.get_vev("sz")` before reading anything into the excitation spectrum: an RPA calculation on top of an unpolarized reference state runs perfectly happily and tells you nothing about magnons. Two things decide it. The unit cell must be able to hold the order – a one-site `geometry.chain()` cell cannot represent Neel order at all, and seeding it with `mf="antiferro"` converges to exactly zero moment; `bichain` and `honeycomb_lattice` both have

the two sublattices. And U must be past the ordering transition: on honeycomb at half filling $U=2$ gives a moment of 0.002 (i.e. none) while $U=3$ gives 0.44. Note also that `get_qdos_iets`'s cost is the product of its `nq`, `nk` and energy-grid sizes, so it grows quickly – the grid above takes well under a minute, while a 80x40x100 one is closer to an hour.

See `examples/1d/rpa/main.py` (RPA spin response vs q for an antiferromagnetic chain), `examples/2d/rpa_triangular/main.py`/`examples/2d/rpa_honeycomb/main.py` (`get_qdos_iets` dispersion along a q -path) and `examples/0d/rpa_island/main.py`/`examples/0d/rpa_finite` (`get_iets_ldos` real-space IETS maps) for runnable versions.

Running the response on a GPU Every one of these RPA entry points – `get_spinchi_ladder`, `get_spinchi_full`, `get_qdos_iets`, `get_iets_ldos`, `get_rpa_kernel_poles`, `get_magnon_bands(method="rpa")` and the density-channel `get_densitychi_RPA` – bottoms out in the same Lindhard kernel, a sum over pairs of eigenstates at every k -point of the mesh. Its cost is $36N^4n_\omega$ complex multiply-adds *per k -point* for the full (S_x, S_y, S_z) response of an N -site cell, so it is what decides whether a large unit cell is feasible at all: measured on one workstation core, one k -point takes 0.035 s at $N = 10$ and 5.9 s at $N = 40$, and the N^4 scaling puts $N = 100$ at roughly four minutes *per k -point*.

That kernel can be dispatched to a GPU through `jax` instead of `numba`, with `chi_cpugpu="GPU"`, forwarded down from any of those entry points:

```
import numpy as np
from pyqula import geometry
g = geometry.bichain() # two sites per cell, so Neel order fits in the cell
h = g.get_hamiltonian(has_spin=True)
seed = h.copy() ; seed.add_antiferromagnetism(0.5) # symmetry-breaking seed
hmf = h.get_mean_field_hamiltonian(U=3.0,filling=0.5,mf=seed,nk=100)
es,chis = hmf.get_spinchi_full(q=[0.2,0.,0.],nk=8,
                              energies=np.linspace(0.01,1.0,100),delta=1e-2,
                              chi_cpugpu="GPU") # dispatch the Lindhard kernel to a GPU
qs,ws,gammas = hmf.get_magnon_bands(method="rpa",nq=20,nk=8,
                                    energies=np.linspace(0.01,2.0,200),delta=1e-2,
                                    chi_cpugpu="GPU")
```

The device path is exact, not an approximation: it rewrites the four-fold loop as one matrix multiplication per frequency over the electron-hole pairs that survive the occupation cutoff, and agrees with the `numba` kernel to the last bits of `complex128`. Like `kpm_cpugpu`, it transparently falls back to `jax`'s own CPU backend when no GPU is visible, so it is always safe to pass – but on such a machine it is a `jax`-CPU calculation and not a fast one, and the whole point of the switch is FP64 throughput, so do not read a speedup into it that was not measured on a device.

Two behaviours worth knowing. The switch refuses rather than silently falling back: `mode="trace"/"diagonal"` (`h.get_chiAB_trace`) and

`imode="adaptive"` use different kernels that are not ported, and asking for them together with `chi_cpugpu="GPU"` raises. And a q-scan (`get_magnon_bands`, `get_qdos_iets`) that normally spreads its q-points over `parallel.pcall`'s worker processes runs the loop in a single process under `chi_cpugpu="GPU"`, since several processes sharing one device is contention rather than parallelism.

Measured on a Tesla V100 (complex128, one BLAS-pinned CPU core as the baseline, `nk=4`, `n_omega=40`), the crossover sits at about $N = 7$: below it the CPU kernel wins, and above it the device pulls away as the CPU side grows like N^4 while the device time stays nearly flat – 13x at $N = 12$, 207x at $N = 32$, and 833x at $N = 64$, with the two backends agreeing to 10^{-16} relative throughout. `benchmarks/cases/rpa_spin_response.py` reproduces that sweep on any machine, and `future_development/gpu_rpa_spin_response.md` records the full measurements and what is still open.

RPA kernel poles and magnon bands

The RPA-dressed response $\chi_{RPA} = \chi(1 - U\chi)^{-1}$ diverges wherever the kernel $1 - U\chi(q, \omega)$ becomes singular: these poles are the system's collective modes (spin waves/magnons, plasmons) or, if a kernel eigenvalue crosses zero at $\omega = 0$, a sign of a Stoner/RPA instability. `h.get_rpa_kernel_poles` scans a frequency window at a fixed `q` and returns every such pole, generalizing `chi_AB_RPA` (any operators `A,B` and interaction matrix `V`, defaulting to the charge channel and $q = 0$ like the other generic response functions above):

```
from pyqula import geometry
import numpy as np
g = geometry.bichain() # two sites per cell, so Neel order fits in the cell
h = g.get_hamiltonian()
seed = h.copy() ; seed.add_antiferromagnetism(0.5) # symmetry-breaking seed
hmf = h.get_mean_field_hamiltonian(U=3.0,nk=100,mf=seed,filling=0.5)
N = len(g.r) # the response matrix is one entry per site
V = 3.0*np.identity(N) # charge-channel interaction, in site space
poles = hmf.get_rpa_kernel_poles(V=V,q=[0.1,0.,0.],
                                energies=np.linspace(0.,4.,200),delta=2e-2,nk=40)
```

Two things there are easy to get wrong. The mean field needs a unit cell that can *hold* the order being sought – a one-site `geometry.chain()` cell cannot represent Neel order at all, so seeding antiferromagnetism on it is meaningless; `bichain` gives the two sublattices, and the converged state has `hmf.get_vev("sz")` equal and opposite on them. And `V` must live in the same space as the response matrix: `get_rpa_kernel_poles` defaults to the charge channel, one entry per *site*, so `V` is $N \times N$. It is **not** `hmf.V`, which is the mean-field interaction in spin-orbital space ($2N \times 2N$) and raises a dimension error here. For the spin channel use `get_magnon_bands` below, which builds the S_x, S_y, S_z vertex from `hmf.V` itself.

`poles` is an `(npoles,2)` array, one row per collective mode found: the pole frequency and its residual imaginary part. The latter is signed (it is the kernel

eigenvalue's actual imaginary part at the crossing, which can lie on either side of the real axis) – judge how sharp/well-defined a mode is by its *magnitude*: small `abs(gamma)` means a sharp mode, large `abs(gamma)` means it is heavily damped or the crossing is numerical noise.

`h.get_magnon_bands` specializes this to the full spin channel used by `get_spinchi_full/get_iets_ldos` (the S_x, S_y, S_z tensor, with U taken automatically from the mean-field `h.V`, same convention as `get_spinchi_full`) and scans it along a q-path, directly giving the magnon dispersion of a magnetically ordered mean-field state:

```
qs,ws,gammas = hmf.get_magnon_bands(nq=40,energies=np.linspace(0.01,3.,200),delta=2e-2,nk=40)
```

Since different q-points can have a different number of poles, `qs,ws,gammas` are flat 1D arrays of equal length ready for a scatter-style dispersion plot – `qs` holds the integer index of the q-point along the path (the same convention `get_bands` uses for its k-axis), `ws` the pole frequency and `gammas` its (signed) residual imaginary part, so filtering `np.abs(gammas) < threshold` keeps only the sharp, well-defined branches. See `examples/1d/magnon_bands/main.py` for a runnable version (an antiferromagnetic Hubbard chain, showing both its acoustic and optical magnon branches).

Interactions beyond onsite

The V passed to `get_rpa_kernel_poles` is not restricted to a single onsite matrix: it can also be a real-space hopping-like dictionary `{(n1,n2,n3): matrix}`, keyed by lattice-vector offset in the same convention as `h.get_hopping_dict()`, for an interaction with support beyond the same unit cell. It is Fourier-transformed to $V(q)$ at whatever q the response is evaluated at, using the same Bloch-phase convention as the Hamiltonian's own hoppings – an extended interaction is dressed exactly like an extended hopping. For a nearest-neighbor V_1 on a chain this gives the expected $V(q) = 2V_1 \cos 2\pi q$, so the interaction is repulsive at $q = 0$ and attractive at the zone boundary:

```
import numpy as np
from pyqula import geometry
g = geometry.chain()
h = g.get_hamiltonian(has_spin=False)
N = h.intra.shape[0] # one site per cell here
I = np.identity(N)
V = {(0,0,0): 0.0*I, (1,0,0): 0.6*I, (-1,0,0): 0.6*I} # nearest-neighbor V1
poles = h.get_rpa_kernel_poles(V=V,q=[0.5,0.,0.],energies=np.linspace(0.,2.,60),
                               delta=2e-2,nk=200)
```

Note the channel: `get_rpa_kernel_poles` dresses the **charge** response by default, so a dictionary V here must be a density-density interaction in site space, of the same dimension as the response matrix `chiAB(...,mode="matrix")` returns. It is *not* the place to put a spin vertex.

In the **spin** channel the two kinds of non-onsite interaction behave very differently, and only one of them works here.

A neighbor-shell **exchange** interaction ($J_1/J_2/J_3/J_r$, isotropic or anisotropic) **does** work. That is worth spelling out because it used to be refused. The mean field for one is not Ising-like at all: **VJinteraction** builds all three channel matrices and decouples the x and y ones by rotating the density matrix into the frame where that axis is the computational z , so the converged state is a genuinely $SU(2)$ -symmetric Hartree-Fock one. What was missing was only a vertex to match it, because $\mathbf{h.V}$ is a single matrix and an isotropic J_1 and an anisotropic J_{1z} leave exactly the same one in it. The SCF now also records the three channels separately in **h.Vchannels**, the vertex is built per channel, and the two cases are both correct and distinguishable. The check is the Goldstone theorem: on a $J_1=3$ honeycomb Neel state the RPA kernel's smallest eigenvalue at $q = 0, \omega = 0$ is $4.3e-10$, while the same kernel at finite q (0.10) and on a non-magnetic reference (0.50) is order one.

A neighbor-shell **density-density** interaction is still refused, and this one is about representation rather than bookkeeping. The vertex here is site-separable, $\chi(1-V\chi)^{-1}$ with one index per site, while the rung an extended V_{ij} contributes to the spin response is its Fock term acting on the electron-hole *pair* index. There is nowhere to put it: the extraction that builds the vertex maps a spin-independent V_{ij} to exactly zero. Whether that matters depends on the converged state rather than on the interaction, which is precisely why it is refused rather than decided for you – on a Neel state V_1 's Fock term renormalizes the hopping spin-*independently* (the two sublattices swap under a spin flip), never enters the exchange splitting, and the Goldstone mode survives ($3.0e-9$ at $U=3, V_1=0.5$); on a ferromagnetic chain ordered by V_1 alone the vertex is identically zero, the kernel is the identity, and its smallest eigenvalue is 1.0 where the Goldstone theorem demands 0 – no magnon of any kind.

For that case there are two routes that are right by construction rather than by a cancellation you would have to check state by state – **method="pair"** and **method="tdhf"**, both below. A hand-built $\mathbf{h.V}$ (one not produced by an SCF, so with no **h.Vchannels** alongside it) is also refused, for the same reason the old gate existed: nothing says which interaction it came from. To use one anyway, build the vertex explicitly and call **chitk.rpa.rpa_kernel_poles_ops/chi_ops_RPA** directly, by-passing $\mathbf{h.V}$ – **tests/chi/test_rpa_nononsite_interaction.py** and **tests/scf/test_rpa_nononsite_ferro_chain.py** do exactly that, and are worked examples of the low-filling ferromagnetic (Stoner) instability a nearest-neighbor exchange drives on a chain.

The three magnon routes

h.get_magnon_bands takes a **method**, and the three cover different interactions because they keep different amounts of the ladder:

interaction	"rpa"	"pair"	"tdhf"
onsite Hubbard U	yes	yes	yes
neighbour-shell density-density $V_1, V_2, \dots$	no	yes	yes
exchange $J_1, J_2, \dots$ (isotropic or anisotropic)	yes	no	no
metallic reference	yes	yes	with <code>metal=True</code>
non-collinear (canted, spiral) state	yes	yes	yes
frequency- resolved $\chi(\omega)$	yes	yes	no (an eigenproblem)

The reason the middle row splits is a matter of which index the ladder rung is diagonal in. Writing $H_{int} = \frac{1}{2} \sum V_{ij} n_i n_j$, the transverse rung is $K_{(ij),(kl)} = -V_{ij} \delta_{ik} \delta_{jl}$ – diagonal in the **pair** index (i, j) , not the site index. An onsite U collapses that onto the pairs with $i = j$, which is exactly what the site basis of `method="rpa"` holds, and that collapse is why it is exact there. An extended V_{ij} lives in the pairs with $i \neq j$, which that basis does not have.

`method="pair"` (`chitk/pairchi.py`) keeps the pair index and solves the same ladder there, $\chi = \chi_0(1 + V\chi_0)^{-1}$ with V diagonal. The cost is set by how many pairs the interaction has rather than by N^2 – only pairs in its support enter the inversion, so a short-ranged V gives $N(z+1)$, linear in N and eight pairs for a honeycomb cell with a nearest-neighbour V . It keeps the frequency scan, needs no gap, and assumes nothing about the spin structure of the state:

```
import numpy as np
from pyqula import geometry
from pyqula.meanfield import VJinteraction
nk = 6 # the SCF and the magnon share this mesh
g = geometry.honeycomb_lattice()
hmf = VJinteraction(g.get_hamiltonian(), U=3.0, V1=0.5, filling=0.5,
                    mf="antiferro", nk=nk, maxerror=1e-10).hamiltonian
qs, ws, gammas = hmf.get_magnon_bands(method="pair", nq=20,
                                       energies=np.linspace(1e-3, 3., 400), delta=1e-3, nk=nk)
es, chi = hmf.get_transverse_spinchi(energies=np.linspace(0., 2., 100),
                                     q=[0.1, 0., 0.], delta=1e-2, nk=nk) # (Sx, Sy, Sz) x site chi(w)
```

Two terms make up the kernel, and which of them can be dropped is a good illustration of what the Goldstone residual actually tests. Differentiating the

Hartree-Fock Hamiltonian gives an *exchange* rung W_{ab} , diagonal in the pair index, and a *Hartree* rung $-W_{ac}(q)$ between diagonal pairs. Restricted to one spin-flip sector every pair has $a \neq b$, the Hartree rung drops out, and a transverse-only ladder is complete – but only for a state with a global spin quantization axis. For a non-collinear mean field the two sectors mix, dropping the Hartree rung breaks the SU(2) Ward identity, and the acoustic branch comes out gapped by 0.41 on a 120-degree triangular spiral, independent of the broadening. That is not a contradiction of the Goldstone theorem: the theorem constrains the *exact* response, and an approximation inherits it only if it is conserving. Keeping both rungs, in a basis of spin-orbital pairs that assumes no axis at all, makes the truncation conserving again – the same spiral then gives $9.4\text{e-}12$ at `delta=1e-5`, proportional to δ^2 and limited by nothing else.

The last row is why `"tdhf"` still exists: it returns magnon energies as eigenvalues, with no frequency grid and no broadening, which is what the Goldstone residual is measured on. The three agree wherever more than one applies – the acoustic magnon of a Néel honeycomb at $q = 0.1$ is 0.49165 from all of them, and 0.59492 from `"pair"` and `"tdhf"` once a V_1 is added. In a metal, `"pair"` reproduces the closed-form saturated-ferromagnet dispersion to five decimals (0.00173, 0.01291, 0.07756 at $q = 0.02, 0.05, 0.1$), as does `"tdhf"`.

Exchange goes the other way: its transverse rung $J/2(S_i^+ S_j^- + \text{h.c.})$ is a spin-flip two-body term with no density-density representation at all, so neither `"pair"` nor `"tdhf"` can carry it, while `"rpa"` handles it through the mean field's own three spin channels. For an isotropic J , use `"rpa"`.

Magnons from time-dependent Hartree-Fock

A magnon is the same kind of object as an exciton – a bound two-particle excitation of a mean-field state – built from electron-hole pairs whose electron and hole have *opposite* spin instead of the same spin. So the Bethe-Salpeter machinery of the exciton sections below already contains it, and restricting its pair basis to that spin-flip block is what `method="tdhf"` does. This is the route that handles a neighbor-shell interaction properly, because the pair index is exactly where the missing rung lives:

```
import numpy as np
from pyqula import geometry
from pyqula.meanfield import VJinteraction
nk = 6 # the SCF and the magnon share this mesh
g = geometry.honeycomb_lattice()
scf = VJinteraction(g.get_hamiltonian(), U=3.0, V1=0.5, filling=0.5,
                    mf="antiferro", nk=nk, maxerror=1e-10)
hmf = scf.hamiltonian
print(hmf.get_goldstone_residual(nk=nk)) # 2e-10: the mode is exact
qs, es = hmf.get_magnon_bands(method="tdhf", nk=nk, nq=20, n=2)
qs, es are flat 1D arrays in the same convention as the RPA get_magnon_bands
```

above (`qs` is the integer index along the q -path), and `es` is complex whenever the mean-field state is unstable against some excitation. `h.get_magnon_energies(Q=...)` gives the spectrum at a single momentum.

The test that this is right is the **Goldstone theorem**: a state that orders magnetically without spin-orbit coupling breaks $SU(2)$ spontaneously, so a uniform spin rotation costs nothing and there must be a magnon at exactly zero energy at $Q = 0$. `h.get_goldstone_residual()` measures how far the calculation is from that, as $\|Mv\|$ with v the spin generator written in the pair basis. It comes out proportional to the SCF tolerance the mean field was converged to and to nothing else – 1.8e-6, 1.8e-8, 1.8e-10 at `maxerror` 1e-6, 1e-8, 1e-10 – which is what makes it worth checking before reading any dispersion. Note it is deliberately not “the eigenvalue nearest zero”: that eigenvalue is defective, so it only converges as the *square root* of the same error (4e-5 where the residual is 2e-10), and a small imaginary part of that size on the acoustic branch is the expected signature rather than a problem.

Three constraints come with it, all of them consequences of that same Ward identity:

- **the mean field and the magnon must use the same `nk`.** A mean field converged at `nk=20` and a magnon solved at `nk=4` is not self-consistent on the magnon’s mesh, and the acoustic branch acquires a real gap (measured 0.38, against 1e-5 for matched meshes). Nothing can check this for you – the mesh the SCF ran on is not recorded on the Hamiltonian – so `get_goldstone_residual` is how to find out.
- **a metallic reference needs `metal=True`.** By default the pair basis takes the same number of occupied bands at every k -point, which a metal does not have. Passing `metal=True` decides the occupied and empty sets per k -point instead, so the number of pairs varies across the mesh – everything downstream already tolerated that. This is what covers an itinerant magnet, and in particular a ferromagnet ordered by a neighbour-shell V_1 alone, which neither route could do before. It changes nothing for a gapped reference (bit-identical pair counts and Goldstone residual), so it is safe to leave on when unsure.

Two things are different once it runs. The magnon is no longer the lowest mode – it sits inside the Stoner continuum – so `h.get_magnon_bands(method="tdhf", metal=True, by="weight")` selects branches by how much of the spin generator they carry rather than by energy, and `bsetk.spinflip.magnon_spectrum` returns the energies with those weights. And $E(q)$ is even in q only if the *occupied set* is symmetric under $k \rightarrow -k$, which on a finite mesh it need not be: with an even number of occupied points around $k = 0$ the $+q$ and $-q$ magnons genuinely differ (0.02413 against 0.00559 at $q = 0.05$ on one such mesh). That is not something any method can paper over – pick the mesh so the occupied set is symmetric.

The metallic case has an exact reference and is checked against it rather

than only against a symmetry argument: for a *saturated* ferromagnet the single-magnon sector is a two-body problem, solvable in closed form, and the TDHF magnon reproduces it to five decimals at every q (0.00173, 0.01291, 0.07756 at $q = 0.02, 0.05, 0.1$). With a symmetric occupied set the site-basis RPA agrees with both, to the same five decimals.

- **the interaction must be a density-density one.** An exchange interaction ($J_1/J_2/J_3/J_r$, or $S_z S_z$) is refused, because `h.v` holds only its Ising part – the transverse rung $J/2(S_i^+ S_j^- + \text{h.c.})$ that makes it isotropic is a spin-flip two-body term with no density-density representation, so this kernel cannot carry it. Solving the Ising part alone returns an ordinary-looking dispersion gapped by of order J (measured 1.81 at $J_1=3$), so it is rejected rather than returned. This is a limitation of the *kernel*, not of the mean field: `VJinteraction` decouples the x and y exchange channels too (by rotating the density matrix into the frame where that axis is the computational z), so an isotropic- J mean field is genuinely $SU(2)$ symmetric – and `method="rpa"`, which rebuilds the x/y vertices by replicating the z one, keeps its Goldstone mode there (measured 4.3e-10). **For isotropic exchange, use `method="rpa"`.** See `future_development/magnons_tdhf.md`.

By default the calculation restricts itself to the spin-flip block of the pair basis, which is exact for a collinear state and about eight times cheaper; a non-collinear or tilted state has no such block and the whole basis is kept instead, transparently (`channel="all"` forces that, `channel="spinflip"` refuses instead of falling back). That path is checked on a genuinely non-collinear state too – the 120-degree spiral of the triangular-lattice Hubbard model, whose Goldstone residual comes out at 2.9e-9 for an SCF converged to 1e-9. Where both routes are valid – a plain onsite Hubbard U , where the site-separable RPA vertex is exact – they agree: the acoustic magnon of the Neel honeycomb comes out at 0.4917 and 0.9037 at $q = 0.1$ and 0.2 from both, and the two share no code below the Hamiltonian.

See `examples/2d/magnon_bands_tdhf/main.py` for a runnable version and `tests/magnon/` for the Goldstone checks on a Neel antiferromagnet, a V_1 -dressed one, a tilted one, a 120-degree non-collinear spiral and a saturated ferromagnet, plus that cross-check.

Density (charge) response

`h.get_densitychi_RPA` and `h.get_plasmon_bands` are the density-density (charge) channel analogs of `get_spinchi_full/get_magnon_bands`, for a $V_1/V_2/V_3$ -neighbor-shell (+ onsite U , + a general $V_r(r)$) density-density interaction, same convention as `Vinteraction/VJinteraction`. Unlike the spin-channel functions, they take the interaction directly as parameters instead of reading it from `h.v`, so no mean-field convergence is needed first – they dress the bare susceptibility of whatever Hamiltonian is passed in (which can also be

an already-converged one, if the RPA response about that reference state is wanted):

```
import numpy as np
from pyqula import geometry
g = geometry.chain()
h = g.get_hamiltonian(has_spin=True) # 1D chain at half filling
qs,ws,gammas = h.get_plasmon_bands(V1=0.6,qpath=[[0.3,0.,0.],[0.4,0.,0.],[0.5,0.,0.]],nq=3,
                                   energies=np.linspace(0.,1.,100),delta=2e-2,nk=2000)
```

A 1D chain at half filling has perfect Fermi-surface nesting at $q = \pi$, strongly enhancing the static charge susceptibility there – the charge-channel analog of the low-filling ferromagnetic instability above, driven by a repulsive $V1$ instead (see `tests/chi/test_plasmon_bands.py`).

Excitons and the Bethe-Salpeter equation

The RPA sections above dress a response function at fixed frequency. The Bethe-Salpeter equation (BSE) instead solves the two-particle problem directly, by diagonalizing the electron-hole pair Hamiltonian, which gives the exciton energies *and* the electron-hole amplitudes that say what each exciton is made of. This matters because a mean-field band structure can only ever absorb above its gap: the electron-hole interaction binds the pair, and the exciton appears below the gap by its binding energy.

The exciton at center-of-mass momentum Q is written as a superposition of the mean field’s own transitions,

$$|X\rangle_Q = \sum_{v,c,k} A_{vc}(k) c_{c,k+Q}^\dagger c_{v,k} |MF\rangle$$

and the BSE is the eigenvalue problem for the amplitudes $A_{vc}(k)$, with a kernel made of a *direct* term (the screened electron-hole attraction, which binds) and an *exchange* term (which splits singlet from triplet and, on its own, reproduces the RPA). The formalism is the localized-orbital (“point-like orbitals”) BSE of the Xatu code, arXiv:2307.01572, solved here in its full non-Tamm-Dancoff form.

By default the interaction is read straight off a converged mean-field Hamiltonian (`h.V`), so the same interaction that generated the Fock self-energy inside `h` also generates the BSE kernel and nothing is double counted – this is time-dependent Hartree-Fock on top of Hartree-Fock:

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian(has_spin=True)
h.add_sublattice_imbalance(0.6)
```

```
hmf = h.get_mean_field_hamiltonian(U=1.5,filling=0.5,mf="antiferro",nk=6) # converge a mean
es = hmf.get_exciton_energies(nk=6,n=8) # the eight lowest excitons
```

A purely onsite Hubbard U is far too short ranged to bind a Wannier-Mott exciton in two dimensions, so realistic exciton calculations want a long-ranged interaction instead. `bsetk.interaction.density_interaction` builds one from the same $U/V1/V2/V3/Vr$ parameters `Vinteraction` uses, and it can be handed to any of the exciton methods through `V=`:

```
import numpy as np
from pyqula.bsetk.interaction import density_interaction
h2 = g.get_hamiltonian()
h2.add_sublattice_imbalance(1.0) # a gapped semiconductor
W = density_interaction(h2,Vr=lambda r1,r2: 0.8/np.sqrt((r1-r2).dot(r1-r2)+0.25)) # Coulomb
bse = h2.get_bse(V=W,nk=8)
print(bse.get_energies()[0], bse.get_binding_energies()[0]) # lowest exciton, and how far b
```

- `h.get_exciton_energies` returns the exciton energies, `h.get_exciton_binding_energies` how far below the lowest independent-particle transition each one lies (positive means bound), and `h.get_exciton_states` both the energies and the amplitudes $A_{vc}(k)$
- `h.get_bse` returns the full solved object, whose `pairs` attribute holds the k -mesh, the band window and the `(ik,iv,ic)` label of every pair index, so the amplitudes can be resolved in momentum or by band
- `Q=[qx,qy,qz]` gives the excitons at finite center-of-mass momentum, and `h.get_exciton_bands` scans that over a q -path to give the exciton band structure (below); `nv/nc` restrict the calculation to the `nv` highest valence and `nc` lowest conduction bands
- `kernel="full"` (default) uses both kernel terms; `"direct"` is the ladder alone (no singlet/triplet splitting), `"exchange"` is exactly the RPA, and `"none"` collapses the spectrum onto the bare transition energies
- `tda=True` applies the Tamm-Dancoff approximation, diagonalizing only the resonant block: four times smaller and Hermitian, and a good approximation at weak coupling

Since the Hamiltonian is spinful, spin is simply part of the orbital index, so singlet and triplet excitons come out of a single calculation with no separate spin channel – the exchange term is what splits them. On a spin-rotation-invariant reference the lowest transition starts out four-fold degenerate and the full kernel resolves it into a three-fold triplet with the singlet pushed up above it, which neither kernel term produces on its own (see `tests/bse/test_bse_physics.py`).

One caveat on the default `V=None`: `h.V` does not capture an *anisotropic* exchange run (`J1x/J1y` alongside `J1z` store only the z channel), and after `SxSx/SySy` it is left in the internally-rotated spin frame while the returned Hamiltonian is rotated back. In either case build the interaction explicitly and pass it as `V=` instead.

The size of the problem is $N_{pair} = n_v n_c N_k$, and the matrix is dense and $2N_{pair}$

square, so the k-mesh is the expensive knob: `max_memory` (default 2 GB) refuses a calculation that would not fit rather than letting it exhaust memory. A gapped reference state is required – a metallic filling has no well-defined electron-hole pair basis and is rejected, as are Nambu/BdG Hamiltonians, whose two-particle structure is different.

Large k-meshes: solving the BSE without its matrix

The size of the problem is the reason the previous paragraph exists, and the k-mesh is exactly the knob that decides whether an exciton is converged at all. A tightly bound exciton is broad in the Brillouin zone and converges on a coarse mesh; a shallow Wannier-Mott one, whose envelope $A(k)$ is sharply peaked, does not, and that is the case the dense solver cannot reach.

It does not have to be reached with a matrix. Because a real-space interaction Fourier transforms as $W_{ab}(k - k') = \sum_d W_{ab}(d) e^{2\pi i k \cdot d} e^{-2\pi i k' \cdot d}$, the direct term of the kernel separates exactly into one rank-one term per non-zero entry (a, b, d) of that interaction – a *fixed* number, set by how far the interaction reaches and not by the mesh. The exchange term is already a product of density form factors. So the whole resonant block is a diagonal plus a fixed-rank correction, and can be applied to a vector without being built. `solver=` chooses what to do with that:

```
import numpy as np
from pyqula import geometry
from pyqula.bsetk.interaction import density_interaction
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian()
h.add_sublattice_imbalance(1.0) # a gapped semiconductor
W = density_interaction(h, Vr=lambda r1, r2: 0.8/np.sqrt((r1-r2).dot(r1-r2)+0.25))
h.get_bse(V=W, nk=1024, tda=True, solver="iterative", neig=4) # no matrix
h.get_bse(V=W, nk=65536, tda=True, solver="qtt", neig=1)      # no mesh either
```

- `solver="dense"` (default) is the behavior described above and is unchanged. It is the only one that solves the *full* non-Tamm-Dancoff problem and the only one that returns every exciton.
- `solver="iterative"` applies the factorized kernel through a `LinearOperator` and finds the `neig` lowest excitons with a preconditioned block LOBPCG from a deterministic start. It is exact – the operator applied is the same matrix `build_blocks` assembles – and it removes the memory wall outright, but it still diagonalizes every k-point, so its cost is linear in the mesh.
- `solver="qtt"` binary-encodes the pair index, cross-interpolates the kernel into a matrix product operator with the bundled `qutecipy`, and diagonalizes it with DMRG from `dmrgpy`'s `pyitensor`. Its cost grows like $\log N_k$: on a gapped chain the number of k-points it actually diagonalizes went 1011, 2657, 5721, 6592, 8757 as the mesh went 1024, 4096, 16384, 65536,

262144 – a factor of 8.7 for a factor of 256 in mesh – with the wall time flat at 14-23 s throughout. The dense solver stops around $N_{pair} \sim 2000$. Two dimensions costs considerably more per mesh point, since the MPO bond dimension is roughly eight times larger there.

Both new solvers need `tda=True`. The full BSE matrix is not Hermitian and is diagonalized through a Cholesky factorization of SH , which needs the matrix; large-scale BSE codes make the same restriction for the same reason, and `tests/bse/test_bse_physics.py` shows the Tamm-Dancoff and full answers converging at weak coupling, which is where bound excitons live.

The quantics solver has one requirement the others do not: a **smooth gauge**. Diagonalization returns each Bloch eigenvector with an arbitrary phase (and, inside a degenerate multiplet, an arbitrary unitary), which makes the kernel a discontinuous function of k and destroys its tensor-train rank completely – in the raw gauge the operator is exactly incompressible. `gauge="projection"` (the default for this solver) rotates each band subspace onto fixed trial orbitals, `gauge="phase"` fixes one component of each eigenvector, and the gauge is unobservable, so it can be switched on for the dense solver too and the spectrum must not move. Measured maximum tensor-train rank of the kernel’s factors at tolerance `1e-6`, as the mesh is refined 16-fold:

model	raw gauge	fixed gauge
1D chain, spinless	16 → 32 → 64	8 → 8 → 8 (phase)
2D honeycomb, spinless	96 → 192 → 383	57 → 62 → 63 (phase)
2D honeycomb, spinful	256 → 512 → 1024	182 → 248 → 274 (projection)

A phase fix cannot help a degenerate multiplet – what is arbitrary there is a whole unitary – which is why `"projection"` is the default and why a spinful model with no spin-orbit coupling, whose every band is two-fold degenerate, needs it.

When "qtt" is the right choice, and when it is not. It wants a narrow band window on a fine k -mesh, and that is a real restriction. In 1D with one band pair it is the clear winner (mesh $\times 256$, work $\times 8.7$, wall time flat). In 2D on a primitive cell it is *slower* than `"iterative"` at every mesh measured – 103 s against 0.7 s at 32×32 – not because the quantics side degrades but because the exact solver is so cheap there (4096 pairs cost it 3.1 s in 2D against 24 s in 1D); the ratio does improve with mesh, so a crossover exists beyond what was measured. On a **supercell** it is the wrong tool outright: a 3×3 supercell has 9 valence and 9 conduction bands, and a band label is not a smooth coordinate the way k is, so the MPO rank saturates (bond dimension 729 out of a possible 729, with 450 s spent in cross-interpolation, at $nk=4$). The same physics from the primitive cell at three times the mesh keeps $n_v n_c = 1$ and compresses. Use the

primitive cell and a fine mesh; if a supercell is unavoidable, narrow the window with `nv/nc`, or use `solver="iterative"`, which is indifferent to it.

Other things worth knowing before turning `solver="qtt"` on: `nk` must be a power of two (the `k` index is binary encoded, one tensor-train site per bit); a tabulated screened interaction (`screening="rpa"`) is refused, because inverse transforming it over the mesh gives N_k lattice vectors and the kernel's rank would grow with the mesh – truncate it in real space with `ScreenedInteraction.get_dict()` and pass the result as `V=`; and it returns the **lowest exciton only**. Excited excitons would need an overlap-penalty DMRG, which does not converge on this problem (measured errors up to 0.4, and sometimes below the true eigenvalue, at any penalty weight tried), so it is refused rather than approximated – use `solver="iterative"`, which is exact and also needs no matrix.

See `examples/1d/exciton_qtt/main.py` for a runnable comparison of the three solvers across meshes spanning three orders of magnitude, and `examples/2d/exciton_qtt/main.py` for the 2D case, where the bit ordering is measured rather than assumed and the exciton envelope $|A(k)|^2$ is plotted over the Brillouin zone.

See `examples/2d/excitons_bse/main.py` for a runnable version (the lowest exciton of a gapped honeycomb detaching from the absorption edge as the Coulomb tail is turned up) and `tests/bse/` for the correctness checks, including a cross-check that the exchange-only BSE reproduces the poles of the independently implemented RPA kernel of `chitk/rpa.py`.

Exciton band structure

An exciton is a two-particle state, so besides its binding energy it has a dispersion of its own: the bound electron-hole pair propagates with a center-of-mass momentum Q , and $E_X(Q)$ is the exciton band structure. It is not the difference of two band energies – the electron-hole interaction bends it – so its curvature is the exciton's effective mass, and a flat exciton band means a strongly bound, spatially compact exciton. `h.get_exciton_bands` solves one BSE per q-point along a path and returns the result in the same flat form `get_bands` uses:

```
# h2 and W as in the section above
import numpy as np
from pyqula import geometry
from pyqula.bsetk.interaction import density_interaction
g = geometry.honeycomb_lattice()
h2 = g.get_hamiltonian()
h2.add_sublattice_imbalance(1.0) # a gapped semiconductor
W = density_interaction(h2, Vr=lambda r1,r2: 0.8/np.sqrt((r1-r2).dot(r1-r2)+0.25))
# nv=nc=2 keeps both members of each spin-degenerate pair
opts = dict(V=W, nq=20, nk=8, nv=2, nc=2, n=4)
qs, es = h2.get_exciton_bands(**opts) # the four lowest excitons along the path
```

```
qs0,es0 = h2.get_exciton_bands(kernel="none",**opts) # the bare continuum
```

`qs` holds the integer index of the q-point along the path and `es` the exciton energy, both flat 1D arrays ready for a scatter plot; `n` keeps only the `n` lowest excitons at each q-point and every other argument is passed straight to `get_bse`. `qpath` takes the same input as `get_bands` – a list of high-symmetry labels or of explicit q-vectors – and Q is not restricted to the k-mesh, since the pair basis diagonalizes at k and $k + Q$ independently. Running it a second time with `kernel="none"` is the same call with the kernel construction skipped, and gives the bottom of the electron-hole continuum to plot the exciton band against, which is what makes the binding visible. If the mean-field reference is unstable against some excitation at some q-point, `es` comes back complex there rather than silently losing the imaginary part. The cost is `nq` full dense diagonalizations, so `nv/nc` and `tda=True` are the knobs that make a long path affordable, and `parallel.set_cores` parallelizes over the path.

One trap worth knowing about, since it is invisible at $Q = 0$: `nv/nc` must not cut a degenerate multiplet in half. Every band of a spinful Hamiltonian with no spin-orbit coupling and no magnetic order is two-fold degenerate, so `nv=1` there keeps an arbitrary state out of a two-dimensional degenerate subspace, and the exciton energies inherit that arbitrariness – measurably, $E_X(Q)$ stops being even in Q on a time-reversal-symmetric model by ~ 0.1 , where the full multiplet gives equality to 10^{-15} . The library warns when the window splits a multiplet; use an even `nv/nc` on a spin-degenerate Hamiltonian.

See `examples/2d/exciton_bands/main.py` for a runnable version (the exciton band of a gapped honeycomb, plotted below the electron-hole continuum of the same model) and `tests/bse/test_bse_bands.py` for the checks.

The screened interaction

Everything above uses the *bare* interaction in both kernel terms, which makes the BSE time-dependent Hartree-Fock. But an electron and a hole added to a solid do not feel the bare interaction: the other electrons rearrange around them, and what survives that rearrangement is the *screened* interaction. Since the mean-field step has already left the bands $\{e_n(k), C^n(k)\}$ on a k-mesh, the static RPA screening can be computed from them directly rather than postulated,

$$\chi_{ab}^0(q) = \frac{1}{N} \sum_k \sum_{n,m} (f_{nk} - f_{m,k+q}) \frac{\rho_a^{nm}(k, q) \rho_b^{nm*}(k, q)}{e_{nk} - e_{m,k+q}}, \quad \rho_a^{nm}(k, q) = C_a^{n,k*} C_a^{m,k+q}$$

$$\varepsilon(q) = 1 - v(q) \chi^0(q), \quad W(q) = \varepsilon^{-1}(q) v(q)$$

with a, b running over the spin-orbitals of the unit cell, in the same point-like-orbital approximation the rest of the BSE uses. `screening="rpa"` computes this and puts W in the direct (ladder) term:

```

import numpy as np
from pyqula import geometry
from pyqula.bsetk.interaction import density_interaction

h = geometry.honeycomb_lattice().get_hamiltonian()
h.add_sublattice_imbalance(1.0) # a gapped semiconductor

# a BARE interaction: an onsite term plus a soft-cutoff Coulomb tail
coulomb = lambda r1,r2: 0.6/np.sqrt((r1-r2).dot(r1-r2)+0.25)
V = density_interaction(h,U=1.0,Vr=coulomb)

bare = h.get_bse(V=V,nk=8) # time-dependent Hartree-Fock
screened = h.get_bse(V=V,nk=8,screening="rpa") # GW-BSE style

```

The **exchange term keeps the bare interaction** whatever screening is set to. That is the standard GW-BSE split, not an oversight: screening the exchange term as well would resum the same RPA bubbles a second time. A consequence worth knowing is that `kernel="exchange"` is completely unaffected by `screening` (a warning is raised if both are given).

A fitted Hubbard U must not be screened. A U chosen to reproduce a material is already an effective, screened interaction; running it through this a second time is double counting and gives a spuriously weak interaction. Screening is for a genuinely *bare* interaction – a long-range Coulomb tail from `density_interaction(Vr=...)`, or bare model $V1/V2/V3$ shells. The dangerous case is precisely the default one, $V=h.V$ from a Hubbard SCF. Note that this is a different question from RPA-versus-cRPA below: bubbles inside W and ladders in the BSE are different diagram classes, so a full-RPA W with a BSE ladder is the standard construction and is *not* double counting.

`screening="crpa"` is the constrained variant, which leaves the transitions inside the n_v/n_c BSE band window out of the polarization (arXiv:0710.4013). That is the right choice when the band window is being treated as a downfolded model to be solved exactly afterwards, and it screens strictly less than the full RPA. It needs a genuine subset of the bands: with the default `nv=nc=None` the window is the whole spectrum, nothing is left outside it to do the screening, and the call is refused rather than silently returning the bare interaction.

The screened interaction is also available on its own, as a **ScreenedInteraction**:

```

W = h.get_screened_interaction(V=V,nk=8)
print(W.epsmin) # smallest dielectric eigenvalue over the mesh
Wq = W.at(W.qs[3]) # the screened interaction at one q-point
d = W.get_dict() # ... and back in real space, usable at any q

```

Unlike the real-space dictionaries used elsewhere, this object exists only on its mesh: $W(q)$ is the result of a matrix inversion at each q , not the Fourier transform of anything short ranged. That is exactly what the direct term needs, because the distinct $k - k'$ differences of a Γ -centered mesh are mesh points

themselves, so W is tabulated at precisely the points it is consumed at with no interpolation anywhere. Asking for it at any other q raises rather than snapping to the nearest point. `get_dict()` is the escape hatch: it inverse Fourier transforms back to a real-space interaction, which can be evaluated anywhere, inspected to see how far the screened interaction reaches, or fed to `get_mean_field_hamiltonian(V=...)` for a screened-exchange mean field – at the price of aliasing the tail beyond the mesh supercell. `nkW` takes a denser screening mesh than the BSE mesh (it must be an integer multiple of `nk`, so that the q -points the direct term needs are still tabulated).

Where the dielectric matrix is built matters. Screening is a property of the *charge* channel: what polarizes the medium is the total density, and what the medium’s induced charge acts back on is again the total density. So `channel="charge"` (the default) builds ε on **site** indices,

$$\varepsilon_{ij}(q) = \delta_{ij} - \sum_k v_{ik}^c(q) \chi_{kj}^c(q), \quad \chi_{ij}^c = \sum_{\sigma\sigma'} \chi_{(i\sigma)(j\sigma')}^0, \quad v_{ij}^c = \frac{1}{4} \sum_{\sigma\sigma'} v_{(i\sigma)(j\sigma')}$$

and adds the resulting correction, which is spin-independent, to the bare interaction:

$$W_{(i\sigma)(j\sigma')} = v_{(i\sigma)(j\sigma')} + [v^c \chi v^c]_{ij}, \quad \chi = \chi^c (1 - v^c \chi^c)^{-1}$$

This is the standard GW construction. Note the one convention it fixes: $v_{ij}^c = V_{ij}$ off-site, but $v_{ii}^c = U/2$, not U , because a Hubbard term couples only opposite spins, so only half of a site’s own density acts on a given electron. Note also that the correction is $v^c \chi v^c$ rather than the naive $\varepsilon^{-1}v$ – with ε and v living on different index spaces the latter is not even Hermitian.

Two things follow, and they are why this is the default. First, it **preserves spin-rotation invariance exactly**: the same-spin minus opposite-spin part of W , which is an Ising $S_i^z S_j^z$ coupling, is left exactly as the bare interaction had it. Second, v^c picks up a positive diagonal from the Hubbard term, which restores ordinary screening.

The alternative, `channel="orbital"`, dresses the full spin-orbital matrix as $W = \varepsilon^{-1}v$. It screens the charge and spin channels with a single density-density kernel and therefore **breaks SU(2)**: on the gapped honeycomb an exciton multiplet four-fold degenerate to 10^{-14} bare splits by 4.3×10^{-3} in the orbital channel and stays degenerate to 8×10^{-15} in the charge one. It is kept because it is the honest full-matrix RPA and is useful for quantifying that error, not because it should be used. The two channels coincide exactly for a spinless Hamiltonian.

Screening still does not always weaken the interaction, if the model has no onsite term. A density-density matrix that excludes self-interaction

is traceless, hence necessarily indefinite; the charge channel escapes that only through $v_{ii}^c = U/2$, so with $U = 0$ it is traceless too and the interaction comes out *enhanced* rather than screened – the same Stoner enhancement `chitk/rpa.py` reports as a magnetic instability. Measured on the gapped honeycomb with an $e2 = 0.6$ tail, the change in binding energy against the bare result is $+0.0055$ at $U = 0$ in both channels, and at $U = 2.0$ it is $+0.0078$ in the orbital channel but -0.0030 in the charge one. A point-orbital Coulomb tail with no onsite term is simply missing its largest matrix element; include a realistic U .

Finally, if an eigenvalue of $\varepsilon(q)$ actually reaches zero, the RPA has diverged: that is a charge or spin instability of the mean field at that wavevector – the same $1 - V\chi = 0$ condition `chitk.rpa.rpa_kernel_poles` reports as a collective mode – and the call raises rather than returning a huge number.

One cost note: an RPA-screened W is dense in the orbital indices, where a bare Hubbard-like interaction is very sparse. The direct kernel exploits that sparsity, so turning screening on can slow the kernel build noticeably on a model whose bare interaction was nearly diagonal. That is unavoidable – it is what screening physically does.

See `examples/2d/screened_bse/main.py` for a runnable version and `tests/bse/test_bse_screening.py` for the correctness checks. There is no external benchmark for this: Xatu, whose formalism the rest of this BSE follows, uses a *phenomenological* Rytova-Keldysh screening rather than an RPA W , and no comparable open tight-binding implementation was found – so the tests are internal cross-checks (an exact $q = 0$ sum rule, supercell folding, the weak-coupling series $W = v + v\chi^0 v + O(v^3)$, the reciprocity $W(-q) = W(q)^*$ on a magnetic model, and agreement of χ^0 with the independently implemented response function of `chitk/chiAB.py`).

Quantum transport

In this section we discuss how we can perform quantum transport calculations with `pyqula`.

Magnetoresistance in metal-metal transport

As specific example, here we will address how we can compute magnetoresistance in transport between two magnetic metals. We build two copies of the same lead, give each one an exchange field pointing in a different direction, and compare the conductance of the parallel and antiparallel configurations

```
from pyqula import geometry
from pyqula import heterostructures
import numpy as np
g = geometry.chain() # create the geometry
h = g.get_hamiltonian() # create the Hamiltonian
```

```

es = np.linspace(-.5,.5,50) # set of energies for dIdV
Gs = dict()
for name,m2 in [("parallel",[0.,0.,0.5]),("antiparallel",[0.,0.,-0.5])]:
    h1 = h.copy() ; h1.add_exchange([0.,0.,0.5]) # first lead, fixed magnetization
    h2 = h.copy() ; h2.add_exchange(m2) # second lead, parallel or antiparallel
    HT = heterostructures.create_leads_and_central(h1,h2,h1) # create the junction
    Gs[name] = [HT.didv(energy=e) for e in es] # calculate conductance

```

The magnetoresistance follows from the two conductance curves, e.g. $MR = (G_P - G_{AP})/G_{AP}$ evaluated at the Fermi energy.

Superconductor-metal transport

Here we address how transport between a superconducting lead and a metallic lead can be computed. As paradigmatic example, we will focus on the Andreev reflection regime and the tunneling regime

```

from pyqula import geometry
from pyqula import heterostructures
import numpy as np
g = geometry.chain() # create the geometry
h = g.get_hamiltonian() # create the Hamiltonian
h1 = h.copy() # first lead
h2 = h.copy() # second lead
h2.add_swave(.01) # the second lead is superconducting
es = np.linspace(-.03,.03,100) # set of energies for dIdV
for T in np.linspace(1e-3,1.0,6): # loop over transparencies
    HT = heterostructures.build(h1,h2) # create the junction
    HT.set_coupling(T) # set the coupling between the leads
    Gs = [HT.didv(energy=e) for e in es] # calculate conductance

```

Transport through an arbitrary finite region

The two examples above build the central scattering region out of copies of the leads' own unit cell. `Hamiltonian.get_central_heterostructure(i,j,left=None,right=None)` instead lets *any* finite (0d) Hamiltonian act as the central region, contacted by two semi-infinite 1D chain leads attached at sites *i* and *j*:

```

from pyqula import geometry

g = geometry.chain()
gc = g.get_supercell(5)
gc.dimensionality = 0 # a finite, 5-site cluster (no periodicity)
hc = gc.get_hamiltonian() # the central region -- can be any 0d Hamiltonian

h_normal = g.get_hamiltonian() # normal lead
h_sc = g.get_hamiltonian(); h_sc.add_swave(0.05) # superconducting lead

```

```
ht = hc.get_central_heterostructure(0,4,left=h_normal,right=h_sc)
G = ht.didv(energy=0.02) # Andreev conductance, via the BdG scattering-matrix formula
```

It returns a plain `Heterostructure`, so every existing method (`didv`, `get_dos`, `get_kappa`...) applies unmodified – except `landauer`, which refuses a junction carrying the Nambu degree of freedom, as in the example above: the Landauer formula counts single-particle transmission and a Cooper pair carries charge $2e$, so use `didv`, which applies the BTK/BdG scattering formula. `left/right` default to a plain spinless chain when omitted, and `j` defaults to the last site. At most one of `{hc, left, right}` may carry an actual pairing amplitude – e.g. a normal lead + normal lead + superconducting central region (a proximitized molecule) is fine, as is the normal + superconducting lead case above, but two superconducting leads raise a `ValueError` (there would be no normal lead left to define a reflection amplitude against; use `heterostructures.build + get_dc_current` for that case instead, see below). Only 0d central regions are supported so far. See `examples/transport/central_region_ij/main.py` for a runnable script.

Landauer transmission through a real-space device

`heterostructures` builds a junction out of two periodic Hamiltonians. `multiterminal.Device` takes the other route: the leads and the scattering region are given as *geometries*, the hoppings between them are generated from the actual interatomic distances, and the Landauer transmission follows from the leads' self-energies. It is the natural form for a disordered or irregularly-shaped conductor, where there is no unit cell to repeat.

```
import numpy as np
from pyqula import geometry
from pyqula import multiterminal

def lead_cell(x): # a one-site chain unit cell sitting at x
    g = geometry.chain() ; g.r = np.array([[x,0.,0.]]) ; g.r2xyz()
    return g

gc = geometry.chain().supercell(4) # the scattering region
xs = gc.r[:,0]
d = multiterminal.Device()
d.biterminal(left_g=lead_cell(min(xs)-1.),right_g=lead_cell(max(xs)+1.),
             central_g=gc,disorder=0.0)
print("ballistic transmission:",d.transmission(energy=0.0)[0])
```

A perfectly matched chain has one open channel and nothing to scatter off, so the transmission is 1 everywhere inside the band, to the tolerance of the lead decimation – which is the check `tests/transport/test_multiterminal_landauer.py` makes. Raising `disorder` puts random onsite energies in the central region and

the transmission drops below 1 by an amount that depends on the realization drawn. `d.transmission(energy=e)` returns a list with one entry per lead pair, the pair (0,1) by default; `multiterminal.landauer(d,energy,ij=[(i,j)])` is the same quantity as a plain function, with the pairs named explicitly.

Multiple Andreev reflection and AC-Josephson current

`didv/landauer` above are equilibrium, zero-bias linear-response quantities. For a voltage-biased junction between two superconductors (an SNS junction), a finite bias makes each lead’s pairing phase wind in time, giving rise to multiple Andreev reflections (MAR) and an AC Josephson effect; the physically meaningful, measurable quantity is the time-averaged (DC) current $I_{dc}(V)$. `Heterostructure.get_dc_current(voltage)` computes this with the Floquet-Keldysh formalism of San-Jose, Cayao, Prada and Aguado, *New J. Phys.* **15**, 075019 (2013) (arXiv:1301.4408): the bias is gauged away from the (static) leads into a single, periodically time-dependent “weak link” hopping, and the resulting Floquet-space Dyson/Keldysh equations are solved to get $I_{dc}(V)$. It works for any combination of normal and superconducting leads – including the case of **two** superconducting leads, which the scattering-matrix formula behind `didv` cannot handle on its own (it has no normal lead to define a reflection amplitude against).

`didv` takes this formalism as an optional `method` argument, so a linear-response conductance can be obtained without calling `get_dc_current`/differentiating by hand: `method="smatrix"` is the original zero-temperature scattering-matrix/BdG conductance; `method="keldysh"` returns dI/dV at bias `energy`, computed as a central finite difference of `get_dc_current` (`HT.didv(energy=v, method="keldysh", dv=..., nmax=..., temperature=...)`, extra keyword arguments are forwarded to `get_dc_current`); the default, `method="auto"`, picks `"keldysh"` when both leads carry an actual (nonzero) pairing amplitude and `"smatrix"` otherwise, since a single/no superconducting lead is already handled exactly by the scattering matrix.

“Works for any combination of leads” means `get_dc_current` runs without error for a mixed normal/superconducting **Heterostructure** junction too – but its result is **not** directly comparable to `didv(method="smatrix")` on the same junction, even in the limit where the superconducting lead’s pairing amplitude is taken to zero (this caveat is specific to **Heterostructure**; see below for why **LocalProbe** is different). The two methods use different bias conventions: `smatrix` freezes the normal lead’s self-energy at absolute energy 0 (a grounded, wide-band probe with the entire bias dropped on the other lead), while `dc_current` evaluates *both* leads’ self-energies at their actual Floquet side-band energies (needed for, and validated against, a normal-normal rigid two-terminal bias reference in `tests/keldysh`). Forcing `method="keldysh"` where `method="auto"` would have picked `"smatrix"` therefore computes a different physical quantity, not a slower/more-general version of the same one – confirmed by direct comparison to disagree by an $O(1)$, non-vanishing factor as the extra

lead’s pairing amplitude shrinks to zero. Exactly at sub-gap bias this is compounded further by real (not numerical) physics: an infinitesimal pairing amplitude still opens a hard gap exactly at the Fermi level, which `get_dc_current`’s sideband ladder always samples (its quasienergy integral starts at 0), so its zero-pairing limit at that energy does not equal the exactly-normal-lead value there either.

```
from pyqula import geometry
from pyqula import heterostructures
import numpy as np
g = geometry.chain() # create the geometry
h = g.get_hamiltonian() # create the Hamiltonian
h1 = h.copy() ; h1.add_swave(0.1) # left superconducting lead
h2 = h.copy() ; h2.add_swave(0.1) # right superconducting lead
HT = heterostructures.build(h1,h2) # create the SNS junction
HT.set_coupling(0.5) # set the normal transparency of the weak link
vs = np.linspace(0.02,1.5,40)*0.1 # bias voltages
Is = HT.get_iv_curve(vs) # MAR/AC-Josephson dc current
```

Budget for the cost: this is the most expensive family in the guide, at roughly 15 seconds per bias point for a two-lead SNS junction, so the 40-point curve above is several minutes. Note also the bottom of that voltage grid. The number of sidebands needed grows as the bias falls (the MAR order $2\Delta/eV$ does), so a grid reaching very close to zero bias – `0.02*delta` here – can genuinely exhaust `nmax_max` and warn that “sidebands did not converge”, meaning that point is inaccurate rather than that anything crashed. Raising the grid’s lower endpoint (to `0.15*delta`, say) is the cheap fix; raising `nmax_max` is the expensive one.

The number of Floquet sidebands is increased adaptively (as in the paper) until I_{dc} converges; see `examples/transport/floquet_keldysh_mar/main.py` for a runnable script and `tests/keldysh/` for correctness tests (a normal-normal junction must reduce exactly to a directly biased, non-Floquet Landauer calculation, and a normal-superconductor junction’s zero-bias slope must match the existing equilibrium Andreev conductance from `didv`). Only 1D leads are supported. An explicit central region (`heterostructures.build(h1,h2,central=[hc])`), e.g. a quantum dot detuned from the leads) works too, solved through the general dense Floquet inversion rather than the fast two-block chain decomposition – correspondingly slower, since the whole (block x sideband) matrix is inverted at every quasienergy. Note where the bias is assumed to drop: the AC-carrying bond is the junction’s rightmost one, so the central region sits at the **left** lead’s electrostatic potential. That is a physical model choice, not a gauge choice – a comparison against a static-bias reference has to shift the central region along with the left lead, and a central Hamiltonian must be a valid BdG (particle-hole symmetric) one, so detune it with `hc.shift_fermi(eps)` rather than by adding `eps` to the diagonal of `hc.intra`.

`transporttk.localprobe.LocalProbe` models a single STM-like tip weakly

coupled to one site of an infinite/bulk sample (used e.g. for `get_kappa`, a decay-constant/transparency-scaling diagnostic – see `examples/transport/decay_constant/main.py`). The same routing applies there: `LocalProbe.didv/get_kappa` use the ordinary scattering-matrix formula by default, but switch to the Floquet-Keldysh MAR current when the probe lead (`lp.lead`) is itself superconducting *and* the sample is superconducting too, since a normal-metal probe no longer applies and the same “no normal lead to reflect against” problem as above appears. The probe’s unit cell and the sample’s local (single-site) Hamiltonian play the role of the two leads.

Unlike `Heterostructure`, forcing `method="keldysh"` on a `LocalProbe` whose probe is normal (or negligibly paired) *is* consistent with `method="smatrix"`: `LocalProbe`’s Keldysh path grounds a normal probe lead (freezes its self-energy at absolute energy 0, `smatrix`’s own convention for it) precisely so the two agree – exactly in the wide-band-lead limit, to within a few percent for a probe with genuine band structure over the bias window (e.g. a plain tight-binding chain). This grounding only applies when the probe lead itself has no real pairing; when it does (the case below), the probe’s self-energy is evaluated at its actual Floquet sideband energy instead, exactly as for `Heterostructure` – grounding a genuinely superconducting probe would pin every evaluation at its own gap center and was confirmed to suppress the MAR current by over an order of magnitude, so the two-superconductor case keeps working as originally validated.

```
from pyqula import geometry
from pyqula.transporttk.localprobe import LocalProbe

g = geometry.chain()
h = g.get_hamiltonian() ; h.shift_fermi(1.) ; h.add_swave(0.1) # SC sample
lead = geometry.chain().get_hamiltonian() ; lead.shift_fermi(1.) ; lead.add_swave(0.1) # SC
lp = LocalProbe(h, lead=lead, delta=1e-3)
lp.T = 0.3 # reference transparency
G = lp.didv(energy=0.25, nmax=4, nmax_max=12, tol=5e-2) # routed through Keldysh automatically
k = lp.get_kappa(energy=0.25, nmax=4, nmax_max=12, tol=5e-2)
```

This is considerably more expensive than the normal-probe case (each `didv/get_kappa` call runs several Floquet-Keldysh sideband sweeps), especially deep below the combined gap at low transparency, where the sideband sum converges slowly; see `examples/transport/decay_constant_keldysh/main.py` for a runnable script using a coarse energy grid and a modest sideband cutoff to keep the runtime reasonable.

`get_kappa` also accepts a `temp` argument for a thermally-averaged kappa (each conductance entering the power-law fit is `didv(temp=...)`’s thermal average rather than the zero-temperature value); `temp=0` (the default) is unchanged. Pass a single `energy` (returns a scalar, as above) or a whole `energies=[...]` array at once (returns an array):

```
k = lp.get_kappa(energies=[0.1,0.25,0.4],temp=0.02,nmax=4,nmax_max=12,tol=5e-2)
```

`Heterostructure.get_kappa` takes the same `temp/energies` arguments.

A single `get_dc_current/keldysh_didv/get_kappa` call solves each lead's Sancho-Rubio/bloch_selfenergy self-energy directly at every energy the sideband sweep visits by default (`selfenergy_method="direct"`), since building an AAA interpolant (many true solves at increasingly refined candidate energies) usually costs more than that one "direct" call by itself – the win comes from reuse, not the first call. The opt-in `selfenergy_method="aaa"` (`use_aaa=True` for `didv/keldysh_didv`) instead replaces most of those many-thousands of individual solves with evaluations of a compact rational (AAA) interpolant built from far fewer true solves (`keldyshtk.current.build_selfenergy_aaa`).

A **sweep** over many calls is exactly the workload that reuse pays for, so the three sweep entry points default the OTHER way, to "aaa"/`use_aaa=True`, building and sharing one interpolant across the whole sweep instead of leaving each call to independently build (and discard) its own: `get_iv_curve` (a `get_dc_current` voltage sweep), `didv(energies=...)/didv_curve` (a `didv` energy sweep), and `get_kappa(energies=...)`'s finite-temperature path (whose internal thermal quadrature alone can visit well over a hundred nearby energies for just one nominal (`energy, temp`) point). Pass `selfenergy_method="direct"` (or `use_aaa=False`) explicitly to any of these to opt back out. Each falls back to "direct" automatically, per-sweep, if the shared fit doesn't converge within its budget.

An earlier version of this had a real accuracy gap, growing with the sideband window (`nmax_max`) – up to ~10% relative current error in the worst case investigated. `documentation/keldysh_aaa_selfenergy_accuracy_plan.md` root-caused this to the interpolant's candidate grid being under-resolved (both at a lead's own gap-edge singularities and, more consequentially for the current-error trend, across the fit's broader domain) in a way the interpolant's own held-out validation check – confined too close to existing candidates – never detected. Both the validation sampling and the grid-refinement strategy (`aaatk.selfenergy_aaa._refine_grid`) were fixed and validated directly against the current (not just the self-energy fit) across the same `nmax_max` sweep that exposed the original gap: relative current error is now consistently under ~1% throughout, with no growth trend (see that document's closing update for the full measurement). `selfenergy_method="aaa"` still checks its own convergence (`.converged`) and safely falls back to "direct" if a fit can't reach its target tolerance within a bounded budget, rather than silently returning an under-resolved answer – but as with any interpolation-based shortcut, checking agreement with "direct" for your own system/parameter range before relying on it is still good practice.

`didv`'s `energy` and `energies` arguments are mutually exclusive, the same convention `get_kappa` already uses: pass a single `energy` (returns a scalar) or a whole `energies=[...]` array at once (returns an array, and internally dis-

patches to `didv_curve(ht, energies, **kwargs)`). Passing an array directly as the scalar `energy=` is not supported (it fails, a numpy broadcasting error on the `smatrix` path, an “ambiguous truth value” error on the `keldysh` path) – every example that plots dI/dV vs. energy predates `energies=` and loops explicitly instead, `[ht.didv(energy=e) for e in es]`, which still works but is no longer necessary. `energies=` additionally – like `get_iv_curve` does for `get_dc_current` – builds and shares ONE AAA interpolant across the whole sweep by default (see above), instead of a raw loop’s `didv(energy=e, use_aaa=True)` independently building (and discarding) its own interpolant at every single energy:

```
es = np.linspace(0.15,0.25,40)*0.1 # bias energies
Gs = HT.didv(energies=es, nmax_max=40) # shared-AAA dI/dV curve (use_aaa=True is now the de
```

`HT.didv_curve(es, **kwargs)/lp.didv_curve(es, **kwargs)` are the same thing called directly, for a caller who wants the array entry point without going through `didv`.

For a Floquet-Keldysh-eligible junction (both leads, or a `LocalProbe`’s `probe+sample, superconducting`), `didv(temp=...)/get_kappa(temp=...)` now default to evaluating `dc_current`’s own `temperature` parameter directly (2 `dc_current` calls, a central finite difference in bias) rather than the internal thermal quadrature described in the previous paragraph (`keldysh_thermal_mode="convolution"` on `transporttk.thermaldidv.finite_T_didv` recovers the old behavior, still used for non-Keldysh junctions where it is exact). This is not merely a faster way to get the same number: the two compute genuinely different quantities away from `temp->0` (direct broadens each Floquet sideband’s own occupation by `temp`; convolution smears the bias voltage, an n -dependent displacement of the whole sideband ladder) – see `documentation/keldysh_sideband_decimation_plan.md`’s “direct finite-T Keldysh evaluation” entry for the validation and the measured ~100x-plus speedup.

The outer quasienergy integral itself is evaluated with a batched adaptive quadrature (`keldyshtk.quadrature.adaptive_quad_batch`): the same 21-point Gauss-Kronrod rule and embedded QUADPACK error estimator `scipy.integrate.quad` uses, at the same tolerance, but with the refinement loop restructured so that every panel awaiting evaluation in a round is evaluated in a single batched, `numba`-parallel chain solve rather than one scalar Python callback per node. It visits essentially the same nodes as `scipy.integrate.quad` did (measured over a whole `get_dc_current` call: 630 vs 588, 1197 vs 1197, 3906 vs 3906 on three superconducting cases, and 84 vs 210 on a normal junction) while collapsing those hundreds-to-thousands of scalar dispatches into 4-54 batched ones. Together with a companion fix to the self-energy cache’s per-energy Python bookkeeping (which profiling exposed as the next bottleneck once the quadrature stopped dominating), that is worth 5.0x-11.8x in wall clock across those four cases, with the returned current unchanged to 1e-16 relative on three of them and 1.5e-6 on the fourth. This is

the default; it needs no opt-in, and the previous `scipy.integrate.quad` implementation stays reachable as `quadrature="adaptive_scipy"` (the reference the batched rule is validated against, not a mode to choose on its own).

`dc_current` also takes an opt-in `quadrature` argument for the outer quasienergy integral: `"adaptive"` is the batched adaptive rule just described (the default); `"fixed"` instead evaluates a deterministic, fixed-node composite Gauss-Legendre rule whose node/weight set is a pure function of `voltage` alone (`quad_panel_width/quad_min_panels/quad_order` control it), known in full before any integrand evaluation and solved with a batched, `numba`-parallel chain solver rather than one Python callback per node. Accuracy is not the concern (validated to within $\sim 6e-4$ of a tight reference across a broad SC-SC/normal sweep); speed is case-dependent and often *worse* than adaptive quadrature, since a fixed grid has to be dense enough to resolve a gap-edge singularity wherever it happens to land, with no way to discover at runtime that a given case (e.g. a normal junction with no singularity at all) didn't need that density. `"fixed"` is kept as tested, opt-in infrastructure for callers that specifically need a deterministic/cacheable node set (see `documentation/keldysh_sideband_decimation_plan.md`'s "item 2b"/"item 2c" entries for the full numbers) – not a general replacement for the default `"adaptive"` path.

Experimental: a JAX-differentiable Floquet-Keldysh current

`keldyshtk.current_jax.JaxKeldyshCurrent` (needs the optional `jax` extra: `pip install pyqula[jax]`) is an independent reformulation of `dc_current` for zero-temperature, fixed-sideband-count (`nmax`, not adaptively grown) work: instead of a central finite difference of two separate `dc_current` calls, it batches the whole quasienergy quadrature into one compiled, `vmapped` computation and differentiates it directly with `jax.grad`. Reused across many voltages (build once per (`junction`, `nmax`, `vmax`) combination, call `.current(v)/.didv(v)` many times), this is a genuine reformulation, not a drop-in speedup: measured on the same superconducting-probe `LocalProbe` workload the rest of this section targets, both `.current()` and `.didv()` came out roughly break-even to about 2x slower than the direct path once implemented and benchmarked rigorously (see the module's own docstring for the full story, including two real self-energy numerical-edge-case bugs and one silently-dropped derivative term found and fixed along the way). Kept as tested, documented, opt-in infrastructure – a reformulation that did not pay off for the specific workload it was built for, potentially useful for a different one (a system that converges at a smaller `nmax`, or a workload needing only `current()` and not `didv()`) – not something `didv/get_dc_current` route to automatically.

Single defects in infinite systems

A single point defect or impurity embedded in an otherwise infinite, periodic system cannot be handled by a plain supercell calculation without artificially periodizing the defect. `embedding.Embedding` solves this properly with a Green's function embedding technique: it takes the pristine, periodic Hamiltonian `h` and a modified intracell matrix `m` describing the defect (or another Hamiltonian from which the modified matrix is taken), and gives access to the observables of the infinite system as perturbed by that single, non-periodic defect

```
from pyqula import geometry
from pyqula import embedding
g = geometry.chain() # create the geometry
h = g.get_hamiltonian() # pristine, infinite Hamiltonian
hv = h.copy()
hv.add_onsite(lambda r: 1.0 if r[0]<0.01 else 0.0) # a single-site onsite defect
eb = embedding.Embedding(h,m=hv) # embed the defect in the infinite system
(x,y,d) = eb.get_ldos(energy=0.0,delta=1e-2,nsuper=200,nk=400) # LDOS around the defect
```

`get_ldos` returns the real-space positions and the LDOS profile in a window of `nsuper` unit cells around the defect, showing e.g. Friedel oscillations or bound/in-gap states induced by the impurity (it also writes LDOS.OUT; pass `write=False` to suppress that). `eb.get_dos()` gives the total DOS, `eb.multildos()` scans the LDOS over many energies (written to a MULTILDOS/ folder), `eb.get_gf()` returns the embedded Green's function itself (with `operator=` it returns AG , the same convention `get_ldos` uses), and `eb.get_didv()` computes transport through the embedded defect. See `examples/embedding/single_impurity_1D/main.py` and `examples/embedding/honeycomb_vacancy/main.py` for runnable versions, and the other scripts under `examples/embedding/` for further defect scenarios (vacancies, boundaries, Yu-Shiba-Rusinov states, self-consistent defects...).

Wannierization

`h.get_wannier_hamiltonian()` Wannierizes a contiguous range of a periodic Hamiltonian's bands and returns a new, smaller multicell Hamiltonian whose real-space hoppings reproduce that band subspace on the wannierization k-mesh. By default the range is taken as a fixed subspace, Wannierized jointly as one group, and reproduced exactly; passing a `num_wann` smaller than the range instead disentangles, which is the subject of the second subsection below.

As an example, consider a staggered honeycomb lattice, where a sublattice potential opens a gap and gives a genuinely dispersive valence band to Wannierize

```
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian(has_spin=False)
```

```

h.add_onsite([0.8,-0.8]) # sublattice potential opens a gap

# Wannierize just the lowest (valence) band: bands=[0,0] selects band
# index 0 at every k-point of a 12x12 Monkhorst-Pack wannierization mesh
hwan = h.get_wannier_hamiltonian(bands=[0,0],nk=12)

print("Number of Wannier functions:",hwan.intra.shape[0])
print("Wannier centres (Cartesian):\n",hwan.wannier_centres)
print("Wannier spreads:",hwan.wannier_spreads)
print("Total spread Omega:",hwan.wannier_spread_total)

```

The Wannierized Hamiltonian `hwan` behaves like any other pyqula Hamiltonian, so its bands can be compared directly against the original model's

```

(k,e) = h.get_bands(write=False)
(kw,ew) = hwan.get_bands(write=False)

```

Symmetry-enforced Wannierization

Passing `symmetries="auto"` makes `get_wannier_hamiltonian` check, before Wannierizing, that the selected band range is a genuine union of point-group-related multiplets everywhere on the mesh (point-group operations are auto-detected from the geometry+Hamiltonian via `symmetrytk.pointgroup.find_point_group`). A band selection that instead slices through a symmetry-related degeneracy is rejected with a `ValueError` rather than silently returning a mis-symmetrized model. A list of explicit `symmetrytk.pointgroup.SymmetryOperation` can be passed instead of "auto" to enforce a specific subgroup.

A good illustration is kagome's flat band: it is exactly degenerate with the dispersive middle band at the K point, so no selection containing only the flat band is a union of whole multiplets – this is the well-known topological obstruction behind kagome's flat band having no symmetric exponentially-localized Wannier function, and the check catches it instead of returning a broken model

```

from pyqula import geometry
from pyqula.symmetrytk import pointgroup

g = geometry.kagome_lattice()
h = g.get_hamiltonian(has_spin=False)

try:
    h.get_wannier_hamiltonian(bands=[2,2],nk=12,symmetries="auto")
except ValueError as e:
    print("Flat band alone correctly rejected:",str(e).splitlines()[0])

# the full 3-band manifold has no such obstruction
hwan_sym = h.get_wannier_hamiltonian(bands=[0,2],nk=12,symmetries="auto")
print("Symmetries enforced:",[c.op.name for c in hwan_sym.wannier_symmetries])

```


See `examples/wannier/get_wannier_hamiltonian/main.py` and `examples/wannier/symmetric_wannierization/main.py` for runnable versions of these two examples.

Disentanglement

A fixed band range only works when the range is separated from everything else by a gap across the whole Brillouin zone. Graphene's two p_z bands are the canonical counterexample: they touch at K, so neither of them is a smooth subspace on its own. Souza-Marzari-Vanderbilt disentanglement (PRB 65, 035109 (2001)) handles that case by extracting `num_wann` optimally connected states out of a larger set of *offered* bands, k-point by k-point. It is switched on by passing a `num_wann` smaller than the selected range, in which case `bands=[a,b]` only says which bands are offered.

```
import numpy as np
from pyqula import geometry
g = geometry.honeycomb_lattice()
h = g.get_hamiltonian(has_spin=False)

# one Wannier function out of both pz bands, with the deep part of the
# valence band frozen (kept exactly rather than optimized)
hwan = h.get_wannier_hamiltonian(bands=[0,1],num_wann=1,nk=12,
                                dis_froz_max=-1.0,cutoff=0.0)

print("Wannier functions:",hwan.wannier_num_wann)
print("window used:",hwan.wannier_disentanglement_window)

f0,fw = h.get_hk_gen(),hwan.get_hk_gen()
k = [2./12.,1./12.,0.] # a wannierization-mesh k-point
e0 = np.sort(np.linalg.eigvalsh(f0(k))) ; ew = np.linalg.eigvalsh(fw(k))
print("frozen state reproduced to:",abs(ew[0]-e0[0]))
```

What is reproduced changes with it, and this is the trade rather than a loss of accuracy: a disentangled Hamiltonian reproduces the states inside the **frozen inner window** (`dis_froz_min/dis_froz_max`) exactly at every mesh k-point, and deliberately does not reproduce the selected bands outside it – outside the frozen window the extracted subspace is a different, smoother one. Pass `cutoff=0.0` when testing that exactness, as above: the default `cutoff=1e-6` drops small real-space hoppings and with them the last few digits. An **outer window** (`dis_win_min/dis_win_max`) narrows which bands are offered at each k-point, so the number available varies across the mesh, which is the point of a window rather than a band range; every eigenvalue of the result then lies inside it. `dis_num_iter` (default 200) bounds the Ω_I minimization – not reaching convergence within it is a warning rather than an error, since the frozen window is reproduced either way.

The returned Hamiltonian carries `wannier_num_wann` and `wannier_disentanglement_window`

(the four window values actually used, or `None` when not disentangling) alongside the centres and spreads.

Combinations that would silently do nothing raise instead: a `dis_*` window without a smaller `num_wann` (the engine gates disentanglement on the band count exceeding `num_wann`, so the window would be accepted and ignored), `dis_froz_min` without `dis_froz_max`, or a `num_wann` outside `1..len(bands)`. Disentanglement is not implemented together with a Nambu/BdG Hamiltonian, with `symmetries=`, or with `auto_split_clusters=True`, each naming the combination. One gap in that guard is worth knowing: `win_keywords=` is by design an unchecked passthrough merged after the check, so a window smuggled in through it can still reach the engine and be ignored – use the real `dis_froz_max=` argument, which is guarded.

Chebyshev kernel polynomial (KPM) methods

For very large systems, exact diagonalization becomes impractical. Passing `mode="KPM"` to observable methods switches to a stochastic Chebyshev kernel polynomial expansion, which never builds the full spectrum and scales to systems with millions of sites on a single core. It requires a sparse Hamiltonian (`is_sparse=True`)

```
from pyqula import geometry
import numpy as np
g = geometry.chain()
g = g.get_supercell(3000) # a big supercell
g.dimensionality = 0
h = g.get_hamiltonian(is_sparse=True, has_spin=False)
(x,y) = h.get_dos(mode="KPM",
                  energies=np.linspace(-3.0,3.0,200), # energies
                  delta=1e-4, # effective smearing (~1/npol)
                  ntries=10 # number of random vectors for the stochastic trace
                  )
```

The same expansion also gives non-local correlators and Green's functions without inverting a matrix, through the lower-level `kpm` module

```
from pyqula import kpm
(x,y) = kpm.dm_ij_energy(h.intra, npol=200, i=0, j=9, ne=1000)
```

See `examples/0d/kpm_dos/main.py` and `examples/0d/kpm_correlator/main.py` for runnable versions, including a comparison of the KPM correlator against the exact Green's function calculation.

The batched Chebyshev-moment kernels underneath KPM (one starting vector per random try, or per site for a full trace) run on numba by default, but can be dispatched to a GPU through `jax` instead by passing `kpm_cpugpu="GPU"` down to any KPM entry point – `h.get_dos(mode="KPM",`

...), `kpm.tdos/kpm.pdos/kpm.ldos`, or the lower-level `kpm` module functions themselves all forward it:

```
(x,y) = h.get_dos(mode="KPM",
                 energies=np.linspace(-3.0,3.0,200),
                 delta=1e-4,ntries=10,
                 kpm_cpugpu="GPU") # dispatch the Chebyshev moments to a GPU
```

`kpm_cpugpu="GPU"` transparently falls back to running on the CPU (through jax's own CPU backend) if no GPU is visible, so it is always safe to pass even on a machine without one. The batch of starting vectors is sent to the device in fixed-size chunks (`gpu_batch_size`, default 256, currently a reasoned default rather than a GPU-benchmarked one) rather than all at once, so device memory use stays bounded even for a full-space trace over a large system; pass a smaller `gpu_batch_size` to trade fewer, larger device dispatches for a smaller memory footprint. `kpm_prec="single"` (also forwarded the same way) switches the moments to single precision, useful for squeezing more parallelism out of a GPU when double-precision accuracy is not needed.

There are two distinct KPM code paths for an operator-weighted quantity (a projected DOS/spectral function): passing `operator=` reaches the operator-weighted moments directly (`kpm.tdos/kpm.pdos`'s `operator=` argument, or `h.get_kdos_bands(mode="KPM", operator=...)`), while `h.get_dos(mode="KPM", operator=...)` instead confines the random starting vectors to the operator's subspace and falls back to the plain (non-operator-weighted) moments internally. `kpm_cpugpu="GPU"` reaches both, but only the first actually exercises the operator-weighted GPU kernel (`kpm_momentsA_batch_gpu`).

Classical spin models

`classicalspin.SpinModel` models classical (non-quantized) spins on a lattice, each parametrized by a pair of angles (θ_i, ϕ_i) , interacting through a real-space tensor exchange $\vec{S}_i \cdot J_{ij} \cdot \vec{S}_j$ and an optional Zeeman field. Like `LatticeGas`, it reuses `Geometry` for the lattice/neighbor shells but is otherwise independent of the quantum `Hamiltonian` machinery: the energy is evaluated directly from the angles, and the ground state is found by local (gradient-based, via `jax` autodiff) multistart minimization rather than diagonalization – only the Γ point is supported, so incommensurate (e.g. spiral) textures need an explicit supercell

```
from pyqula import geometry
from pyqula import classicalspin

g = geometry.triangular_lattice() # geometrically frustrated lattice
g = g.get_supercell(3)

sm = classicalspin.SpinModel(g) # classical spin model on this geometry
```

```
sm.add_heisenberg(Jij=[1.0]) # first-neighbor antiferromagnetic exchange
sm.minimize_energy(tries=10) # multistart local minimization
```

```
mx,my,mz = sm.get_magnetization() # per-site magnetization components
```

`add_heisenberg` builds shell-based isotropic (or, via `Jm=[Jx,Jy,Jz]`, diagonally anisotropic/XXZ) couplings the same way `Geometry.get_hamiltonian(tij=...)` does. `classicalspin.generating_functions(name=...)` returns ready-made two-point coupling functions for other common forms – "Linear" (dipolar $1/r^3$), "RKKYTI" (RKKY on a topological-insulator surface, PRB 81 233405), "ZZ"/"XYZ" (Ising/anisotropic-diagonal), "DM" (Dzyaloshinskii-Moriya) – to pass into `add_tensor(fun)` (couplings within the home cell) or `add_tensor_2d(fun,ncells=...,vspiral=...)` (also sums periodic images, and can twist the coupling tensor by a per-image angle to embed a spin-spiral wavevector). `get_local_energy()` gives the per-site energy for spatially resolved maps (e.g. of a skyrmion or domain wall), and `classicalspintk.align.most_perp_basis()` rotates a magnetization texture into the frame where it is mostly in-plane, for quiver-style plotting. See `examples/classicalspin/` for runnable demos, including a frustrated triangular-lattice ground state and a modulated-exchange ladder.

Lattice gas models

`latticegas.LatticeGas` models classical, occupation-based (0/1) degrees of freedom on a lattice – e.g. adsorbates, vacancies, or any classical binary order parameter – interacting through a real-space coupling $J_{ij}n_i n_j$ and a site-dependent chemical potential $\mu_i n_i$. It reuses `Geometry` to define the lattice and its neighbor shells, but is otherwise independent of the quantum `Hamiltonian` machinery: the energy is evaluated directly from the occupation array, and the ground state is searched with a Metropolis-annealed discrete swap optimizer, not diagonalization

```
from pyqula import geometry
from pyqula import supercell
from pyqula import latticegas
```

```
g = geometry.triangular_lattice()
g = supercell.turn_orthorhombic(g)
g = g.get_supercell(10)
g.dimensionality = 0
```

```
lg = latticegas.LatticeGas(g,filling=1./3.) # 1/3 of the sites randomly occupied
lg.add_interaction(Jij=[1.,1.,1.]) # first, second and third neighbor repulsion
es = lg.optimize_energy(temp=0.5,ntries=1e4) # simulated annealing
```

`lg.den` holds the current 0/1 occupation array and `es` the energy tra-

jectory of the anneal. `get_local_energy()/get_local_mu()` give the per-site energy/chemical-potential contribution for the current snapshot, and `get_correlator()/get_structure_factor()` give the real-/reciprocal-space density-density correlator – useful for detecting ordered (e.g. striped, honeycomb-vacancy) ground states and their ordering wavevector. `anneal()` wraps `optimize_energy()` in a decreasing-temperature schedule, and `optimize_energy_multistart()` keeps the best of several independent restarts. `optimize_grand_canonical()` switches from fixed-filling swap moves to single-site flips under `lg.mu`, letting the filling itself fluctuate – useful for scanning a phase diagram vs. chemical potential, or for estimating thermodynamic quantities like the specific heat (`get_specific_heat()/get_susceptibility()`) from an equilibrium trajectory at fixed temperature. `add_tensor()` adds couplings beyond fixed neighbor shells, and `write()/read()` checkpoint a snapshot to/from disk. See `examples/latticegas/` for runnable demos of annealing, local-energy maps, correlators, and grand-canonical sampling.

Ising models

`latticeising.LatticeIsing` models classical Ising spins $s_i \in \{-1, +1\}$ on a lattice, interacting through a real-space coupling $-J_{ij}s_i s_j$ and a site-dependent field $-b_i s_i$ – the standard textbook Ising Hamiltonian, with $J > 0$ ferromagnetic (favoring alignment). It mirrors `LatticeGas` closely (same `Geometry`-driven pair list, CSR adjacency cache, and Metropolis machinery, several of whose module-level functions – the adjacency builder, `add_tensor()`, `regroup()`, `get_specific_heat()/get_susceptibility()` – are reused directly rather than reimplemented) but uses the **opposite** energy sign convention from `LatticeGas` (whose $\sum J_{ij}n_i n_j$ has no minus sign, so positive J there is a *repulsion*): with Ising spins, positive J_{ij} in `add_interaction()` means ferromagnetic alignment, matching the literature convention.

```
from pyqula import geometry
from pyqula import latticeising
```

```
g = geometry.square_lattice() # bipartite, so ferromagnetic order is not frustrated
g = g.get_supercell(12)
g.dimensionality = 0
```

```
li = latticeising.LatticeIsing(g,m=0.0) # random +/-1 spins, zero net magnetization
li.add_interaction(Jij=[1.]) # first-neighbor ferromagnetic coupling
es,ms = li.anneal(temps=[3.,1.,0.3,0.1,0.03],ntries=1e4) # simulated annealing
```

`li.s` holds the current ± 1 spin array. `li.optimize_energy()` runs single-spin-flip Metropolis dynamics – the standard Ising Monte Carlo move set, in which the total magnetization is *not* conserved (it fluctuates under `li.b`), so, mirroring `LatticeGas.optimize_grand_canonical()`, it returns `(es, ms)`: the energy and total magnetization ($\sum_i s_i$) trajectories, the latter usable di-

rectly with `latticegas.get_susceptibility()`. `li.optimize_conserved()` instead uses Kawasaki spin-exchange (swap) dynamics, which *does* conserve the total magnetization – the analog of `LatticeGas.optimize_energy()`’s fixed-filling swaps. `li.anneal()` wraps `optimize_energy()` in a decreasing-temperature schedule, and `optimize_energy_multistart()` keeps the best of several independent restarts. `get_local_energy()/get_local_field()` give the per-site energy/effective-field for the current snapshot, and `get_correlator()/get_structure_factor()` reuse `LatticeGas`’s real-/reciprocal-space correlator machinery directly (it operates on any per-site array, not just 0/1 occupations) to locate ordered (ferromagnetic, checkerboard antiferromagnetic, ...) ground states and their ordering wavevector. Because `li.pairs` lists both directions of every bond (same convention as `LatticeGas`), `get_energy()` is twice the usual sum-over-unordered-bonds convention – e.g. the 2d square-lattice ferromagnet’s critical temperature sits near 2×2.269 in these units, not 2.269. See `examples/latticeising/` for runnable demos of annealing, a temperature scan (magnetization and specific heat), and local-energy/local-field maps.

Parallelism and reproducibility

Parameter sweeps – over k-points, energies, q-points, restarts – are farmed out with `parallel.pcall`, which is serial by default. `parallel.set_cores(n)` gives it a pool of `n` worker processes, and `parallel.set_enabled(False)` forces the whole package strictly serial (no pool, numba and BLAS threads clamped to one), which is the setting to reach for when debugging or profiling.

```
from pyqula import parallel
parallel.set_cores(4)           # spread pcall's work over 4 processes
```

Results do not depend on the core count, including for the routines that draw random numbers – KPM’s stochastic trace, the multi-start anneals of the classical models. Each *task* is seeded from the parent process’s own random stream, so a task’s random numbers depend on its index in the sweep and not on which worker happened to run it: `cores=1` and `cores=8` give identical answers, and seeding the parent

```
import numpy as np
np.random.seed(42)
```

makes the whole sweep reproducible run to run. A `pcall` costs the parent exactly one draw off its own stream, whatever the tasks do with theirs.

Errors and unsupported inputs

Most routines in `pyqula` only make sense for a Hamiltonian of a particular dimensionality, or in a particular Hilbert space, or for one of a fixed set of

`mode=/solver=/channel=` strings. Those requirements are checked up front, and a violated one raises an exception whose message says what the routine needed:

- `ValueError` for a value that cannot work – the Berry curvature of a Hamiltonian that is not two-dimensional, an unknown `mode` string, a spin operator on a spinless Hamiltonian, an array of the wrong length
- `NotImplementedError` for a combination that is simply not built yet – a spin rotation of a 3d non-multicell Hamiltonian, say, where the fix is usually to call `h.turn_multicell()` first
- `TypeError` for an argument of the wrong kind entirely – a string where an `Operator` was expected

Where the check can name the offending value it does, so a message reads

```
ValueError: unknown mode ED2; the DOS accepts 'ED', 'KPM', 'adaptive', 'Green' and 'RG'
```

rather than only saying that something went wrong. For string-selected options the accepted values are listed in the message, which makes a typo self-diagnosing without a trip to the source.

Two of these messages are worth knowing in advance because they point at the fix rather than the failure: a superconducting quantity asked of a normal Hamiltonian says to call `h.setup_nambu_spinor()` first, and a spin quantity asked of a spinless one says to call `h.turn_spinful()`.

Those two requirements are checked by three shared guards, `require_spin`, `require_nambu` and `require_sublattice` in `pyqula.check`, so the message reads the same wherever it comes from and always names three things: what was being computed, what is missing, and how to get it

```
from pyqula import geometry
h = geometry.honeycomb_lattice().get_hamiltonian(has_spin=False)
h.extract("mz")
# ValueError: the magnetization 'mz' needs a spinful Hamiltonian; call
# h.turn_spinful() first, or build it with g.get_hamiltonian(has_spin=True)
```

They can be reused when adding a routine of your own. Note that the requirement is on *reading* a degree of freedom: the Hamiltonian methods that *add* a spin term – `h.add_exchange`, `h.add_zeeman`, `h.add_kane_mele` – promote a spinless Hamiltonian to a spinful one rather than rejecting it. The third guard covers the sublattice, which several sublattice-resolved quantities need and which an unlabeled cell does not have: `h.extract("CDW")` on such a cell says so instead of returning `None`, and so does `h.extract("superfluidity")` on a Hamiltonian with no electron-hole grading.

A second family of checks exists not because the routine cannot run, but because it could run and return a number that means nothing. Those raise rather than answer:

- `h.get_average_spin_splitting()`, `h.get_spin_splitting_density()` and `h.get_spin_splitting_vs_energy()` refuse a Hamiltonian whose spin off-diagonal block does not vanish (Rashba, any spin-orbit term, non-collinear order), naming the largest off-diagonal element found – they are built on `remove_spin`, which would drop that block without warning
- `h.get_dos()` on a non-Hermitian Hamiltonian refuses every mode but "ED", and refuses `use_kpm=True`: the Chebyshev and adaptive expansions assume a real spectrum (see “Non-Hermitian Hamiltonians”)
- `h.get_kdos_bands(frand=...)` refuses unless `mode="KPM"`, since only the KPM path draws the random vectors that argument supplies
- `h.get_total_energy(fermi=...)` refuses a Nambu Hamiltonian, where a nonzero `fermi` is not a rigid shift of the spectrum (the electron and hole blocks move opposite ways) – use `h.shift_fermi(-mu)` on the Hamiltonian instead
- `h.get_multidos(projection="atomic",operator=...)` refuses the combination and names `projection="TB"` as the one that projects
- the surface density of states refuses a momentum-dependent operator such as "valley", which its Green's function has already integrated over
- the lead decimation behind `HT.didv()`, `h.get_dos(mode="RG")` and `Embedding.get_gf()` refuses an energy sitting exactly on a lead level with a broadening below about `1e-7`, naming the energy, the broadening, the residual it reached and the tolerance it needed, and recommending a larger `delta`
- `h.check()`, the consistency check on a Hamiltonian, raises naming the deviation when a matrix is not Hermitian to within `tol` (`1e-5` by default), or when a Nambu Hamiltonian's electron-hole symmetry is broken

Passing an argument a routine cannot honour is also an error rather than a silent drop: an unknown keyword to `h.get_multidos()` is a `TypeError`, and so is passing both the old `op=` and the new `operator=` spelling to it.

Main functions and methods

Geometry functions and methods

`g.get_hamiltonian()`

Generate the Hamiltonian from a geometry.

Optional arguments

- `tij = [1.0,.0,0.]`: List with 1st, 2nd, 3rd nearest neighbor hopping, or a function of two positions returning the hopping between them, or a `specialhopping.HoppingGenerator`
- `has_spin=True`: include the spin degree of freedom
- `is_sparse=False`: store the matrices in sparse form, for large cells
- `non_hermitian=False`: build a non-Hermitian Hamiltonian, which

routes the band structure, DOS, LDOS and Berry curvature to their non-Hermitian implementations (see “Non-Hermitian Hamiltonians”)

- `is_multicell=False`: store the hoppings as a multicell dictionary
- `nc=2`: neighbor cutoff used by the multicell construction

An unrecognized keyword raises **TypeError** listing the accepted ones, so a misspelled `has_spin` is not silently the default.

Returns the Hamiltonian

`g.remove()`

Return a copy of the geometry with sites removed. Used to carve a vacancy or an antidot out of a flake or a supercell before building its Hamiltonian (see “Valley operator” for an example).

Arguments

- `i=0`: which sites to drop – a single index into `g.r`, a list of indices, or a callable of the position, in which case every site where it returns True is removed

Returns a new geometry; `g` itself is untouched

`g.get__supercell()`

Generate a supercell

Arguments

- `nsuper`: size of the supercell to create, number or tuple, or a 3x3 integer matrix `M` for a general non-diagonal/non-orthogonal supercell (see “Electronic structure folding and unfolding” for how this interacts with `operator="unfold"`) (positional, or by that name)

Optional arguments

- `store_primal=False`: keep a reference to the primitive-cell geometry on the supercell, needed by `operator="unfold"` (see “Electronic structure folding and unfolding”)

Returns a new geometry

Hamiltonian functions and methods

`h.get__bands()`

Compute band structure

Optional arguments:

- `nk = 400`: number of k-points

- `operator`: a single operator, or a list of operators, to compute expectation values for at each eigenstate
- `kpath`: an explicit k-path, either as reduced coordinates or as a list of high-symmetry labels ("G", "M", "K", "X", "Y", and in three dimensions "Z", "R", "A", "B")
- `ewindow`: a function of the energy returning whether to keep that band, used to restrict the output to an energy window (e.g. `ewindow=lambda e: abs(e)<0.5`)
- `eigmode="complex"`: non-Hermitian Hamiltonians only – which part of the complex eigenvalue is returned and written, "complex", "real" or "imag" (see “Non-Hermitian Hamiltonians”). With "complex" the written `BANDS.OUT` carries `k`, `Re E`, `Im E` and then the operator columns

Without `kpath` the path is Γ -M for a square-like 2D lattice, Γ -K-M-K'- Γ for a triangular-like one, and Γ -X-M- Γ -R for a 3D one – the 3D path genuinely leaves the $k_3 = 0$ plane, so the band edges of a 3D crystal are reached.

Returns kpoint index and energies, plus one extra row per operator if `operator` is given

`h.get_kdos_bands()`

Compute a k-resolved spectral function (band structure dressed with a projection operator, or an unfolded spectral function) along a k-path.

Optional arguments:

- `kpath`: k-point path (auto-generated if not given)
- `operator=None`: operator used to weight the spectral function, e.g. "unfold" (see “Electronic structure folding and unfolding”)
- `energies`, `delta`, `nk`: frequency range, broadening, k-point density
- `mode="ED"`: "ED" or "KPM" (equivalently `use_kpm=True`)
- `frand=None`: generator of the random vectors the KPM stochastic trace draws. Only the KPM path uses them, so passing it without `mode="KPM"` raises `ValueError` saying so, rather than being dropped

Returns k-path fraction, energy and spectral weight

`h.get_dos()`

Compute the density of states, normalized so that its integral over the energies is the number of states per unit cell.

Optional arguments:

- `energies`: array with frequencies of the DOS

- `delta=None`: broadening of the DOS. Left as `None` it is chosen from the `k`-mesh, $5/nk$ (so 0.05 at the default `nk=100`), which keeps the curve smooth as the mesh is refined
- `mode="ED"`: "ED", "KPM", "adaptive", "Green" or "RG" (see the “Density of states” section). Anything else raises, listing those five
- `nk=100`: `k`-points per direction, for "ED" and "KPM". "adaptive" integrates with error-controlled quadrature (`error=1e-1`) instead of sampling a mesh, and uses `nk` only as a subdivision limit; "Green"/"RG" forward it to the self-energy’s own `k`-sum, where it matters for `gmode="full"` and (in 2D) `gmode="renormalization"`
- `operator=None`: operator the DOS is projected onto (a name, a matrix or an `Operator`)

For a non-Hermitian Hamiltonian only `mode="ED"` exists, and the extra `eigmode` argument chooses which part of the complex eigenvalue the broadening is centred on – see “Non-Hermitian Hamiltonians”.

Return energies and DOS

`h.get_gap()`

Return the indirect gap, i.e. the smallest energy difference between an empty and an occupied state anywhere in the Brillouin zone (the two need not sit at the same `k`-point). Obtained by numerically minimizing over `k` rather than by scanning a fixed mesh, so a gap that closes at an incommensurate `k`-point is not missed.

Optional arguments:

- `ntries=1`: repeat the minimization this many times and keep the smallest result. Since the search was made deterministic (a fixed coarse grid picks the starting point, and `differential_evolution` runs with a fixed seed) every repetition returns the identical number, so raising this only costs time – it is kept for backwards compatibility

Returns a single number, the gap. Zero (up to numerical noise) for a metal or a Dirac semimetal

`h.get_bandwidth()`

Return the bottom and top of the spectrum, (`emin,emax`) – note this is the pair of band edges, not their difference. Uses the same `k`-space optimization as `h.get_gap()`, so the edges are the true extrema over the Brillouin zone rather than the extrema of a `k`-mesh sample

h.get_filling()

Return the fraction of states below zero energy, i.e. the filling measured with the Fermi energy at $E = 0$. Half filling gives 0.5. Use **h.set_filling(nu)** to shift the onsite energy so that a target filling is realized, and this method to check the result.

For a Hamiltonian with the electron-hole (Nambu) degree of freedom the spectrum is particle-hole symmetric, so exactly half of it lies below zero whatever the density is; there the filling is instead the weight that the occupied BdG states put on the electron components, normalized by the two electron states per site. At zero pairing this reproduces the normal-state filling exactly.

Optional arguments:

- **nk**: k-point density used to sample the spectrum

h.get_total_energy()

Return the total energy, i.e. the sum of the occupied single-particle eigenvalues. For a mean-field Hamiltonian this is the band energy only – **h.get_mean_field_hamiltonian(...,return_total_energy=True)** returns the interacting total energy including the double-counting correction instead

Optional arguments:

- **nk=10**: k-point density of the Brillouin-zone sum
- **fermi=0.0**: energy below which states are counted as occupied. Not accepted for a Nambu Hamiltonian (raises **ValueError**): a nonzero **fermi** is not a rigid shift of a BdG spectrum, since the electron and hole blocks shift by $-\mu$ and $+\mu$ – shift the Hamiltonian instead, with **h.shift_fermi(-mu)**, and leave **fermi=0**
- **mode="mesh"**: k-space sampling; **use_kpm=True** switches to a Chebyshev estimate for large systems

For a Nambu/BdG Hamiltonian this returns the *electronic* energy, i.e. the energy of the physical electrons rather than of the doubled Nambu spectrum, so at zero pairing it agrees with the normal-state answer for the same model. **nbands=** is not implemented there (raises **NotImplementedError**). Note the caveat under “Spin-spin exchange interactions”: the double-counting correction subtracted by the mean-field total energy is the normal (Hartree-Fock) one only, never a matching anomalous one, so a total energy with genuine pairing is still missing that term

h.get_density_matrix()

Return the full density matrix of the occupied states, as a dense matrix in the same basis as **h.intra**. See “Interactions at the mean-field level” for the k-resolved, hopping-resolved version the self-consistent loops use.

Index convention. This is $\text{dm}_{ij} = \sum_{\text{occ}} \psi_i^* \psi_j$, the *transpose* of the usual $\rho_{ij} = \sum_{\text{occ}} \psi_i \psi_j^*$ – the mean-field machinery is built around it. So an expectation value is `np.trace(dm.T@A)`, not `np.trace(dm@A)`: the two agree for any real operator (the density, σ_x , σ_z , a projector) and differ by a sign for a purely imaginary one (σ_y , the valley operator, any current operator $i[H, r]$). Use `h.get_vev(operator)` rather than contracting the matrix yourself

`h.get_ipr()`

Return the inverse participation ratio of every eigenstate, as `(energies, ipr)`. A delocalized state in a system of N sites gives $\text{IPR} \sim 1/N$ and a state localized on one site gives $\text{IPR} \sim 1$, so this is the usual diagnostic for Anderson localization or for in-gap bound states. **Finite (0d) systems only** – it raises `NotImplementedError` for a periodic Hamiltonian; for those use the IPR operator instead (see “Inverse participation ratio operator”)

`h.get_vev()` / `h.get_single_vev()` / `h.get_several_vev()`

Ground-state expectation values of operators, evaluated by summing over the occupied states.

- `h.get_vev(operator=...)` returns one real number **per site**: the site-resolved expectation value of `operator` (any name accepted by `h.get_operator`, e.g. “sz”), or the site occupation if `operator` is omitted. This is what produces a magnetization or charge-density map
- `h.get_single_vev(A)` returns the single number $\langle A \rangle$ for one operator `A`, summed over the whole system
- `h.get_several_vev([A,B,...])` does the same for a list of operators in one pass, sharing the diagonalization

Optional arguments:

- `nk=30`: k-point density of the Brillouin-zone sum

`h.get_magnetization()`

Site-resolved magnetization, as an `(nsites,3)` array. Two different quantities go by this name, and the `mode` argument picks between them:

- `mode="vev"` (the default) returns the physical moment: the per-site expectation value ($\langle S_x \rangle, \langle S_y \rangle, \langle S_z \rangle$) over the occupied states, i.e. `get_vev("sx"/"sy"/"sz")`. Being a Brillouin-zone integral it needs a k-mesh – pass `nk`, or rely on the mesh a self-consistent Hamiltonian remembers from its own loop. On a metal at $T = 0$ the moment is quantized in steps of $2/n_k$, so use a fine enough mesh (or `mode="field"`) when comparing weakly polarized states
- `mode="field"` reads the magnetic *term written in the Hamiltonian* instead, i.e. the coefficients of $\sigma_{x,y,z}$ on each site. After a self-consistent calculation

this is the mean-field exchange field: the natural order parameter of the loop, proportional – not equal – to the moment, and continuous where the moment is quantized. On a Hamiltonian whose field you put in by hand it hands that field straight back

Note the sign: `add_zeeman/add_exchange` add $+\vec{h} \cdot \vec{\sigma}$, so the occupied states polarize *against* $\vec{h}$ and the moment comes out antiparallel to the field you applied.

```
from pyqula import geometry
h = geometry.chain().get_hamiltonian()
h.add_exchange([0.,0.,0.5])
h.get_magnetization(nk=40)           # the moment: [0,0,-0.15], against the field
h.get_magnetization(mode="field")    # [0,0,0.5], the field you put in
```

Optional arguments:

- `mode="vev"`: "vev" or "field", as above
- any further keyword (e.g. `nk`) is forwarded to `get_vev` in "vev" mode

`h.get_topological_invariant()`

Return a topological invariant of the occupied bands: the Berry (Zak) phase in one dimension, the Z_2 invariant in two dimensions when the Hamiltonian is time-reversal symmetric, and the Chern number otherwise. A 0d Hamiltonian has no Brillouin zone and 3D is not implemented, so both raise; for a 3D system take the Chern number of a 2D slice, or `topology.berry_phase(h,kpath=...)` along a chosen path.

`h.add_soc()`

Add Kane-Mele intrinsic spin-orbit coupling

Arguments:

- `t`: value of the SOC (positional, or by that name)

`h.add_zeeman()`

Add a Zeeman field to the Hamiltonian

Arguments:

- `zeeman`: value of the Zeeman, as a number (assumes $[0,0,B_z]$), array or callable function (positional, or by that name)

`h.add_rashba()`

Add Rashba spin-orbit coupling

Arguments:

- `c`: value of the Rashba SOC (positional, or by that name)

h.add_onsite()

Add a local onsite energy

Arguments:

- `fermi`: value of the onsite energy, as a number, an array with one entry per site, or a callable of the position (positional, or by that name)

h.add_sublattice_imbalance()

Add a staggered onsite energy, $+m$ on one sublattice and $-m$ on the other – the mass term that gaps a honeycomb lattice into a boron-nitride-like semiconductor

Arguments:

- `mass`: the imbalance, as a number or as a callable of the position

The term is written with the geometry’s sublattice index, so it needs one. A geometry with no sublattice (a chain, a triangular or square lattice) and a geometry with more than two (kagome, pyrochlore, whose index runs 0, 1, 2, ... rather than ± 1) both raise a `ValueError` rather than returning an unchanged Hamiltonian. On a bipartite lattice you can label one yourself with `g = g.get_supercell(2)` followed by `g.get_sublattice()`; otherwise write the profile explicitly with `h.add_onsite(f)`.

h.add_antiferromagnetism()

Add a staggered exchange field: a Neel pattern on two sublattices, and the 120-degree frustrated pattern on more than two

Arguments:

- `mass`: the exchange field, as a number or as a callable of the position

Like `add_sublattice_imbalance` this needs a sublattice and raises a `ValueError` when the geometry has none. Unlike it, more than two sublattices are supported, through the frustrated pattern

h.add_haldane()

Add a Haldane term: a complex second-neighbor hopping whose sign is set by the chirality of the two-step path, which opens a gap and makes a honeycomb lattice a Chern insulator (see “Chern number”).

Arguments

- `t`: amplitude of the second-neighbor hopping, a number or a callable of the position

Needs a geometry with a sublattice. Breaks time-reversal symmetry

h.add_modified_haldane() / h.add_antihaldane()

Same second-neighbor complex hopping, but with the sign also flipped between the two sublattices, so the two valleys acquire opposite masses and the total Chern number is zero – a valley-Hall rather than a Chern insulator. `add_antihaldane` is a second name for the same method.

Arguments

- `t`: amplitude of the second-neighbor hopping

h.add_kane_mele()

Add a Kane-Mele spin-orbit term: the Haldane hopping with opposite sign for the two spin channels, so time-reversal symmetry is preserved and the model is a quantum spin Hall insulator instead of a Chern insulator. Turns the Hamiltonian spinful if it is not already.

Arguments

- `t`: amplitude of the spin-orbit second-neighbor hopping

Its spin Chern number is what `topology.spin_chern` computes

h.add_anti_kane_mele()

The sublattice-staggered counterpart of `add_kane_mele`, in the same way `add_modified_haldane` is the counterpart of `add_haldane`.

Arguments

- `t`: amplitude of the second-neighbor hopping

h.add_kekule() / h.add_chiral_kekule()

Add a Kekule bond modulation: a period-tripling pattern on the first-neighbor bonds of a honeycomb lattice, which folds the two Dirac points onto Γ and gaps them. `add_kekule(t)` takes the modulation amplitude. `add_chiral_kekule(t1=...,t2=...)` adds the bond-direction-aware chiral version, whose two amplitudes are the two independent complex components of the modulation, optionally on an explicit `registry=` of retained hexagon centers rather than the default one

h.add_valley_exchange()

Add a valley-space exchange term $\vec{v} \cdot (\tau_x, \tau_y, \tau_z)$, the valley-pseudospin analogue of `add_exchange` for real spin (see “In-plane valley operators”).

Arguments

- $v = (v_x, v_y, v_z)$: the valley field

Built from the same in-plane valley operators, so for a periodic Hamiltonian it needs a Kekule-commensurate cell – a multiple-of-3 supercell of the primitive honeycomb cell – and raises `ValueError` otherwise. A finite (0d) flake needs no such commensurability

h.add_peierls() / h.add_orbital_magnetic_field()

Add an out-of-plane orbital magnetic field as a Peierls phase on every hopping. `add_orbital_magnetic_field` is a second name for the same method.

Arguments

- `mag_field`: the field, in units of the flux quantum per unit area of the lattice – the Peierls phase on a bond is $2\pi B y dx$ in the Landau gauge. A periodic (commensurate) calculation needs the flux through the unit cell to be a rational multiple of the flux quantum, so build the supercell first and pick `mag_field` to match it

Optional arguments

- `gauge="Landau"`: "Landau" or "symmetric"

Refuses a Hamiltonian that already carries a pairing amplitude (`NotImplementedError`): a Cooper pair has charge $2e$, so the anomalous term has no single Peierls phase, and an orbital field in a superconductor means vortices. Add the field to the normal-state Hamiltonian first, then `h.turn_nambu()/h.add_swave()`

h.add_inplane_bfield()

Add an in-plane magnetic field, as a Peierls phase built from the out-of-plane coordinate – meaningful for a multilayer or a system with finite thickness, where an in-plane field still threads flux between the layers.

Optional arguments

- `b=0.0`: field strength
- `phi=0.0`: in-plane direction of the field, in units of π

Only implemented up to two dimensions

h.add_strain()

Modify the hoppings according to a strain field, the standard way of producing a pseudo-magnetic field in graphene.

Arguments

- `sr`: a callable of the position returning the local strain

Optional arguments

- `mode="scalar"`: `"scalar"` rescales every hopping by `sr(r)` alone; `"directional"` also uses the bond direction, for a genuinely anisotropic strain

Turns the Hamiltonian multicell. Not implemented for a spinless Nambu Hilbert space

`h.add_crystal_field()`

Add a crystal field: an onsite potential built from the local atomic environment, so that sites with fewer or more distant neighbors (an edge, a vacancy, the two inequivalent stackings of a bilayer) sit at different energies.

Arguments

- `v`: strength of the crystal field. The built-in potential is normalized and has its average removed, so `v` sets the spread of the onsite energies rather than their offset

Optional arguments

- `rcut=6.0`: distance cutoff of the neighbor sum

`h.generate_spin_spiral()`

Rotate the Hamiltonian into a spin-spiral ansatz, i.e. impose a magnetic texture whose quantization axis winds with a wavevector q . Used both as a starting point for a spiral mean-field calculation and to scan the energy of the spiral as a function of q (see “Spin-spin exchange interactions”).

Optional arguments

- `vector=[0.,0.,1.]`: axis the spins rotate about
- `qspiral=[1.,0.,0.]`: the spiral wavevector
- `fractional=True`: read `qspiral` in fractional (reduced) coordinates. With `False` only the inter-cell hoppings are rotated

Needs a spinful Hamiltonian

`h.add_pairing()`

Add a general superconducting pairing term, given by its symmetry channel, turning the Hamiltonian into its Nambu (BdG) form if it is not already (see “Spin-triplet d-vector and non-unitary superconductivity”).

Optional arguments

- `delta=0.0`: pairing amplitude, a number or a callable of the position
- `mode="swave"`: the pairing symmetry. The accepted names are listed in `pyqula.sctk.pairing.pairing_modes` – `"swave"`, `"extended_swave"`, `"triplet"`, `"pwave"`, `"chiral_pwave"`, `"dx2y2"`, `"dxy"`, `"nodal_dwave"`,

"chiral_dwave", "nodal_fwave", "chiral_fwave", "chiral_gwave", "haldane", "antihaldane", and several others. A callable returning the 2x2 pairing matrix for a pair of positions is also accepted. An unknown name raises `ValueError` listing every accepted one

- `d=[0.,0.,1.]`: the d-vector, for the triplet channels

h.setup_nambu_spinor()

Put the Hamiltonian into its Nambu (electron-hole doubled) form, with zero pairing. This is the call the error messages of the superconducting routines point at: a superconducting quantity asked of a normal Hamiltonian needs the electron-hole degree of freedom to exist first. Equivalent to `h.add_swave(0.0)`. Modifies in place

h.turn_nambu()

Add the electron-hole degree of freedom without adding any pairing. Lower level than `setup_nambu_spinor()`, which is what user code normally wants. Modifies in place

h.turn_spinful()

Add the spin degree of freedom to a spinless Hamiltonian, doubling every matrix with an identity in spin space. This is the call the error messages of the spin-resolved routines point at. Modifies in place, and does nothing if the Hamiltonian is already spinful

h.remove_spin()

The inverse: keep one spin block and discard the other, returning to a spinless Hamiltonian.

Optional arguments

- `channel="up"`: which block to keep, "up" or "dn"

This **drops the spin off-diagonal block**, so it is only meaningful when spin is a good quantum number – with spin-orbit coupling or non-collinear order the result is a different model, not a projection of this one. The spin-splitting routines built on it check that explicitly (see “Errors and unsupported inputs”)

h.turn_multicell()

Convert the Hamiltonian to multicell form, i.e. store the inter-cell hoppings as a dictionary keyed by lattice vector instead of the single `inter` matrix of the nearest-neighbor-cell form. Several routines require it, and their error messages say so; hoppings beyond the first neighboring cell need it. Modifies in place. `h.get_no_multicell()` returns a non-multicell copy where that is possible

h.get_hk_gen()

Return the Bloch generator: a function of the reduced k-vector returning the Bloch matrix $H(k)$ of the Hamiltonian. This is the object every k-space routine in the library is built on, and the way to evaluate a model at a chosen k-point directly (`h.get_hk_gen()([0.3,0.,0.])`). pyqula's convention is the periodic one, $H(k) = \sum_R t(R)e^{2\pi i k \cdot R}$ with k in reduced coordinates and no intra-cell atomic positions in the phases

h.get_gf()

Return the bulk Green's function of the Hamiltonian at one energy, as a matrix in the same basis as `h.intra`, obtained by integrating over the Brillouin zone.

Optional arguments

- `energy=0.0`: energy at which it is evaluated
- `delta=1e-5`: imaginary part (analytic continuation)
- `mode="adaptive"`: how the Brillouin-zone integral is done – "adaptive" (error-controlled), "full" (a fixed `nk` mesh) or "renormalization"
- `gtype="bulk"`: "bulk" or "surface"

– $\text{Im Tr } G/\pi$ is the density of states, which is what `h.get_dos(mode="Green")` computes from it

h.get_hopping_dict()

Return the real-space hoppings as a dictionary `{(n1,n2,n3): matrix}`, keyed by the lattice vector connecting the two cells, with (0,0,0) the intracell block. This is the representation the response functions accept for an interaction with support beyond one cell (see "Interactions beyond onsite"), and the natural way to inspect or modify a Hamiltonian's hoppings directly. `h.get_multihopping()` returns the same thing wrapped in a `multihopping.MultiHopping`, which supports addition and scalar multiplication, and `h.set_multihopping()` writes one back

h.get_rkky()

Compute the RKKY interaction between two magnetic impurities mediated by the electrons of `h`, i.e. the effective exchange coupling as a function of their separation.

Optional arguments

- `mode="pm"`: "pm" computes it explicitly ("poor man's"), by adding two local exchange fields to the Hamiltonian and taking the energy difference between their parallel and antiparallel alignment; "LR" computes it from linear response instead, which is much faster for a map over many separations. An unknown mode raises `ValueError` listing both

- for `mode="pm"`: `ri`, `rj` the positions of the two impurities (both required), `nk=10`, `dj=1e-1` the strength of the probe exchange fields
- for `mode="LR"`: `R=[0,0,0]` the lattice vector between the two cells, `ii=0`, `jj=0` the sites inside them, `nk=100`, `delta` the analytic continuation (default `1/nk`)

`rkky.rkky_map(h,n=...)` sweeps either mode over a range of separations. On a bipartite lattice at half filling the sign follows the usual theorem: ferromagnetic between sites of the same sublattice, antiferromagnetic between opposite ones

`h.get_ldos()`

Compute the local density of states.

Optional arguments:

- `e`: energy of the LDOS (`energy` is accepted as an alias, since that is how the Green's function, embedding and transport routines spell it)
- `delta=0.001`: broadening of the LDOS, which must be positive
- `operator=None`: operator the LDOS is projected onto (a name, a matrix or an `Operator`); see the LDOS section above for how the two modes weight it
- `mode="arpack"`: `"arpack"` (diagonalization on a k-mesh) or `"green"` (Green's function, 2D only)
- `projection="TB"`: `"TB"`, `"TBRS"` (real-space interpolated) or `"atomic"`
- `eigmode="complex"`: non-Hermitian Hamiltonians only – whether `e` is read on the real or the imaginary axis of the complex spectrum (see “Non-Hermitian Hamiltonians”). Only `mode="diagonalization"` is implemented there

Return `x`, position, `y` position and LDOS

`h.get_multildos()`

Compute the LDOS at many energies, writing one file per energy to a `MULTILDOS/` folder, together with a `MULTILDOS/DOS.OUT` holding the total DOS on the same energies and a `DOSMAP.OUT` in the working directory. The maps and that DOS carry the same normalization as `h.get_ldos()` and `h.get_dos()`, so they can be read against each other directly.

Optional arguments:

- `energies=linspace(-1,1,100)`: energies to compute
- `delta`, `nk`: broadening and k-point density, as in `h.get_ldos()`
- `operator=None`: operator the LDOS is projected onto, weighting each eigenstate by $\langle \Psi | A | \Psi \rangle$ – the same convention `h.get_ldos(mode="arpack")`

uses. The older spelling `op=` is still accepted; passing both raises `TypeError`

- `projection="TB"`: "TB" or "atomic". Anything else raises `ValueError` listing the two, and "atomic" together with `operator=` raises `NotImplementedError` naming "TB" as the projection that supports it

An unrecognized keyword raises `TypeError` rather than being ignored.

`h.get_chi()`

Compute a non-interacting operator-operator response function (charge-charge by default).

Optional arguments:

- `q=None`: momentum transfer. Left as `None` the response is *averaged over the whole q-mesh* rather than evaluated at `q=0` – pass `q=[0.,0.,0.]` explicitly for the uniform response
- `A=None, B=None`: operators defining the response (default: identity, i.e. charge-charge)
- `energies, delta, nk`: frequency range, broadening, k-mesh density

Returns energies and the response function

`h.get_spinchi_ladder()`

Compute the transverse (S^+/S^-) spin susceptibility, RPA-dressed by default using the Hubbard U of a mean-field Hamiltonian.

Optional arguments:

- `q=None, energies, delta, nk`: as above, `None` again meaning the q-average
- `RPA=True`: dress with the random-phase approximation; `False` for the bare response
- `chi_cpugpu="CPU"`: backend for the underlying Lindhard kernel; "GPU" dispatches it to jax (falling back to jax's CPU backend if no device is visible). Also accepted by `get_spinchi_full`, `get_qdos_iets`, `get_iets_ldos`, `get_rpa_kernel_poles` and `get_magnon_bands(method="rpa")`, which all share that kernel. Unsupported combinations (`mode="trace"/"diagonal"`, `imode="adaptive"`) raise instead of falling back to the CPU

`h.get_rpa_kernel_poles()`

Compute the poles of the generic RPA kernel $1 - V(q)\chi(q,\omega)$: the frequencies of the collective modes/instabilities of the interacting response.

Optional arguments:

- `V=None` (required): the interaction; a `ValueError` is raised if not given. Either a plain matrix (q-independent, onsite-only) or a real-space hopping dict/`MultiHopping` `{(n1,n2,n3): matrix}` for an interaction with support beyond the onsite cell, Fourier-transformed to $V(q)$ at this call's `q` (see “Interactions beyond onsite”)
- `A=None`, `B=None`, `q=None`, `energies`, `delta`, `nk`: as in `get_chi`, `None` again meaning the q-average

Returns an `(npoles,2)` array: pole frequency and its (signed) residual imaginary part – filter on its magnitude, not its raw value, to keep only sharp/well-defined modes – one row per collective mode found, sorted by frequency.

`h.get_magnon_bands()`

Compute the magnon bands of a magnetic mean-field state, scanned along a q-path. Two methods, with different domains of validity:

Optional arguments:

- `method="rpa"` / `"pair"` / `"tdhf"`: which ladder to sum. See “The three magnon routes” above for the coverage table; in short, `"rpa"` is the site basis (onsite `U` or neighbour-shell exchange), `"pair"` keeps the interaction's pair index (any density-density interaction, onsite or not, metals included, frequency-resolved), `"tdhf"` solves the electron-hole pair eigenproblem (any density-density interaction, no frequency grid).
- `method="rpa"`: the poles of the full spin RPA kernel (the same S_x, S_y, S_z channel as `get_spinchi_full/get_iets_ldos`, with the interaction taken from the mean field – an onsite `h.V`, or a neighbor-shell **exchange** interaction through `h.Vchannels`, which the SCF records; a neighbor-shell density-density interaction is refused). Works for metals as well as insulators, and needs a frequency grid. `method="tdhf"` instead solves the time-dependent Hartree-Fock problem in the spin-flip electron-hole pair basis: it handles a neighbor-shell density-density interaction, has an exact Goldstone mode, needs no frequency grid, and requires a gapped reference converged on the same `nk`
- `qpath=None`, `nq=20`: the q-path (default path of the geometry) and number of q-points
- `energies`, `delta`, `nk`: as above (`method="rpa"` only)
- `nk`, `n`, `channel`, `V`: for `method="tdhf"` – the k-mesh (which must match the SCF's), how many branches to keep per q-point, whether to restrict to the spin-flip block (`"auto"`, `"spinflip"`, `"all"`), and an interaction overriding `h.V`

Returns `(qs,ws,gammas)` for `method="rpa"`: three flat 1D arrays of equal length, `qs` the integer q-point index along the path, `ws` the pole frequency,

`gammas` its residual imaginary part. `method="tdhf"` returns `(qs,es)`, with `es` the (complex) magnon energy.

h.get_transverse_spinchi()

Return the spin response computed in the basis of the interaction's *pair* index rather than of sites, which is what lets it carry a neighbour-shell density-density interaction – the one the site-basis RPA maps to exactly zero. Needs no gapped reference and no global spin quantization axis, so it covers metals and non-collinear states alike, and returns a frequency-resolved, spin- and site-resolved $\chi(\omega)$.

Optional arguments:

- `W=None`: the interaction, defaulting to the one the mean field was converged with
- `q=None`, `energies`, `delta`, `nk`: as in `get_chi`, `None` again meaning the q-average
- `component=None`: a pair of spin indices `(a,b)` to return only that spin block instead of the full tensor

Returns `(energies,chi)` with `chi` one $3N \times 3N$ tensor per frequency, in the same $(S_x, S_y, S_z) \times \text{site}$ layout `get_spinchi_full` uses.

h.get_magnon_energies()

Return the magnon energies at a single center-of-mass momentum `Q`, from the spin-flip channel of the Bethe-Salpeter equation. Same arguments as `get_magnon_bands(method="tdhf")` with `Q=[qx,qy,qz]` in place of the q-path. A sizable imaginary part on an energy means the mean-field reference is unstable against that excitation.

h.get_goldstone_residual()

Return how far a magnetic mean field is from having a zero-energy magnon at $Q = 0$, as the Goldstone theorem requires of any magnetic state without spin-orbit coupling: $\|Mv\|/\|v\|$ with M the time-dependent Hartree-Fock matrix and v the uniform spin-rotation generator in the pair basis. It is proportional to the SCF tolerance the mean field was converged to and to nothing else, so it is the check to run before trusting a magnon dispersion – in particular it is what catches a mean field converged on a different `nk` than the magnon is being solved on.

Optional arguments:

- `nk=10`, `V=None`, `channel="auto"`: as in `get_magnon_energies`
- `relative=True`: divide by the largest transition energy of the pair basis, so the number is comparable across models with different bandwidths

h.get__densitychi__RPA()

Compute the density (charge) RPA response function for a **V1/V2/V3**-neighbor-shell (+ onsite **U**, + general **Vr(r)**) density-density interaction, same convention as **Vinteraction/VJinteraction**. Unlike **get_spinchi_full**, the interaction is taken directly as parameters, not read from **h.V** – no mean-field convergence is needed first.

Optional arguments:

- **V1=0.0, V2=0.0, V3=0.0, U=0.0, Vr=None**: the density-density interaction, built the same way as **Vinteraction/VJinteraction**'s
- **q=None, energies, delta, nk**: as in **get_chi**, **None** again meaning the q-average

h.get__plasmon__bands()

Compute the plasmon/charge-order bands: the poles of the density RPA kernel for a **V1/V2/V3/U/Vr** neighbor-shell density-density interaction, scanned along a q-path – the charge-channel analog of **get_magnon_bands**.

Optional arguments:

- **V1=0.0, V2=0.0, V3=0.0, U=0.0, Vr=None**: as in **get_densitychi_RPA**
- **qpath=None, nq=20, energies, delta, nk**: as in **get_magnon_bands**

Returns (**qs,ws,gammas**), same convention as **get_magnon_bands**.

h.get__bse()

Solve the Bethe-Salpeter equation (excitons) on top of this mean-field Hamiltonian, and return the solved **BSE** object.

Optional arguments:

- **V=None**: the electron-hole interaction. **None** reads it from **h.V**, the interaction the mean field was converged with (so the BSE kernel and the Fock self-energy inside **h** come from the same interaction). Otherwise a real-space dictionary **{(n1,n2,n3): matrix}**, or a plain matrix for an onsite-only interaction – **bsetk.interaction.density_interaction** builds one from **U/V1/V2/V3/Vr**
- **Q=None**: center-of-mass momentum of the exciton, defaulting to the zone center
- **nk=10**: k-points per direction of the mesh the electron-hole pairs are built on
- **nv=None, nc=None**: restrict to the **nv** highest valence and **nc** lowest conduction bands; **None** takes all of them

- `kernel="full"`: which kernel terms to include – `"full"`, `"direct"` (ladder only), `"exchange"` (exactly the RPA) or `"none"` (bare transitions)
- `tda=False`: apply the Tamm-Dancoff approximation, diagonalizing only the resonant block
- `max_memory=2.0`: refuse, rather than attempt, a calculation whose dense matrix would need more than this many GB
- `screening=None`: the interaction of the *direct* (ladder) term. `None` keeps the bare one, i.e. time-dependent Hartree-Fock; `"rpa"` replaces it by the static RPA screened interaction $W = \varepsilon^{-1}v$ built from this Hamiltonian's own bands; `"crpa"` does the same with the transitions inside the `nv/nc` window left out of the polarization; a `ScreenedInteraction` reuses a pre-computed one. The exchange term always keeps the bare interaction. Do not screen a fitted Hubbard U – see the section above
- `nkW=None`: k-mesh for the screening, defaulting to `nk`. Must be an integer multiple of it
- `channel="charge"`: where the dielectric matrix is built. `"charge"` builds it on site indices (the standard GW construction, spin-rotation invariant); `"orbital"` dresses the full spin-orbital matrix as $\varepsilon^{-1}v$, which does not preserve $SU(2)$. The two coincide for a spinless Hamiltonian
- `solver="dense"`: how the eigenproblem is solved. `"dense"` builds the matrix and diagonalizes it, and is the only route to the full non-Tamm-Dancoff spectrum. `"iterative"` applies the exactly factorized kernel matrix-free and runs a preconditioned block LOBPCG, removing the memory wall. `"qtt"` compresses the kernel into a quantics matrix product operator and solves it by DMRG, with a cost growing like $\log N_k$. The last two need `tda=True`; see “Large k-meshes” above
- `neig=None`: how many excitons `solver="iterative"` returns; `"dense"` returns all of them and ignores it, and `"qtt"` accepts only `neig=1`. `None` means 4 for `"iterative"` and 1 for `"qtt"`
- `gauge="auto"`: smooth the arbitrary phase left on each Bloch eigenvector. `"auto"` turns it on (as `"projection"`) only for `solver="qtt"`, which cannot work without it; `"phase"`, `"projection"` or `None` apply to every solver. It changes no energy, being a unitary on the pair index

`solver="qtt"` additionally takes `tolerance` (cross-interpolation tolerance of the kernel MPO, default `1e-6`), `maxbonddim`, `maxdim/nsweep/cutoff` (the DMRG parameters), `coarse_nk` (the submesh the band window and gauge references are read from) and `unfolding` (`"grouped"`, the measured default, or `"interleaved"`).

The returned object exposes `energies`, `amplitudes` (the resonant amplitudes $A_{vc}(k)$), `amplitudesY` (the antiresonant ones, zero under `tda`),

`pairs` (the k-mesh, band window and (ik,iv,ic) label of every pair index), and the methods `get_energies(n)`, `get_binding_energies(n)` and `get_lowest_transition()` – the last being the lowest independent-particle transition the binding energies are measured from, obtained as a mesh minimum for the first two solvers and, for "qtt", by a binary descent on the mesh index seeded from the k-points already visited (a minimum over the whole mesh would be the one step of that solver that is not logarithmic).

h.get_exciton_energies()

Return the exciton energies from the Bethe-Salpeter equation, sorted. Takes the same arguments as `get_bse`, plus `n=None` to keep only the `n` lowest.

h.get_exciton_binding_energies()

Return the exciton binding energies, i.e. how far below the lowest independent-particle transition each exciton lies; positive means bound. Same arguments as `get_exciton_energies`.

h.get_exciton_states()

Return (`energies,amplitudes`) of the excitons, the amplitudes being the electron-hole amplitudes $A_{vc}(k)$ of each one, indexed by the flattened pair index whose (ik,iv,ic) meaning is in the BSE object's `pairs.labels`. Same arguments as `get_exciton_energies`.

h.get_exciton_bands()

Return the exciton band structure $E_X(Q)$: one Bethe-Salpeter solve per q-point along a path.

Optional arguments:

- `qpath=None, nq=20`: the q-path, same input as `get_bands` (a list of high-symmetry labels, a list of explicit q-vectors, or `None` for the default path with `nq` points)
- `n=None`: keep only the `n` lowest excitons at each q-point
- `V, nk, nv, nc, kernel, tda, max_memory, screening, nkW`: as in `get_bse`, passed through unchanged

Returns (`qs,es`), flat 1D arrays of equal length: `qs` the integer index of the q-point along the path (same convention as `get_bands`) and `es` the exciton energy, complex at any q-point where the mean-field reference is unstable against the excitation. Note that `nv/nc` must not split a degenerate multiplet (a spinful Hamiltonian with no spin-orbit coupling or magnetic order needs an even `nv/nc`); a warning is raised if they do.

h.get__screened__interaction()

Return the static RPA screened interaction $W(q) = \varepsilon^{-1}(q)v(q)$ built from this Hamiltonian's own bands, as a **ScreenedInteraction**.

Optional arguments:

- `V=None`: the *bare* interaction to screen, same forms as `get_bse`'s `V`. `None` reads `h.V`, but note that a fitted Hubbard `U` is already an effective screened interaction and should not be screened again
- `nk=10`: k-mesh for both the Brillouin zone sum and the q-grid `W` is tabulated on
- `screening="rpa"`: `"rpa"` (all transitions polarize) or `"crpa"` (those inside `exclude` do not)
- `exclude=None`: (`vbands`, `cbands`) to leave out of the polarization, required by `"crpa"`
- `channel="charge"`: `"charge"` (standard GW, on site indices, spin-rotation invariant) or `"orbital"` (the full spin-orbital matrix, which breaks $SU(2)$)

The returned object exposes `qs`, `Wq`, `chi0`, `bare`, `epsmin` (the smallest dielectric eigenvalue found over the mesh), `.at(q)` for the value at a mesh q-point and `.get_dict()` for the inverse Fourier transform back to a real-space interaction. Raises if an eigenvalue of $\varepsilon(q)$ reaches zero, which is a charge or spin instability of the mean field at that wavevector.

h.get__polarizability()

Return (`qs`, `chi0`), the static polarizability of this Hamiltonian on its k-mesh, in the spin-orbital basis; `chi0` has shape (`nq`, `norb`, `norb`). Takes `nk`, `exclude` and the precomputed-eigenstate arguments of `get_screened_interaction`.

h.get__fermi__surface()

Compute the spectral weight on a 2D k-mesh at a single energy.

Optional arguments:

- `e=0.0`: energy of the cut
- `nk=50`: k-points per direction
- `delta`: broadening
- `operator=None`: project/weight by an operator (e.g. `"sz"`, `"valley"`, `"unfold"`)

Returns `kx`, `ky` and the Fermi-surface weight

h.get_multi_fermi_surface()

Compute the Fermi surface at many energies, writing one file per energy to a `MULTIFERMISURFACE/` folder.

Optional arguments:

- `energies=[0.0]`: energies to compute
- `nk, delta, operator`: as in `get_fermi_surface`

h.get_surface_kdos()

Compute the surface and bulk spectral function of a semi-infinite system, from the surface Green's function (renormalization/decimation technique).

Optional arguments:

- `kpath`: k-point path (auto-generated if not given)
- `energies, delta`: frequency range, broadening

Returns `k`, energy, surface spectral weight and bulk spectral weight; also writes `KDOS.OUT`

h.get_qpi()

Compute the quasiparticle-interference map (2D systems only). Writes output to disk (default `MULTIQPI/` folder plus `DOS.OUT`) rather than returning arrays.

Optional arguments:

- `energies, nk, delta`: as above
- `mode="response"`: `"pm"` ("poor man's", autoconvolves the actual k-resolved spectral weight – the physical QPI of a real scatterer) or `"response"` (cheaper Lindhard-like joint-DOS convolution of the clean bands)
- `nunfold=1`: unfold the QPI of a defect embedded in an `nunfoldxnunfold` supercell back onto the primitive Brillouin zone

h.get_qpi_impurity()

Compute quasiparticle interference by placing real-space impurities in a supercell, computing the real-space LDOS with ARPACK partial diagonalization, and Fourier transforming it directly (2D systems only). Returns `(r, ldos_r, q, qpi_q)`.

Optional arguments:

- `nsuper=10`: supercell size (scalar or `(n1,n2)`)

- `impurities=[]`: list of dicts, each `{"position":"index": ..., "onsite": v}` or `{"position":"index": ..., "vacancy": True}`
- `energies=0.0`, `delta`, `nk`: as above
- `num_waves=20`: starting number of ARPACK eigenstates computed nearest the requested energies – grown automatically as needed until the diagonalization both reaches `margin` (default 5.0) times `delta` past every requested energy and never stops in the middle of a degenerate manifold (summing over a partial degenerate manifold isn't basis-independent, which otherwise makes the result depend on ARPACK's starting vector – common on symmetric lattices like honeycomb, which have large exact degeneracies at high-symmetry k-points). Picking it too small just costs extra ARPACK calls to grow from, not correctness
- `write=True`, `output_folder="QPI_IMPURITY"`: also write the MULTIQPI-style disk output

`h.get_spin_splitting_density()`

Compute the energy-resolved spin splitting of a collinear magnet as a smooth weighted density: every band pair contributes its squared splitting, broadened, at the mean energy of the pair. Returns `(energies, values)`.

Optional arguments:

- `nk=20`: number of k-points per direction
- `energies`: energies at which the density is evaluated (default 400 points spanning -3 to 3)
- `delta=1e-2`: broadening
- `tol=1e-7`: largest spin off-diagonal element of the Bloch Hamiltonian tolerated, the same guard `get_spin_splitting_vs_energy` applies – above it this raises rather than answering

`h.get_average_spin_splitting()`

Return a single number: the spin splitting of a typical band, averaged over bands as well as over the Brillouin zone. Being an average rather than a sum it is intensive, so the same crystal described in a larger cell gives the same answer.

Optional arguments:

- `nk=20`: number of k-points per direction
- `tol=1e-7`: collinearity tolerance, as above

Use `h.get_spin_splitting_vs_energy()` instead when what is wanted is the largest splitting anywhere in the zone rather than the typical one.

`h.get_spin_splitting_vs_energy()`

Compute the energy-resolved **maximum** spin splitting over the Brillouin zone, so that the largest value returned bounds the spin splitting anywhere in the

zone. Bands are paired by index within each spin channel and binned at the mean energy of the pair. Returns (**energies**,**values**), the same convention as `get_spin_splitting_density`, so the two are directly comparable.

Optional arguments:

- `nk=100`: linear mesh density (nk^d points in d dimensions)
- `energies=None`: explicit bin centers; otherwise `nbins` points spanning `emin` to `emax`
- `nbins=400`, `emin=None`, `emax=None`: used when `energies` is not given; the default range is that of the mean energies actually found, so no state falls outside the window
- `tol=1e-7`: largest spin off-diagonal element of the Bloch Hamiltonian tolerated. Above it this raises rather than answering, since the spin-resolved splitting is not meaningful with spin-orbit coupling or non-collinear order

Empty bins are 0.0 (not NaN), and states outside an explicitly requested window are dropped rather than clamped onto the end bins. Diagonalization is dense throughout, since a sparse solver would return only the eigenvalues nearest $E=0$ and could miss the peak.

`h.get__chern()`

Return Chern number of the Hamiltonian.

Optional arguments: - `nk`: number of kpoints, 10 for the default `integration="grid"` and 20 for `integration="qtci"` - `integration="grid"`: how the Brillouin-zone integral is evaluated. “grid” (default) sums the Berry curvature over a uniform `nk` x `nk` mesh; “qtci” integrates it by quantics tensor cross interpolation plus Gauss-Kronrod quadrature, sampling adaptively instead of uniformly – useful when the curvature is sharply peaked. See “Tensor-cross-interpolation (qtci) integration” - `operator=None`: a name, a matrix or an `Operator`, as for `h.get_berry_curvature()`. The operator-projected invariants (`topology.spin_chern`, `topology.operator_berry` and friends) work on sparse Hamiltonians too, which is what a moire or supercell model is

`h.get__berry_curvature()`

Return the Berry curvature of the occupied bands as a map over the Brillouin zone, (`kx`,`ky`,`berry`) – three flat arrays, so it goes straight into a `plt.scatter(kx,ky,c=berry)` or, after reshaping to (`nk`,`nk`), into a `contourf`. This is the same curvature that `h.get__chern()` integrates.

Optional arguments:

- `nk=100`: linear k-point density of the map (the map has `nk*nk` points)
- `reciprocal=True`: return `kx`,`ky` in Cartesian reciprocal coordinates, which is what you want to plot a hexagonal Brillouin zone undistorted. Pass `reciprocal=False` for fractional coordinates instead, in which the

curvature integrates to the Chern number directly: with `nsuper=1` the map covers $[-1,1)$ along both fractional directions, i.e. four Brillouin zones, so `np.sum(berry)*(2/nk)**2/(2*np.pi)` comes out at four times `h.get_chern()`

- `mode="Wilson"`: how the curvature is evaluated. "Wilson" uses the Fukui-Hatsugai-Suzuki plaquette construction; "Green" uses the Green's-function Kubo formula and is selected automatically when `operator` is given
- `operator=None`: restrict the curvature to a subspace, e.g. "valley" for a valley-resolved curvature (see "Berry curvature operator"). A name, a matrix or an `Operator` are all accepted, the same spellings `h.get_bands(operator=...)` takes; the same holds for `h.get_chern()`, `topology.chern_density` and `topology.chern_qtci`
- `nsuper=1`: extend the map over this many Brillouin zones
- `kpath`: compute along a k-path instead of over a 2D grid
- `delta=0.001`: broadening used by the Green's-function mode

`h.get_quantum_geometric_tensor()`

Return the (multiband/multiorbital) quantum geometric tensor at a single k-point, see "Quantum geometric tensor (multiorbital/multiband)".

Optional arguments: - `k=[0.,0.,0.]`: k-point - `occ_idx=None`: band indices of the chosen subspace (default: the bands with $E < 0$, the same Fermi-level convention `h.get_chern()` uses, so this tracks `h.shift_fermi(...)`) - `non_abelian=False`: if True, return the full band-pair-resolved tensor instead of its trace over the subspace - `degeneracy_tol=1e-8`: energy tolerance used to detect a degeneracy between the chosen subspace and its complement (raises `ValueError`)

`h.get_quantum_metric()`

Same arguments as `h.get_quantum_geometric_tensor()`, but returns only the quantum metric (symmetric part of the tensor).

`h.get_dvector_non_unitarity()`

Non-unitarity vector $\vec{q} = i(\vec{d} \times \vec{d}^*)$ of the spin-triplet d-vector of a BdG (Nambu) Hamiltonian, resolved per site (see "Spin-triplet d-vector and non-unitary superconductivity"). It is real, vanishes for a unitary state, and otherwise is the spin moment of the Cooper pairs – parallel to the magnetization in a ferromagnetic spin-triplet superconductor, and along $+z$ for a pure $\Delta_{\uparrow\uparrow}$ pairing.

Optional arguments

- `nk = 10`: k-points per periodic direction

Returns an array of shape (number of sites, 3).

h.write_non_unitarity()

Write $\vec{q}$ as a real-space map in `NON_UNITARITY_MAP.OUT`, with columns (x, y, z, q_x, q_y, q_z) .

Optional arguments

- `nrep = 2`: number of replicas written per direction
- `nk = 10`: k-points per periodic direction

h.get_average_dvector()

k-averaged squared components $(|d_x|^2, |d_y|^2, |d_z|^2)$ of the spin-triplet d-vector of a BdG (Nambu) Hamiltonian.

Optional arguments

- `nk = 10`: k-points per periodic direction
- `spatial_sum = True`: average over sites, returning a single 3-component vector
- `non_unitarity = False`: average the squared components of $\vec{q}$ instead of those of $\vec{d}$. This is a magnitude only and carries no sign, so use it to ask whether the state is non-unitary rather than in which direction; use `h.get_dvector_non_unitarity()` for the signed vector.

h.get_superfluid_weight()

Superfluid weight tensor D_s^{ab} of a BdG (Nambu) Hamiltonian of dimensionality 1, 2 or 3 (see “Superfluid weight and BKT temperature”).

Optional arguments:

- `nk=20`: k-points per periodic direction
- `T=0.0`: temperature (see the caveat above for gapless normal states at $T=0$)
- `mode="kubo"`: `"kubo"` for the analytic multiband formula, `"finite_difference"` for the assumption-free numerical derivative of the grand potential
- `gauge="atomic"`: `"atomic"` twists with the full bond vector (the physical Peierls substitution), `"lattice"` with the lattice vector alone (the literature cell-gauge convention)
- `decompose=False`: if `True`, return a dict with `"total"`, `"conventional"`, `"geometric"`, `"delta"` and `"gauge"` instead of a bare tensor

Returns a Cartesian (`dim,dim`) array, or a dictionary if `decompose=True`. `decompose=True` raises `ValueError` when the decomposition’s assumptions do not hold.

h.get__bkt__temperature()

Berezinskii-Kosterlitz-Thouless temperature of a 2d BdG Hamiltonian, from the self-consistent Nelson-Kosterlitz criterion $T_{\text{BKT}} = (\pi/8)D_s(T_{\text{BKT}})$ at frozen $|\Delta|$. Arguments `nk=20`, `tmax=None`, `tol=1e-6`, `maxite=60`, plus `gauge`. Returns a float.

h.get__nonlinear__drude__conductivity()

Compute the l -th order nonlinear Drude conductivity $\sigma^{\{x^{l1} y^{l2} ; b\}}$ of a collinear magnet, the quantity whose lowest nonvanishing order measures the X-wave index of an altermagnet (`p:0`, `d:1`, `f:2`, `g:3`, `i:5`). Requires spin to be a good quantum number.

Optional arguments

- `field = "x"`: one character per power of the electric field, e.g. "yyyyy" for the fifth-order response to E_y
- `current = "x"`: Cartesian direction of the measured current
- `channel = "spin"`: "spin", "charge", "up" or "dn"
- `nk = 100`: k-points per periodic direction
- `T = 0.01`: temperature of the Fermi occupations
- `mu = 0.0`: chemical potential
- `tau = 1.0`: relaxation time; the l -th order response scales as τ^l
- `omega = 0.0`: frequency, entering as $1/(i\omega + 1/\tau)^l$
- `degeneracy_tol = 1e-8`: multiorbital cells only, the relative band spacing below which bands are grouped into a degenerate multiplet

Returns a complex scalar

h.get__nonlinear__drude__components()

Every component of the l -th order nonlinear Drude conductivity, as a dict $\{x^{l1} y^{l2}; b\}: \text{value}\}$. Shares one k-mesh across the components, so it is much cheaper than the equivalent individual calls.

h.get__nonlinear__drude__orders()

X-wave selection-rule sweep: for each order $l = 0..l_{\text{max}}$, the largest $|\sigma^{\{x^{l1} y^{l2} ; b\}}|$ over that order's components. The lowest l with a nonzero entry is the wave index readout.

Optional arguments

- `lmax = 6`: highest order to sweep
- plus every optional argument of `h.get__nonlinear__drude__conductivity()`

Returns a list of floats

h.get__optical__conductivity()

Frequency-dependent conductivity tensor $\sigma_{ab}(\omega)$ in the Kubo-Greenwood formalism (see “Optical conductivity”).

Optional arguments:

- `energies=None`: frequencies at which to evaluate σ
- `nk=20`: k-points per periodic direction
- `T=None`: temperature (defaults to `delta`)
- `delta=0.1`: Lorentzian broadening η
- `intraband=True, interband=True`: switch the two channels independently
- `component=None`: e.g. “xy” to return that component alone
- `degeneracy_tol=1e-6`: relative tolerance for treating a pair as degenerate

Returns `(energies,sigma)` with `sigma` of shape `(nw,3,3)` and complex, or `(nw,)` if `component` is given. Units of $e^2/\hbar$ (so e^2/h is $1/(2\pi)$ of it). Raises `NotImplementedError` for 3d and Nambu Hamiltonians.

h.get__drude__weight()

Drude (intraband) weight tensor. Arguments `nk=20`, `T=0.05`, `degeneracy_tol=1e-6`. Returns a real `(3,3)` array.

h.get__sum__rule__weight()

Diamagnetic weight tensor W entering the optical f-sum rule $\int \text{Re } \sigma_{aa}(\omega) d\omega = \pi W_{aa}$. Arguments `nk=20`, `T=0.05`. Returns a real symmetric `(3,3)` array.

h.get__entanglement__entropy()

Entanglement entropy of a real-space region, from the eigenvalues of the region-restricted one-particle correlation matrix (see “Entanglement entropy and entanglement spectrum”).

Optional arguments:

- `region=None`: the region, as a list of site indices, a boolean mask, a callable on positions (the `sculpt` convention) or a float fraction of the cells; `None` takes half the system
- `nsuper=10`: unit cells stacked into the ring that is cut (periodic Hamiltonians)
- `direction=None`: lattice direction normal to the cut, defaults to the last periodic one
- `kpar=None`: momentum parallel to the cut; `None` on a 2d Hamiltonian averages over an `nk` mesh
- `nk=20`: k-points used for that average
- `fermi=0.0`: occupied states are those with `E<fermi` (must stay 0 for BdG)

Returns a float. A level exactly at the Fermi energy raises rather than returning the entropy of an arbitrary determinant.

h.get_entanglement_spectrum()

Single-particle entanglement Hamiltonian eigenvalues $\xi_n = \ln[(1 - \zeta_n)/\zeta_n]$ of a real-space region. Same arguments as `h.get_entanglement_entropy()` (with `nk=41` by default).

Returns a sorted array of ξ_n for a 0d/1d Hamiltonian or a single `kpar`; for a 2d Hamiltonian with `kpar=None` it returns `(ks,xis)` with `xis` of shape `(nk,nA)`, the Li-Haldane entanglement spectrum across the BZ.

h.get_wannier_hamiltonian()

Wannierize a range of bands and return the resulting real-space Hamiltonian.

Arguments:

- `bands = [a,b]`: first and last band to Wannierize (0-indexed, both ends inclusive)

Optional arguments:

- `nk=12`: k-points per periodic direction for the wannierization mesh
- `cutoff=1e-6`: real-space hoppings smaller than this are dropped (the intra-cell block is always kept). Set it to 0.0 to keep the reproduction exact to machine precision
- `symmetries=None`: "auto" to auto-detect and enforce the point group, or an explicit list of `symmetrytk.pointgroup.SymmetryOperation`
- `num_wann=None`: how many Wannier functions to extract. `None` means the whole selected range, Wannierized as a fixed subspace and reproduced exactly. A smaller value switches on Souza-Marzari-Vanderbilt disentanglement, after which only the frozen window is reproduced (see "Disentanglement"). Outside `1..len(bands)` raises `ValueError`
- `dis_win_min, dis_win_max=None`: outer energy window, i.e. which bands are offered to the extraction at each k-point. Every eigenvalue of the result lies inside it
- `dis_froz_min, dis_froz_max=None`: frozen inner window, whose states are reproduced exactly at every mesh k-point. `dis_froz_max` alone is enough; `dis_froz_min` alone raises `ValueError`
- `dis_num_iter=200`: maximum iterations of the Ω_I minimization; not converging within them is a warning, not an error

Any `dis_*` argument given without a smaller `num_wann` raises `ValueError` rather than being ignored. Disentanglement together with a Nambu/BdG Hamiltonian, with `symmetries=`, or with `auto_split_clusters=True` raises `NotImplementedError`.

Returns a new, smaller Hamiltonian; `.wannier_centres`, `.wannier_spreads` and `.wannier_spread_total` hold the Wannier-function geometry, and `.wannier_num_wann/.wannier_disentanglement_window` record what was extracted and through which window (the latter `None` when not disentangling)

`h.get_szsz_mean_field_hamiltonian()`

Self-consistent Hartree-Fock mean field for a $J_z \sum S_i^z S_j^z$ spin-spin exchange interaction (see “Spin-spin exchange interactions”). $J > 0$ is antiferromagnetic, $J < 0$ ferromagnetic.

Optional arguments:

- `J1, J2, J3 = 0.:` first/second/third-neighbor J_z couplings
- `Jr=None:` general distance-dependent coupling function, as `Vr` for `get_mean_field_hamiltonian`
- `filling, mf, nk, maxerror, mix, constrains:` as in `get_mean_field_hamiltonian`
- `return_total_energy=False:` also return the total energy

Also works on BdG (Nambu) Hamiltonians, decoupling both the normal and anomalous (pairing) channels (same generic dispatch `get_mean_field_hamiltonian` already uses).

Returns the converged Hamiltonian (or `None` if the SCF did not converge)

`h.get_sxsx_mean_field_hamiltonian()` / `h.get_sysy_mean_field_hamiltonian()`

Same as `get_szsz_mean_field_hamiltonian()`, for a $S_i^x S_j^x$ / $S_i^y S_j^y$ interaction instead, implemented by rotating the problem so that x (or y) becomes the computational z axis, solving there, and rotating the converged Hamiltonian back. Also works on BdG Hamiltonians: `rotate_spin.global_spin_rotation` already rotates the Nambu-doubled Hamiltonian correctly with no changes needed.

`h.get_exchange_mean_field_hamiltonian()`

Self-consistent anisotropic exchange mean field, combining $J_x S_i^x S_j^x + J_y S_i^y S_j^y + J_z S_i^z S_j^z$ in a single SCF loop.

Optional arguments:

- `Jx1, Jx2, Jx3, Jy1, Jy2, Jy3, Jz1, Jz2, Jz3 = 0.:` first/second/third-neighbor couplings for each axis
- `Jxr, Jyr, Jzr=None:` general distance-dependent couplings, one per axis
- `mf, filling, nk, maxerror, mix, constrains:` as above (only `integration="ed"` and the plain-mixing solver are supported)

Also works on BdG Hamiltonians, with the same full normal-plus-anomalous decoupling as `get_combined_mean_field_hamiltonian`’s density-density channels – exchange can itself induce superconducting pairing (e.g. an antiferromag-

netic isotropic J alone, seeded with a random guess, can spontaneously decouple into a purely superconducting state).

Returns the converged Hamiltonian (or `None` if the SCF did not converge)

`h.get_combined_mean_field_hamiltonian()`

Self-consistent mean field combining density-density interactions (onsite U , $V_1/V_2/V_3/V_r$ neighbor-shell) with spin-spin exchange in a single SCF loop.

Optional arguments:

- U , V_1 , V_2 , V_3 , V_r : as in `get_mean_field_hamiltonian`
- J_1 , J_2 , $J_3 = 0.$: isotropic Heisenberg exchange for the first/second/third-neighbor shells (same shell convention as $V_1/V_2/V_3$)
- $J_r=$ `None`: general distance-dependent isotropic exchange function, as V_r
- J_{1x} , J_{1y} , $J_{1z} = 0.$: optional anisotropic correction added to J_1 on the first-neighbor shell only (e.g. the effective first-neighbor J_z coupling is J_1+J_{1z}); second/third neighbors stay purely isotropic

On a BdG Hamiltonian every channel keeps the full normal+anomalous (pairing) treatment: $U/V_1/V_2/V_3/V_r$ as in `get_mean_field_hamiltonian`, and the exchange (J) channels identically, so exchange alone can induce superconducting pairing rather than only magnetism (see “Spin-spin exchange interactions” above, and the `total_energy` caveat recorded there). - `mf`, filling, `nk`, `max-error`, `mix`, `constraints`: as above (only the plain-mixing solver is supported) - `integration=“ed”`: computes the density matrix each SCF iteration by exact diagonalization. `“kpm”` instead gets it through a per-k Chebyshev-moment (Kernel Polynomial Method) expansion, never diagonalizing the Bloch Hamiltonian $H(k)$ – for large/sparse systems (e.g. a big 0D flake, where the unit cell itself is too large to diagonalize or even hold as a dense matrix) where per-iteration exact diagonalization is the bottleneck; only supported for a normal-state (non-BdG) Hamiltonian. With `“kpm”`: `scale=None` sets the KPM energy rescaling (estimated automatically if not given), `npol` the number of Chebyshev moments, `ne` the number of energies sampled in the occupied window, `cores` the number of parallel workers across k-points; all four are unused for `“ed”`. Also reachable through `h.get_mean_field_hamiltonian(integration=“kpm”,...)`, which dispatches to this function automatically for a spinful `h`.

Performance caveat (measured, not just theoretical): at small/moderate system sizes (order 100-500 sites) `“kpm”` is currently much *slower* per SCF iteration than `“ed”` – roughly 50-60x slower, measured on a 98-site honeycomb Hubbard system – because dense exact diagonalization via LAPACK is extremely fast at that scale regardless of algorithmic complexity, while this KPM implementation still pays real per-orbital and per-matrix-element overhead (see `kpm.tk.densitymatrix_kpm._dm_kpm_from_needed’s` and `get_fermi4filling_kpm’s` docstrings for exactly where). It should in principle win for a large/sparse enough system, but that crossover was not reached

in the sizes tested. Only use "kpm" after confirming it is actually faster for your system. - `use_jax=True, solver="newton"`: solve the same SCF fixed point $x = f(x)$ (x the mean-field parameters, f one SCF iteration) a different way – instead of the default plain-mixing loop above, build a JAX-differentiable version of f and solve it with a JAX-derivative-based root-finder. `solver="newton"` (the default once `use_jax=True`) uses `jax.jacfwd` for the exact Jacobian; `"newton_krylov"` is the matrix-free variant (`jax.jvp` Jacobian-vector products + GMRES), which scales to larger systems than the dense-Jacobian `"newton"` (`gmres_tol=1e-6, gmres_restart=20` tune its linear solve); `"fsolve"` wraps `scipy.optimize.fsolve`/MINPACK with the same `jax.jacfwd` Jacobian as `fprime`; `"linear_mixing"` is plain linear mixing routed through the same machinery, for comparison; `"error_gradient"` instead minimizes the squared SCF residual $\|f(x) - x\|^2$ as a proper nonlinear least-squares problem, via matrix-free Levenberg-Marquardt (`jax.jvp/jax.vjp` Jacobian-vector and Jacobian-transpose-vector products of the residual + `scipy`'s `lsqr` for each damped LM subproblem) – not the physical free energy directly (see `scftk.vjinteraction_jax`'s module docstring for why that alternative was tried and abandoned: the physical SCF solution turned out to be a saddle point, not a minimum, of the free-energy functional). `"error_gradient"` scales per-iteration like `"newton_krylov"/"linear_mixing"` (no dense Jacobian), and matches `"newton"/"newton_krylov"` well from small systems up through at least ~60 orbitals (see `scftk.vjinteraction_jax`'s module docstring for measured numbers), but as a local method it can in principle still stall short of `maxerror` on a sufficiently hard landscape – always check `.converged`. `"broyden_mixing"` is a black-box mixing scheme rather than a root-finder/gradient method (regularized, limited-memory multiseccant Broyden mixing, Marks & Luke arXiv:0801.3098 – see `scftk.broydenmixing`'s module docstring); it only ever evaluates f itself, so it plugs into the same solver dispatch with no Jacobian/gradient machinery of its own, and is also reachable from the plain (non-jax) engine as `solver="broyden_mixing"` (alongside the existing `"broyden1"/"krylov"/"anderson"/"linear"` `scipy.optimize` wrappers in `scftk.densitydensity.generic_densitydensity`). Restricted to a normal-state (non-BdG) Hamiltonian, dense exact diagonalization only (no `integration="kpm"`), and no `constrains`; needs the optional `jax` extra (`pip install pyqula[jax]`). See `scftk.vjinteraction_jax`'s module docstring for how this reuses (unmodified) the solver infrastructure `scftk.densitydensity_jax` already built for the simpler `Vinteraction` (V/U-only) case:

```
from pyqula import geometry
h = geometry.chain().get_hamiltonian()
hmf = h.get_combined_mean_field_hamiltonian(U=4.0, J1=-0.5, filling=0.5,
      use_jax=True, solver="newton") # JAX-derivative-based SCF solver
```

Returns the converged Hamiltonian (or None if the SCF did not converge)

`filling` also accepts a per-SITE array (length `len(h.geometry.r)`), same

0-to-1-fraction-of-2-orbital-capacity convention as the scalar case) instead of only a single lattice-averaged value, enforcing $\langle n_i \rangle = \text{filling}[i]$ at every site independently via a per-site Lagrange multiplier (warm-started and co-converged with the mean field across the same SCF loop, one diagonalization per outer iteration – not solved to tight tolerance every iteration, since a per-site potential generally changes the eigenvectors too, unlike a scalar Fermi shift). `scf.local_occupation` and `scf.lam/h.fermi` (now the converged per-site array) expose the diagnostics; `scf.converged` implies the per-site constraint converged to within `maxerror`, not only the mean field. Only supported for a normal-state (non-BdG), `integration="ed"` Hamiltonian with `mu=None` (the default). This is the mechanism `SpinonHamiltonian` (see “Abrikosov-pseudofermion (spinon) mean field for Heisenberg models”) builds on to enforce exactly one auxiliary fermion per site.

SpinonHamiltonian(g)

Abrikosov-pseudofermion (RVB) mean-field Hamiltonian for a spin- $\frac{1}{2}$ Heisenberg model on geometry `g` – see “Abrikosov-pseudofermion (spinon) mean field for Heisenberg models” above. `from pyqula.spinon import SpinonHamiltonian`; built with zero bare hopping, couplings supplied through `get_mean_field_hamiltonian`’s usual `J1/J2/J3/Jr/J1x/ J1y/J1z` kwargs.

- `h.get_mean_field_hamiltonian(J1=...,nk=...,...)`: same SCF kwargs as `get_combined_mean_field_hamiltonian` above, except `filling` cannot be passed (always exactly one fermion/site, enforced site-by-site). Returns the converged Hamiltonian (or `None` if the SCF did not converge), with two extra diagnostic attributes:
 - `h2.local_occupation`: converged $\langle n_i \rangle$ per site (electron-count convention, 0 to 2 – target is exactly 1.0)
 - `h2.constraint_lambda`: converged per-site Lagrange multiplier (local chemical potential)

KondoLatticeHamiltonian(hc)

Abrikosov-pseudofermion (Read-Newns) mean-field Hamiltonian for the Kondo lattice / periodic Anderson model built from a conduction-electron Hamiltonian `hc` – see “Abrikosov-pseudofermion (Read-Newns) mean field for the Kondo lattice” above. `from pyqula.kondolattice import KondoLatticeHamiltonian`; fuses a second, zero-bare-hopping f-sublattice onto `hc`’s geometry, with the Kondo coupling supplied through `get_mean_field_hamiltonian`’s `J` kwarg.

- `h.get_mean_field_hamiltonian(J=...,filling=...,mf=(V,lam),nk=...,...)`: self-consistently solves for the hybridization and the local constraint’s Lagrange multiplier. Returns the converged Hamiltonian (or `None` if the SCF did not converge), with three extra diagnostic attributes:
 - `h2.local_occupation`: converged $\langle n_f \rangle$ per localized site (target is exactly 1.0)

- `h2.hybridization`: converged V_j per localized site
- `h2.constraint_lambda`: converged per-site Lagrange multiplier

GrapheneGeometry(g)

Geometry subclass wrapping a graphene multilayer geometry `g` (bilayer, twisted bilayer, twisted trilayer, ...), adding a `.relax()` method – see “Twisted bilayer graphene structural relaxation” above. `from pyqula.graphenetsk.geometry import GrapheneGeometry`; `g` must have `has_sublattice=True` (raises `ValueError` otherwise).

- `g.relax(nrep=1, maxiter=500, verbose=False, layer_pairs=None, gsfe_coeffs=..., elastic_coeff=...)` minimizes the GSFE (interlayer) + elastic (intralayer) energy over an in-plane displacement field and returns a new, relaxed **GrapheneGeometry**. `layer_pairs` selects which (lower, upper) pairs of layers (ordered by `z`) get an interlayer GSFE term, defaulting to all adjacent pairs; `gsfe_coeffs/elastic_coeffs` override the graphene Table 1 constants of `pyqula.graphenetsk.gsfe.GRAPHENE_GSFE` / `pyqula.graphenetsk.elastic.GRAPHENE_ELASTIC`.

GrapheneHamiltonian(geometry)

Hamiltonian built from a graphene multilayer `geometry` (typically a **GrapheneGeometry**, relaxed or not) – see “Twisted bilayer graphene structural relaxation” above. `from pyqula.graphenetsk.hamiltonian import GrapheneHamiltonian`; defaults to the distance-decaying hoppings of `specialhopping.twisted_matrix` (`ti=0.12, lambi=8.0, lamb=12.0, dl=3.0`, matching `specialhamiltonian.twisted_bilayer_graphene`’s defaults) rather than the generic first-neighbor default, so relaxed (in-plane displaced) positions feed into the electronic structure automatically. Pass `mgenerator=...` to use a different hopping generator instead.

h.get_central_heterostructure()

Build a two-terminal **Heterostructure** using `h` (a finite, 0d Hamiltonian) as the central scattering region, contacted by two semi-infinite 1D chain leads attached at sites `i/j` (see “Transport through an arbitrary finite region” above).

Optional arguments:

- `i=0, j=None`: 0-indexed sites `h` is contacted at; `j` defaults to the last site
- `left=None, right=None`: lead Hamiltonians; default to a plain spinless `geometry.chain()`. Give one of them (or `h` itself) nonzero pairing (`add_swave`) for a normal-superconductor junction – at most one of `{h, left, right}` may carry pairing

Returns a **Heterostructure**, so `didv`, `get_dos`, `get_kappa`, etc. all apply unmodified; `landauer` is the exception, raising `NotImplementedError` once any of

the three carries pairing (use `didv` there). Only 0d central regions are supported so far (`h.dimensionality>0` raises `NotImplementedError`).

Heterostructure functions and methods

HT.get_dc_current()

Compute the time-averaged (DC) current through a two-terminal junction at a given bias, using the Floquet-Keldysh formalism (see “Multiple Andreev reflection and AC-Josephson current”). Works for any combination of normal/superconducting leads built with `heterostructures.build(h1,h2)`, with or without an explicit `central=` Hamiltonian (the latter is solved by a general dense Floquet inversion, and assumes the bias drops across the junction’s rightmost bond).

Arguments:

- voltage: bias voltage

Optional arguments:

- `nmax=6`, `nmax_max=40`: initial/maximum number of Floquet sidebands (increased adaptively until convergence; a warning is issued if `nmax_max` is reached before `tol` is satisfied)
- `tol=1e-3`: relative convergence tolerance used to decide when to stop increasing the sideband count
- `temperature=0.`: lead temperature
- `delta=None`: broadening (defaults to `HT.delta`); should be much smaller than the smallest relevant gap for small-gap superconductors

Returns the DC current

HT.get_iv_curve()

Convenience wrapper: `get_dc_current` evaluated over an array of voltages.

Arguments:

- voltages: array of bias voltages

Returns an array of DC currents

HT.didv_curve() / lp.didv_curve()

Convenience wrapper: `didv` evaluated over an array of energies, in parallel – the array-native equivalent of `[ht.didv(energy=e) for e in es]`. Also reachable as `didv(energies=...)` (mutually exclusive with `didv`’s scalar `energy=...`, same convention as `get_kappa`), which dispatches straight here. If `use_aaa=True` and the sweep resolves to the Floquet-Keldysh method, builds and shares one AAA self-energy interpolant across the whole sweep instead

of each energy independently building (and discarding) its own – the `didv` counterpart to `get_iv_curve`’s own sharing for `get_dc_current`.

Arguments:

- `energies`: array of bias energies
- any keyword argument accepted by `didv` (`method`, `delta`, `use_aaa`, `nmax_max`, `temp`, ...)

Returns an array of dI/dV values

T means different things on the two classes. On a `Heterostructure`, `T` is the temperature. On a `LocalProbe` it is the probe *transparency* – the same knob as `LocalProbe(..., T=...)`, `set_coupling` and `get_kappa(T=...)` – and the temperature there is `temp` (or its alias `temperature`). Both `lp.didv(T=...)` and `lp.didv_curve(..., T=...)` honour it for that call alone, leaving the probe itself untouched.

`delta=` passed to any of `didv`, `didv_curve` or `get_smatrix` likewise applies to that call, self-energies included, rather than being overridden by the junction’s own attribute. For a `LocalProbe` it sets both the probe broadening and the sample’s bulk broadening, exactly as the constructor argument does. `HT.with_delta(delta)`, `lp.with_delta(delta)` and `lp.with_coupling(T)` return a shallow copy with that one knob rebound, for when the same setting is wanted across several calls.

HT.get_kappa()

Compute the superconducting/normal conductance power-law-ratio “kappa” diagnostic (also available as `LocalProbe.get_kappa()`, see “Multiple Andreev reflection and AC-Josephson current”).

Optional arguments:

- `energy=0.0`: energy at which to evaluate kappa (returns a scalar)
- `energies`: array of energies to evaluate at once instead (returns an array); mutually exclusive with `energy`
- `temp=0.`: temperature; `0.` (the default) uses the original zero-temperature `get_kappa_ratio` path, a nonzero value thermally averages each conductance entering the power-law fit (`didv(temp=...)`) and, for whichever branch is actually superconducting, shares one lead self-energy interpolant across the whole coupling/energy/thermal sweep instead of rebuilding it per call
- `T=1e-2`: reference coupling the power-law exponent is extracted around

Returns kappa (a scalar, or an array matching `energies`)

At `temp=0.` (the default), kappa is $d(\log G)/d(\log T)$: how steeply the conductance scales with the probe-sample coupling. The `get_kappa_ratio` path above estimates this by sampling the conductance at two nearby couplings

(0.9T and 1.1T) and fitting a secant slope through them. For a `LocalProbe` (always 1D) whose probe lead is not itself superconducting – so `didv` uses the ordinary BdG scattering-matrix formula, not Floquet-Keldysh – this is now computed exactly instead: neither lead self-energy depends on the coupling `T` (only the coupling block of the small matrix that gets inverted does), so the self-energies are solved once and the coupling-dependent tail is differentiated exactly with `jax.grad(transporttk.kappa_jax)`, rather than approximated by a secant. This is the default automatically whenever it applies (falling back to the secant otherwise, e.g. when `jax` isn't installed, at finite `temp`, for a superconducting probe, for a spinless-Nambu system, or for a `Heterostructure`); it also cross-checks its own result against the reference `get_smatrix` formula once per call and falls back if they disagree beyond a tight tolerance (guarding against the rare case where `unitarize.check_and_fix`'s unitarity correction on the reference path would have mattered). Benchmarked on `examples/transport/localprobe_kappa_1D`, it matches the secant to within its own finite-difference bias ($\sim 10^{-3}$ out of $O(1)$ values) while running faster (~ 1.2 - $1.8\times$ depending on system size, after accounting for that cross-check). Both this path and the secant fallback also now solve each lead self-energy once per coupling sweep instead of once per coupling point (self-energies never depend on `T`, only the tiny coupling block that gets inverted does), which is what dominates the speedup for systems where the self-energy solve itself (e.g. a 2D sample's bulk Green's function), not the small coupling-dependent tail, is the expensive part.

SpinModel functions and methods

`sm.add_heisenberg()`

Add shell-based exchange couplings to the model, on top of any interactions already added (repeated calls accumulate).

Optional arguments:

- `Jij=None`: list of shell couplings, e.g. `[J1,J2,J3]` for first/second/third neighbor exchange; passed through to `Geometry.get_hamiltonian(tij=Jij)`, so anything that constructor accepts for `tij` works here too (`None` is that constructor's own default, first-neighbor hopping)
- `Jm=[1.,1.,1.]`: diagonal (J_x, J_y, J_z) weights, applied on top of `Jij`; use unequal values for XXZ/Ising-like anisotropy

Mutates `sm` in place, no return value

`sm.add_field()`

Add a uniform Zeeman field to every site's `sm.b`. Repeated calls accumulate (not overwrite).

Arguments:

- $\mathbf{v}$: 3-vector field, e.g. $[0., 0., 1.]$

`sm.add_tensor()` / `sm.add_tensor_2d()`

Add a general pairwise tensor coupling J_{ij} from a function `fun(r1,r2) -> 3x3 matrix` (see `generating_functions` below), rather than the fixed-form couplings `add_heisenberg` builds. `add_tensor` only considers pairs within the home cell; `add_tensor_2d` additionally sums periodic images $\mathbf{ia} \cdot \mathbf{a1} + \mathbf{ja} \cdot \mathbf{a2}$ for $\mathbf{ia}, \mathbf{ja}$ in $[-\text{ncells}, \text{ncells}]$, and can rotate the coupling tensor of each image about z by $\text{vspiral}[0] \cdot \mathbf{ia} + \text{vspiral}[1] \cdot \mathbf{ja}$ (in units of π) – a way to embed a spin-spiral wavevector directly into the exchange tensor.

Arguments:

- `fun`: coupling function, e.g. one returned by `generating_functions`

Optional arguments (`add_tensor_2d` only):

- `ncells=1`: number of periodic images to sum on each side, along each lattice vector
- `vspiral=[0.,0.]`: per-image in-plane rotation angle coefficients, see above

Mutates `sm` in place, no return value

`classicalspin.generating_functions()`

Factory returning a `fun(r1,r2) -> 3x3 matrix` two-point coupling function for a standard exchange form, for use with `add_tensor/add_tensor_2d`.

Optional arguments:

- `name="Heisenberg"`: one of "Heisenberg" (isotropic, cutoff `fc(distance)`), "Linear" (dipolar $1/r^3$ tensor), "RKKYTI" (RKKY on a topological-insulator surface, PRB 81 233405), "ZZ" (Ising, $S_i^z S_j^z$ only), "XYZ" (diagonal, anisotropic weights $\mathbf{v}$), "DM" (Dzyaloshinskii-Moriya, with $\mathbf{v}$ the intermediate-ion/mirror direction, or a function of the bond vector)
- `J=1.0`: overall coupling strength
- `v=[0.,0.,1.]`: form-dependent vector (or vector-valued function of the bond vector), see above
- `fc=None`: distance-dependent cutoff/envelope, e.g. restricting "Heisenberg"/"ZZ" to first neighbors ($0.9 < d < 1.1$, the default)
- `fdiff=lambda x,y: x-y`: how the bond vector is computed from `(r1,r2)`; override for e.g. a minimum-image convention
- `fr=None`: extra rotation matrix `fr(r1,r2)` applied on top of the base coupling (defaults to the identity)

Returns the coupling function

sm.minimize__energy()

Multistart local minimization of the classical energy over every spin's (θ, ϕ) angles, using `scipy.optimize.minimize` with the gradient from `jax.grad` autodiff of the energy (or, if `calle` is given, gradient-free Powell). Only the ground state at Γ is found – there are no twisted boundary conditions, so an incommensurate (e.g. spiral) texture needs an explicit supercell that fits it.

Optional arguments:

- `theta0=None, phi0=None`: initial angles; each try re-randomizes them in $[0, \pi]/[0, 2\pi]$ when left as `None` (the default), which is what makes `tries` explore different basins – passing explicit arrays instead starts every try from the same point, since the optimizer itself is deterministic, so `tries>1` is only useful with the default
- `tries=10`: number of independent minimizations; the lowest-energy one is kept
- `calle=None`: optional extra function `calle(sm) -> float` added to the energy during minimization, e.g. a penalty favoring a particular texture

Updates `sm.theta`, `sm.phi` and `sm.magnetization` in place to the best try found, and returns `(theta, phi)`

sm.get__energy() / sm.energy()

Evaluate the total energy $\sum_i \vec{b}_i \cdot \vec{S}_i + \sum_{ij} \vec{S}_i \cdot J_{ij} \cdot \vec{S}_j$ of the current `sm.theta/sm.phi` snapshot. Returns a scalar.

sm.get__local__energy()

Per-site breakdown of the current snapshot's energy: each site's own field term plus half of every exchange term it takes part in (as with `LatticeGas.get_local_energy()`, each bond's energy is split evenly between its two endpoints), so the values sum exactly to `sm.get__energy()`.

Returns an array over sites

sm.get__magnetization()

Convert the current `sm.theta/sm.phi` angles to Cartesian components.

Returns `(mx, my, mz)`, arrays over sites

align.most_perpendicular_vector() / align.most_perp_basis()

(`classicalspintk.align`) Given a set of vectors (typically a `SpinModel`'s magnetization), find the direction most nearly perpendicular to all of them, and use it to build a rotated basis in which that direction is the new z axis – i.e. the vectors end up mostly in the new xy plane. Useful

for plotting a magnetization texture (e.g. a skyrmion or spiral) as an in-plane quiver plot when it isn't already aligned with a coordinate axis; see `examples/classicalspin/perpendicular/main.py`.

Returns a perpendicular unit vector, or (for `most_perp_basis`) the input vectors expressed in the rotated basis

LatticeGas functions and methods

`lg.add_interaction()`

Add a coupling shell to the model, on top of any interactions already added (repeated calls accumulate).

Optional arguments:

- `Jij`: list of shell couplings, e.g. `[J1,J2,J3]` for first/second/third neighbor $J_{ij}n_i n_j$ repulsion (or attraction, for negative values); passed through to `Geometry.get_hamiltonian(tij=Jij)`, so anything that constructor accepts for `tij` works here too

Mutates `lg` in place, no return value

`lg.set_filling()`

Reset the occupation array `lg.den` to a new random configuration with a given filling fraction, discarding the current snapshot (e.g. to re-seed a fresh anneal, see `examples/latticegas/optimize/main.py`).

Arguments:

- `filling`: fraction of sites occupied (rounded to the nearest integer site count)

`lg.get_energy()`

Evaluate the total energy $\sum_i \mu_i n_i + \sum_{ij} J_{ij} n_i n_j$ of the current occupation snapshot `lg.den`. Returns a scalar.

`lg.optimize_energy()`

Anneal `lg.den` towards a low-energy configuration with a Metropolis discrete-swap optimizer: at each step, 1-3 random occupied/empty site pairs are swapped (preserving the total filling) and accepted unconditionally if the energy doesn't increase, or with probability $e^{-\Delta E/T}$ otherwise. Energy changes are tracked incrementally per swap rather than by recomputing the full energy from scratch, so cost per step scales with each site's number of interaction neighbors, not with the total number of interaction terms in the system.

Optional arguments:

- `temp=0.1`: Metropolis temperature (higher accepts more uphill moves; anneal by calling this repeatedly with decreasing `temp`, or see `lg.anneal()` below). `temp=0` runs zero-temperature (greedy) dynamics: an uphill move is never accepted, equivalent to the $T \rightarrow 0$ limit without the floating-point division by zero that would otherwise imply
- `ntries=1e5`: number of swap attempts
- `resync_every=1000`: how often (in swap attempts) to recompute the energy from scratch, bounding floating-point drift in the incremental tracking
- `patience=None`: if set, stop early once this many attempts have passed without a new best energy being found (the returned array is truncated to what actually ran)
- `checkpoint_at=None`: an int or iterable of ints; captures a copy of `lg.den` after that many attempts (1-indexed) into `lg.checkpoints` (a dict `step` $\rightarrow$ `den` snapshot), independent of the final configuration – e.g. to inspect or animate how the configuration evolves partway through a run

Overwrites `lg.den` with the final configuration and returns the array of energies recorded at each attempt (whether or not it was accepted)

`lg.anneal()`

Simulated annealing over a decreasing temperature schedule: calls `lg.optimize_energy()` once per temperature in `temps`, keeping the best configuration seen across the whole schedule (a single high-temperature step's Metropolis walk can wander back up in energy by its end, so the last step's final state is not necessarily the best one found).

Optional arguments:

- `temps=None`: sequence of temperatures, high to low; defaults to a 10-step geometric schedule from 2.0 down to 0.05 (any entry, including 0, is passed straight through to `lg.optimize_energy()`)
- `ntries=1e4`: number of swap attempts per temperature
- `checkpoint_at=None`: an int or iterable of ints; like `lg.optimize_energy()`'s `checkpoint_at`, but numbered continuously across the whole schedule (e.g. step 120 is the 20th attempt of the 3rd temperature stage if `ntries=100`), so a snapshot can be recovered after any number of annealing steps, not just the final/best configuration
- any other keyword accepted by `lg.optimize_energy()` (e.g. `patience`, `resync_every`), applied at every temperature

Overwrites `lg.den` with the best configuration found and returns the concatenated energy trajectory across all temperatures

lg.optimize_energy_multistart()

Run `nstart` independent anneals from independent random seeds at the current filling, and keep the lowest-energy result – reduces the risk of a single anneal settling into a metastable configuration. Each restart is a full `optimize_discrete` run, farmed out with `parallel.pcall`; whether that runs in parallel depends on `parallel.set_cores()`, same as every other `pcall` call site in this package (serial by default).

Optional arguments:

- `nstart=10`: number of independent restarts
- any other keyword accepted by `lg.optimize_energy()` (e.g. `temp`, `ntries`, `patience`), applied identically to every restart

Overwrites `lg.den` with the best configuration found and returns its energy (a scalar)

lg.optimize_grand_canonical()

Grand-canonical Metropolis sampling/annealing: instead of swapping pairs at fixed filling, single sites are flipped (occupied $\leftrightarrow$ empty) and accepted/rejected the usual Metropolis way, so the total filling fluctuates under `lg.mu` rather than being conserved. This is the standard lattice-gas MC move set, useful for scanning a phase diagram vs. chemical potential, or for equilibrium sampling at one fixed temperature (see `latticegas.get_specific_heat()/get_susceptibility()` below). Unlike `lg.optimize_energy()`, `lg.den` does not need 2 distinct starting values – it can start uniformly empty or full.

Optional arguments: same as `lg.optimize_energy()` (`temp`, `ntries`, `resync_every`; no `patience`), except that `temp` defaults to 1.0 here rather than 0.1 – grand-canonical sampling is usually wanted at a temperature, not as an anneal

Overwrites `lg.den` with the final configuration and returns (`es`, `ns`): the energy trajectory and the filling (occupied-site count) trajectory, both arrays of length `ntries`

latticegas.get_specific_heat() / latticegas.get_susceptibility()

Module-level (not `lg.`-prefixed) post-processing functions that estimate equilibrium thermodynamic quantities from a trajectory sampled at one *fixed* temperature (e.g. from `lg.optimize_energy()` or `lg.optimize_grand_canonical()` called with a constant `temp`, not annealed) – `get_specific_heat(es, temp, burn=0.2)` returns $C = \text{Var}(E)/T^2$ from an energy trajectory, and `get_susceptibility(ns, temp, burn=0.2)` returns the particle-number susceptibility $dN/d\mu = \text{Var}(N)/T$ from a filling trajectory (only meaningful for the grand-canonical `ns`, since `lg.optimize_energy()`'s filling is constant

by construction). `burn` discards that leading fraction of the trajectory as equilibration before computing the variance.

lg.add_tensor()

Add a custom coupling $J_{ij} = \text{fun}(r_i, r_j)$ between every pair of sites, for interactions beyond `add_interaction()`'s fixed neighbor shells – e.g. a screened or dipolar $1/r^n$ form. Scalar analog of `classicalspin.SpinModel.add_tensor` (which returns a 3x3 tensor per pair); self-pairs are skipped, and pairs where `fun` evaluates to (near) zero are dropped.

Mutates `lg` in place, no return value

lg.regroup()

Merge duplicate interaction-pair entries accumulated from repeated `add_interaction()/add_tensor()` calls (e.g. overlapping neighbor shells added twice), summing their couplings – pure performance cleanup, doesn't change `lg.get_energy()`.

Mutates `lg` in place, no return value

lg.write() / lg.read()

Save/load the current occupation snapshot `lg.den` to/from a text file, reusing `Geometry.write_profile()` – the same checkpoint pattern as `classicalspin.SpinModel.write()/load_magnetism()`. `write()` forces `nrep=1` (no periodic replication) by default so `read()` round-trips exactly regardless of `lg.geometry.dimensionality`; `read()` raises `ValueError` if the file's site count doesn't match `lg.nsites`.

Optional arguments:

- `name="DENSITY.OUT"`: file path

lg.get_local_energy() / lg.get_local_mu()

Per-site breakdown of the current snapshot's energy. `get_local_energy()` returns each site's own contribution $\mu_i n_i + \sum_j J_{ij} n_i n_j$, summed over its interaction neighbors j (`lg.get_energy()` itself counts every bond twice, once from each endpoint, so the values here sum exactly to `lg.get_energy()`, not to half of it); `get_local_mu()` instead evaluates that same expression with site `i` forced occupied, i.e. the energy cost/gain of occupying site `i` given its neighbors' current state.

Optional arguments:

- `normalize=False`: divide each site's value by the total coupling weight of its own interaction terms

Returns an array over sites

lg.get_correlator()

Neighbor-shell density-density correlator of the current snapshot `lg.den`, useful for detecting ordered ground states (e.g. after `optimize_energy`). Thin wrapper around `statphystk.correlator.get_nnc`; see its docstring for `n/normalized` options.

Returns (`distances`, `correlators`), arrays of matching length

lg.get_structure_factor()

Reciprocal-space structure factor $S(q) = |\sum_i (n_i - \bar{n}) e^{-iq \cdot r_i}|^2 / N$ of the current snapshot `lg.den`, evaluated directly on the real-space site positions – the reciprocal-space companion to `lg.get_correlator()`: where the neighbor-shell correlator tells you the ordering length scale, $S(q)$ tells you the ordering wavevector. Subtracting the mean occupation makes $S(q = 0) = 0$ identically, so a peak elsewhere in q is what signals order. Thin wrapper around `statphystk.correlator.get_structure_factor`.

Optional arguments:

- `qpath=None`: explicit array of q vectors to evaluate; if omitted, a default square grid of `nq`×`nq` points spanning $\pm q_{\max}$ is used
- `nq=60`: grid resolution when `qpath` is not given
- `qmax=None`: half-width of the default grid; defaults to $2\pi/d$ set by the nearest-neighbor spacing d

Returns (`qpath`, `sq`): the array of q vectors evaluated and the matching array of $S(q)$ values

LatticeIsing functions and methods

li.add_interaction()

Add a coupling shell to the model, on top of any interactions already added (repeated calls accumulate).

Optional arguments:

- `Jij`: list of shell couplings, e.g. [`J1`,`J2`,`J3`] for first/second/third neighbor $-J_{ij}s_i s_j$ exchange; positive is ferromagnetic (opposite sign convention from `LatticeGas.add_interaction()`). Passed through to `Geometry.get_hamiltonian(tij=Jij)`, so anything that constructor accepts for `tij` works here too

Mutates `li` in place, no return value

li.add_field()

Add an external (Zeeman-like) field to `li.b`.

Arguments:

- `h`: a scalar (applied uniformly to every site) or a per-site array

Mutates `li` in place, no return value

li.set_magnetization()

Reset the spin array `li.s` to a new random ± 1 configuration with a given average magnetization, discarding the current snapshot.

Arguments:

- `m=0.0`: target average magnetization in $[-1, 1]$ (rounded to the nearest integer up-spin count)

li.get_energy()

Evaluate the total energy $-\sum_i b_i s_i - \sum_{ij} J_{ij} s_i s_j$ of the current spin snapshot `li.s`. Returns a scalar. Since `li.pairs` lists both directions of every bond, this is twice the usual sum-over-unordered-bonds convention.

li.get_magnetization()

Return `mean(s)`, the average magnetization per site of `li.s` (a scalar in $[-1, 1]$).

li.optimize_energy()

Single-spin-flip Metropolis dynamics – the standard Ising Monte Carlo move set: at each step, one random site is flipped and accepted unconditionally if the energy doesn't increase, or with probability $e^{-\Delta E/T}$ otherwise. Magnetization is *not* conserved (it fluctuates under `li.b`), the spin analog of `LatticeGas.optimize_grand_canonical()`.

Optional arguments:

- `temp=1.0`: Metropolis temperature; `temp=0` runs zero-temperature (greedy) dynamics
- `ntries=1e5`: number of flip attempts
- `resync_every=1000`: how often (in flip attempts) to recompute the energy from scratch, bounding floating-point drift in the incremental tracking
- `checkpoint_at=None`: an int or iterable of ints; captures a copy of `li.s` after that many attempts (1-indexed) into `li.checkpoints` (a dict `step` -> `s` snapshot)

No `patience` option: unlike `optimize_conserved()`, this trajectory is meant to be fed to `latticegas.get_specific_heat()/get_susceptibility()`, and early truncation would silently bias those variance estimates.

Overwrites `li.s` with the final configuration and returns `(es, ms)`: the energy trajectory and the total-magnetization ($\sum_i s_i$) trajectory, both arrays of length `ntries`

`li.optimize_conserved()`

Kawasaki spin-exchange dynamics: at each step, one up spin and one down spin are picked at random and swapped, which conserves the total magnetization – the spin analog of `LatticeGas.optimize_energy()` (swap-based, fixed filling). Raises `ValueError` if `li.s` doesn't currently have both `+1` and `-1` present (e.g. after `set_magnetization(1.0)`).

Optional arguments: same as `li.optimize_energy()`, except that `temp` defaults to `0.1` here rather than `1.0`, plus:

- `patience=None`: if set, stop early once this many attempts have passed without a new best energy being found (the returned array is truncated to what actually ran)

Overwrites `li.s` with the final configuration and returns the array of energies recorded at each attempt

`li.anneal()`

Simulated annealing over a decreasing temperature schedule: calls `li.optimize_energy()` once per temperature in `temps`, keeping the best (lowest-energy) configuration seen across the whole schedule.

Optional arguments:

- `temps=None`: sequence of temperatures, high to low; defaults to a 10-step geometric schedule from `2.0` down to `0.05`
- `ntries=1e4`: number of flip attempts per temperature
- `checkpoint_at=None`: like `li.optimize_energy()`'s `checkpoint_at`, but numbered continuously across the whole schedule
- any other keyword accepted by `li.optimize_energy()`, applied at every temperature

Overwrites `li.s` with the best configuration found and returns `(es, ms)`, the concatenated energy and magnetization trajectories across all temperatures

`li.optimize_energy_multistart()`

Run `nstart` independent flip-based anneals from independent random seeds (same initial magnetization as the current `li.s`) and keep the lowest-

energy result. Each restart is a full `optimize_ising` run, farmed out with `parallel.pcall`, mirroring `LatticeGas.optimize_energy_multistart()`.

Optional arguments:

- `nstart=10`: number of independent restarts
- any other keyword accepted by `li.optimize_energy()` (e.g. `temp`, `ntries`), applied identically to every restart

Overwrites `li.s` with the best configuration found and returns its energy (a scalar)

`li.get_local_energy()` / `li.get_local_field()`

Per-site breakdown of the current snapshot. `get_local_energy()` returns each site's own contribution to `li.get_energy()` (values sum exactly to `li.get_energy()`, mirroring `LatticeGas.get_local_energy()`'s $j/2$ correction for the double-counted bonds). `get_local_field()` returns the effective field $h_i^{\text{eff}} = b_i + 2 \sum_k J_{ik} s_k$ seen by each site, defined so that flipping s_i costs exactly $2s_i h_i^{\text{eff}}$ – the spin analog of `LatticeGas.get_local_mu()`.

Returns an array over sites

`li.get_correlator()` / `li.get_structure_factor()`

Spin-spin correlator and reciprocal-space structure factor of the current snapshot `li.s`, reusing `statphystk.correlator.get_nnc/get_structure_factor` directly (both operate on any per-site array, not just 0/1 occupations) – see `LatticeGas.get_correlator()/get_structure_factor()` for the argument reference.

`li.add_tensor()` / `li.regroup()`

Same as `LatticeGas.add_tensor()/regroup()`, reusing those functions directly (they only touch `geometry.r/nsites/pairs/j`, none of which differ in meaning between the two models).

Mutates `li` in place, no return value

`li.write()` / `li.read()`

Save/load the current spin snapshot `li.s` to/from a text file, reusing `Geometry.write_profile()` – see `LatticeGas.write()/read()`. `read()` rounds to $\{-1, +1\}$ (sign, treating exact 0 as +1) to absorb the text round-trip's floating point noise, and raises `ValueError` if the file's site count doesn't match `li.nsites`.

Optional arguments:

- `name="SPIN.OUT"`: file path